KUBERNETES
ERROR ENCYCLOPEDIA

150 Common Errors, Root Cause Analysis & Solutions

Covering Pod, Node, Networking, Storage, Control Plane, Workload & Security Errors

150	7	150	RCA	K8s
Errors Covered	Categories	Diagrams	Included	v1.29+

Produced by DevOpsShack

devopsshack.com | Kubernetes Training & Resources | 2026

Table of Contents

How to Use This Document

1. Each error entry includes: Error Number, Name, Severity badge, Description, Root Cause Analysis, and Step-by-step Solution.
2. Diagrams show the error flow — from originating component through intermediate failure point to the resulting error state.
3. Severity levels: CRITICAL (immediate cluster impact), HIGH (significant service disruption), MEDIUM (degraded functionality), LOW (minor issues).
4. Solutions are tested against Kubernetes 1.27–1.29 but most apply to 1.24+.
5. kubectl commands assume you are authenticated to the relevant cluster and namespace.

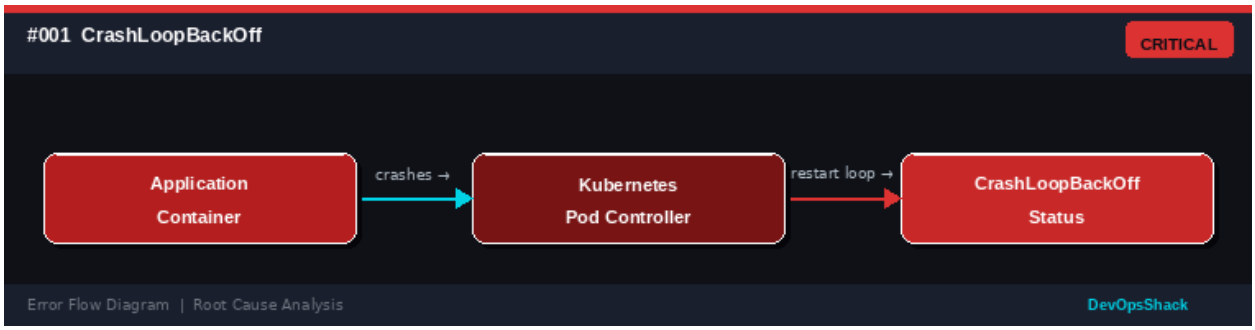
Pod Lifecycle Errors

Errors #001 – #030

These errors occur during the lifecycle of Kubernetes pods — from creation and scheduling through initialization, startup, and runtime. Understanding pod lifecycle errors is fundamental to operating any Kubernetes workload.

#001	CrashLoopBackOff *Pod Lifecycle*	CRITICAL
DESCRIPTION	The container repeatedly crashes after startup and Kubernetes keeps restarting it with exponential backoff delays. You will see the restart count increasing in kubectl get pods output.	
ROOT CAUSE	The application inside the container exits with a non-zero exit code immediately after starting. Common causes: missing environment variables or secrets, misconfigured command/args, application bug or unhandled exception, missing dependency services, and insufficient memory causing immediate OOM kill.	
SOLUTION	Check container logs with kubectl logs <pod> --previous. Verify all required environment variables and secrets exist. Test the container image locally with docker run. Check resource limits are not too restrictive. Fix application startup errors and rebuild the image. Consider adding a startup probe to give the app more time.	

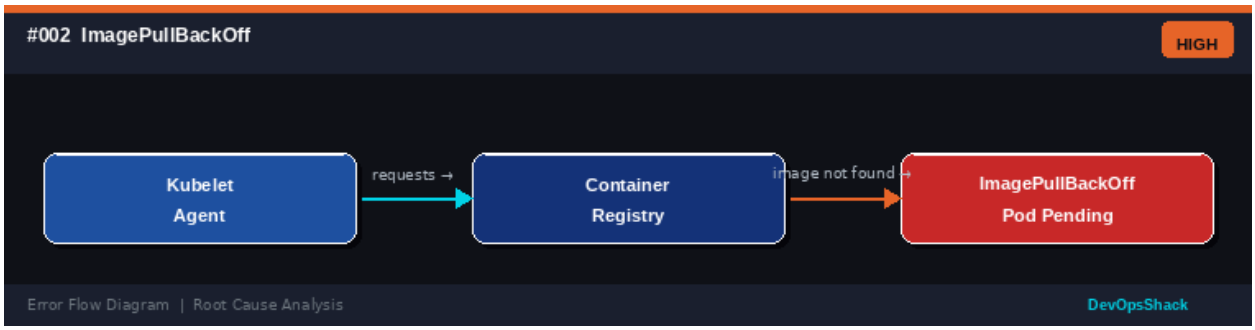
Error Flow Diagram



#002	ImagePullBackOff *Pod Lifecycle*	HIGH
DESCRIPTION	Kubernetes cannot pull the container image from the registry. The pod remains in Pending state and the image pull is retried with increasing backoff intervals.	
ROOT CAUSE	The container image cannot be retrieved. Common causes: incorrect image name or tag, image does not exist in the registry, registry authentication credentials are missing or expired, private registry not configured in the cluster, and network connectivity issues to the registry.	
SOLUTION	Verify the image name and tag are correct. For private registries, create an imagePullSecret: kubectl create secret docker-registry regcred --docker-server=<server> --docker-username=<user> --docker-password=<pass>.	

Reference it in the pod spec under imagePullSecrets. Check network connectivity to the registry from within the cluster.

Error Flow Diagram



#003 ErrImagePull *Pod Lifecycle* HIGH

DESCRIPTION

The immediate (non-backoff) form of image pull failure. Kubernetes attempted to pull the image and received an error. This transitions to ImagePullBackOff after multiple failures.

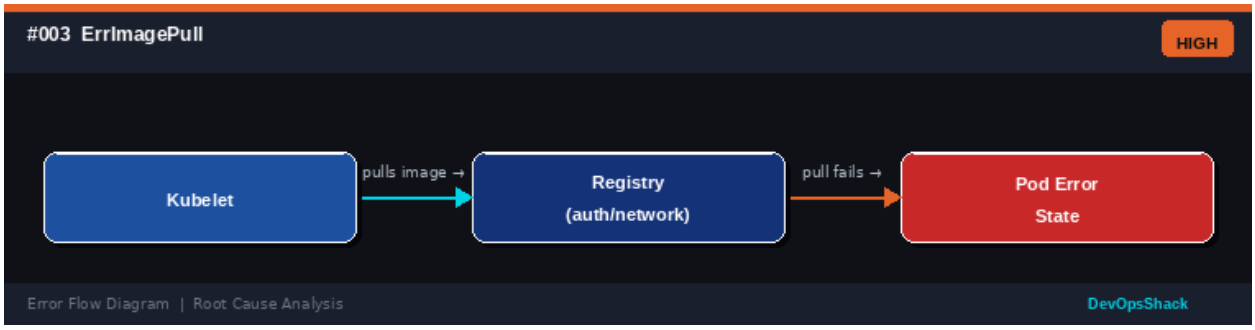
ROOT CAUSE

One-time image pull attempt failed. Causes include: wrong credentials, image not found at the specified tag, registry temporarily unavailable, DNS resolution failure for the registry hostname, and TLS certificate issues with private registries.

SOLUTION

Run kubectl describe pod <pod-name> to see the exact error message. Verify image exists: docker pull <image>. Check imagePullSecrets are attached to the pod's ServiceAccount or directly in the pod spec. Ensure the registry is reachable from nodes. For self-signed certs, configure the container runtime to trust the CA.

Error Flow Diagram



#004 OOMKilled *Pod Lifecycle* CRITICAL

DESCRIPTION

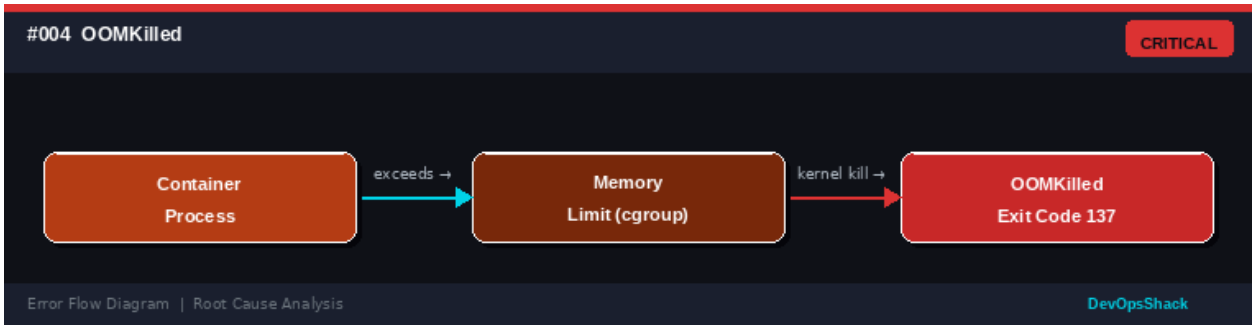
The container was killed by the Linux kernel out-of-memory (OOM) killer because it exceeded its memory limit. The pod shows Exit Code 137 and OOMKilled as the reason.

ROOT CAUSE

The process inside the container used more memory than the resource limit specified in the pod spec. This triggers the Linux kernel's OOM killer which terminates the highest-priority offender. Causes: memory leak in application,

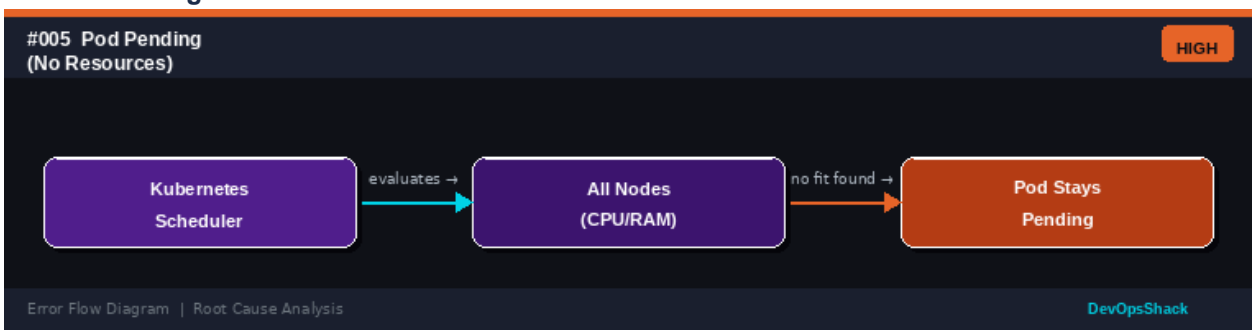
	under-estimated memory requirements, sudden traffic spike causing memory spike, and JVM heap misconfiguration.
SOLUTION	Increase the container memory limit in the pod spec: resources.limits.memory. Use kubectl top pods to observe actual memory usage over time. Fix memory leaks in the application. For JVM apps, set -Xmx below the container limit (leave ~25% headroom). Set up VPA (Vertical Pod Autoscaler) to auto-tune limits based on historical usage.

Error Flow Diagram



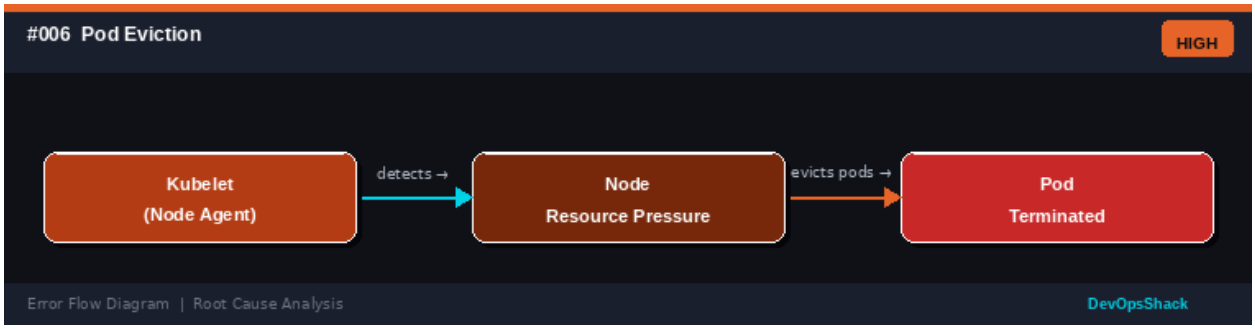
#005	Pod Pending - No Resources *Pod Lifecycle*	HIGH
DESCRIPTION	The pod has been created but remains in Pending state because the scheduler cannot find a suitable node with enough CPU or memory to run it.	
ROOT CAUSE	No node in the cluster has sufficient allocatable resources to satisfy the pod's resource requests. Allocatable resources = Node capacity minus system/kubelet reserved resources minus already scheduled pod requests. The scheduler uses requests (not limits) for placement decisions.	
SOLUTION	Check node resource availability: kubectl describe nodes | grep -A5 Allocated. Reduce resource requests if they are over-specified. Add more nodes to the cluster or use the Cluster Autoscaler. Check if resource quotas in the namespace are blocking scheduling. Remove completed or failed pods consuming reserved resources. Consider using a smaller pod with horizontal scaling instead.	

Error Flow Diagram



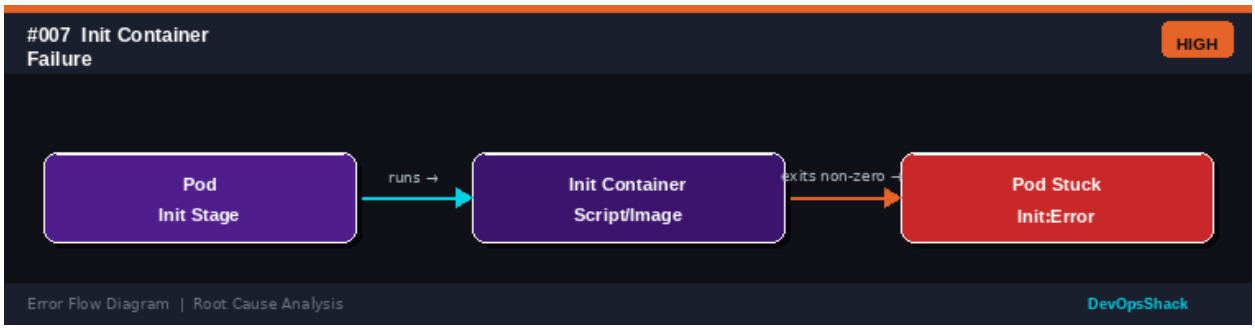
#006	Pod Eviction *Pod Lifecycle*	HIGH
DESCRIPTION	One or more pods have been forcibly terminated (evicted) by the kubelet due to resource pressure on the node. Evicted pods show a Failed phase with reason Evicted.	
ROOT CAUSE	The node is experiencing resource pressure (DiskPressure, MemoryPressure, or PIDPressure). The kubelet evicts pods to reclaim resources, starting with BestEffort QoS pods, then Burstable, and finally Guaranteed. Eviction can also be triggered by taints added by node problem detector.	
SOLUTION	Identify the eviction reason: kubectl describe pod <evicted-pod>. For disk pressure, clean up unused images (crictl rmi --prune) and logs. For memory pressure, increase node memory or reduce pod requests. Set appropriate resource limits and requests on all pods to ensure correct QoS class. Configure eviction thresholds in kubelet config. Use PodDisruptionBudgets to limit voluntary disruptions.	

Error Flow Diagram



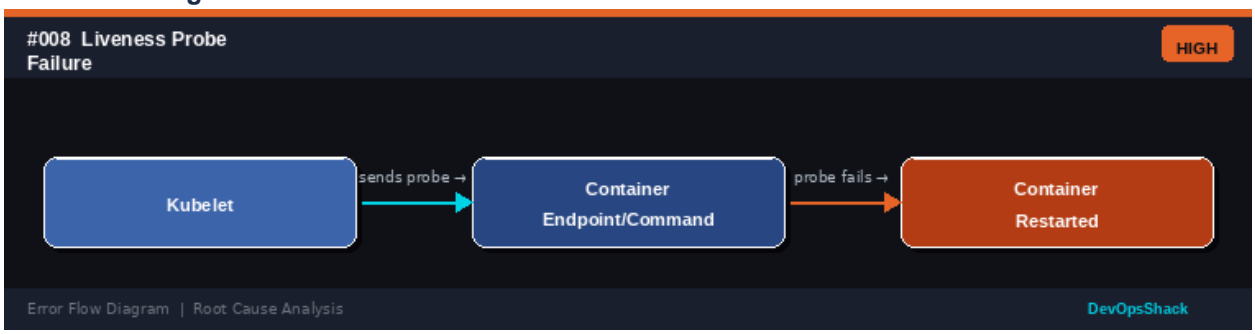
#007	Init Container Failure *Pod Lifecycle*	HIGH
DESCRIPTION	One or more init containers have failed, preventing the main application container from starting. The pod shows Init:Error or Init:CrashLoopBackOff status.	
ROOT CAUSE	Init containers run sequentially before the main container and must all complete successfully. Failures occur due to: script/command returning non-zero exit code, missing dependencies or services (e.g., database not ready), incorrect image, insufficient permissions, and network connectivity issues during initialization.	
SOLUTION	Check init container logs: kubectl logs <pod> -c <init-container-name>. Fix the init container script or command. Use retry logic in init containers for dependent services. Verify the init container has the necessary permissions and secrets. Consider using readiness checks instead of init containers for service dependencies. Ensure the init container image exists and is accessible.	

Error Flow Diagram



#008	Liveness Probe Failure *Pod Lifecycle*	HIGH
DESCRIPTION	The liveness probe is failing repeatedly, causing Kubernetes to restart the container. The pod may show many restarts in kubectl get pods output.	
ROOT CAUSE	The container is running but the liveness probe endpoint is not responding successfully. Causes: the application is deadlocked or in an unhealthy state, the probe is misconfigured with wrong port/path, the probe timeout is too short, the application is temporarily busy (initializing), and firewall or NetworkPolicy blocking probe traffic.	
SOLUTION	Review probe configuration: correct path, port, and thresholds. Increase initialDelaySeconds to give the app time to start. Increase timeoutSeconds and failureThreshold for slow applications. Check kubectl describe pod for probe failure details. Ensure the health endpoint returns HTTP 200. Add proper health check logic to your application. Verify no NetworkPolicy blocks kubelet probe traffic.	

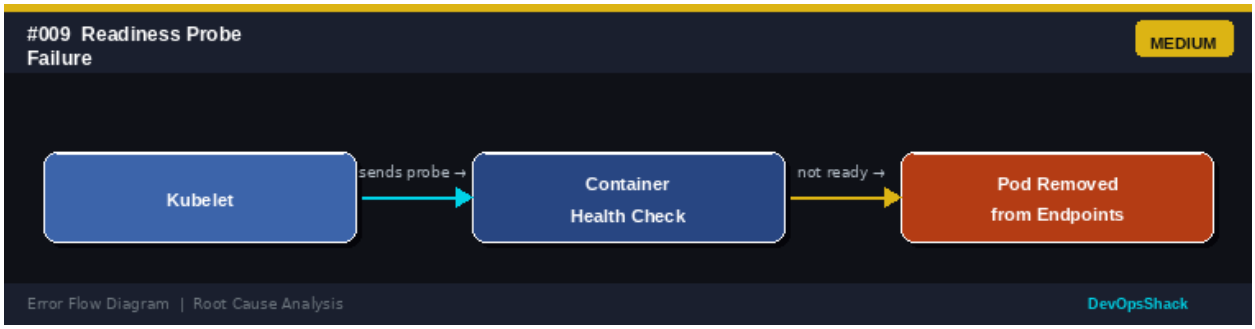
Error Flow Diagram



#009	Readiness Probe Failure *Pod Lifecycle*	MEDIUM
DESCRIPTION	The readiness probe is failing, so the pod is not added to Service endpoints. Traffic is not routed to this pod even though it is running.	
ROOT CAUSE	The container is running but not yet ready to serve traffic. Causes: application still initializing, dependent service is unavailable, readiness endpoint returning non-2xx response, wrong port or path in probe configuration, and application processing a long-running operation.	
SOLUTION	This is often normal during startup - set appropriate initialDelaySeconds. Check the readiness probe path returns 200 only when truly ready. Verify the application	

	properly signals readiness. Check downstream dependencies. Use kubectl describe pod to see probe failure messages. Unlike liveness probes, readiness failure does not restart the container - it only removes it from load balancing.

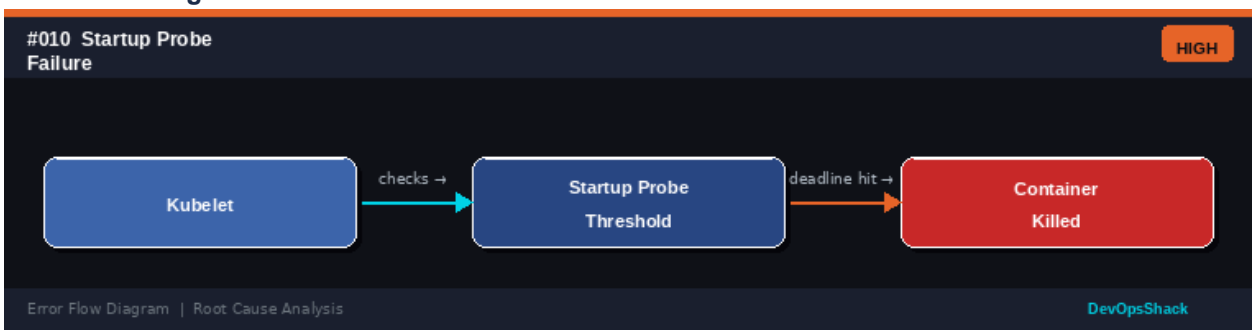
Error Flow Diagram



#010 Startup Probe Failure *Pod Lifecycle* — HIGH

DESCRIPTION	The startup probe did not succeed within the configured deadline (failureThreshold * periodSeconds), causing the container to be killed before it had a chance to fully start.
ROOT CAUSE	Applications with long startup times (e.g., JVM apps, databases) may take longer than the default probe configuration allows. Without a startup probe, liveness probes would kill the container before it finishes starting. The startup probe acts as a one-time gate before liveness probes activate.
SOLUTION	Increase failureThreshold and periodSeconds to provide sufficient startup time: failureThreshold: 30 with periodSeconds: 10 gives 5 minutes. Use the same endpoint as your liveness probe. Once startup probe succeeds, it stops running and liveness/readiness probes take over. Ensure the startup endpoint is the first thing your application initializes.

Error Flow Diagram

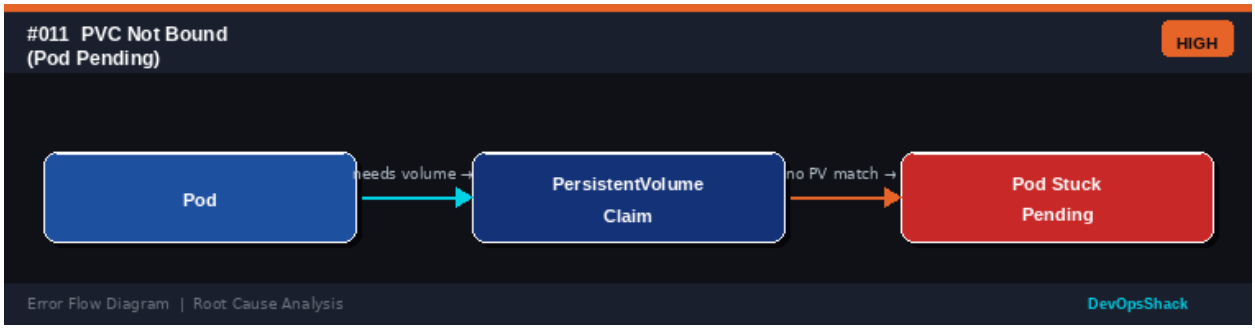


#011 PVC Not Bound - Pod Pending *Pod Lifecycle* — HIGH

DESCRIPTION	A pod is stuck in Pending state because one or more PersistentVolumeClaims (PVCs) it references have not been successfully bound to a PersistentVolume.
ROOT CAUSE	PVCs need a matching PersistentVolume to bind to. Binding fails when: no PV with sufficient capacity exists, PV and PVC storage class do not match, PV and

	PVC access modes are incompatible, dynamic provisioner is not configured or has failed, and the referenced StorageClass does not exist.
SOLUTION	Check PVC status: kubectl get pvc -n <namespace>. If static provisioning, create a matching PV. If dynamic, ensure the StorageClass provisioner is running. Verify StorageClass exists: kubectl get storageclass. Check provisioner logs for errors. Ensure PVC size does not exceed available PV size. Verify access modes match between PVC and PV.

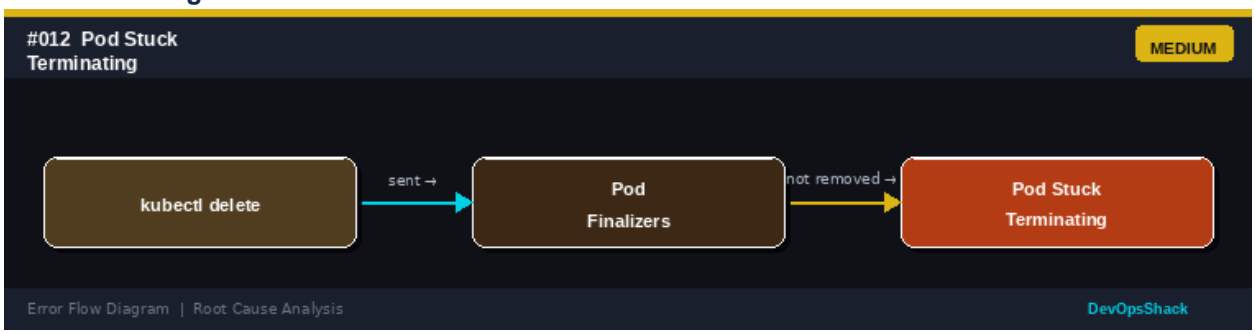
Error Flow Diagram



#012 Pod Stuck Terminating *Pod Lifecycle* MEDIUM

DESCRIPTION	A pod has been deleted but remains in Terminating state indefinitely and does not complete deletion. The pod continues to appear in kubectl get pods.
ROOT CAUSE	Pods get stuck terminating when finalizers are not removed from the pod. This can happen when: a custom controller or operator adds a finalizer but crashes, a volume fails to unmount cleanly, the container runtime fails to stop the process, and network policies or hooks prevent graceful shutdown.
SOLUTION	First check pod events and finalizers: kubectl describe pod <pod>. If a custom finalizer is stuck, manually patch to remove it: kubectl patch pod <pod> -p '{"metadata":{"finalizers":null}}'. Force delete as last resort: kubectl delete pod <pod> --force --grace-period=0. Fix the controller adding the finalizer if it is not cleaning up properly.

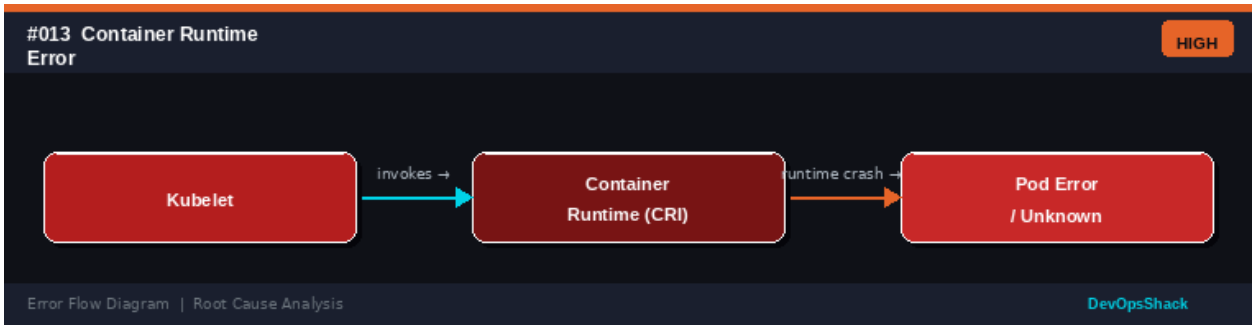
Error Flow Diagram



#013 Container Runtime Error *Pod Lifecycle* HIGH

DESCRIPTION	The container runtime (containerd or CRI-O) encountered an error and cannot create, start, or manage containers on the node. Pods show ContainerCannotRun or similar errors.
ROOT CAUSE	The Container Runtime Interface (CRI) on the node has failed. Causes: containerd or CRI-O process has crashed, runtime socket is inaccessible, incompatible runtime version, storage driver issues, and corrupted container layer data.
SOLUTION	SSH to the affected node and check the runtime: systemctl status containerd. Restart the runtime: sudo systemctl restart containerd. Check runtime logs: journalctl -u containerd. Clean up corrupt state: crictl ps and crictl rm for stuck containers. Verify the kubelet CRI socket path matches the actual runtime socket path in kubelet config.

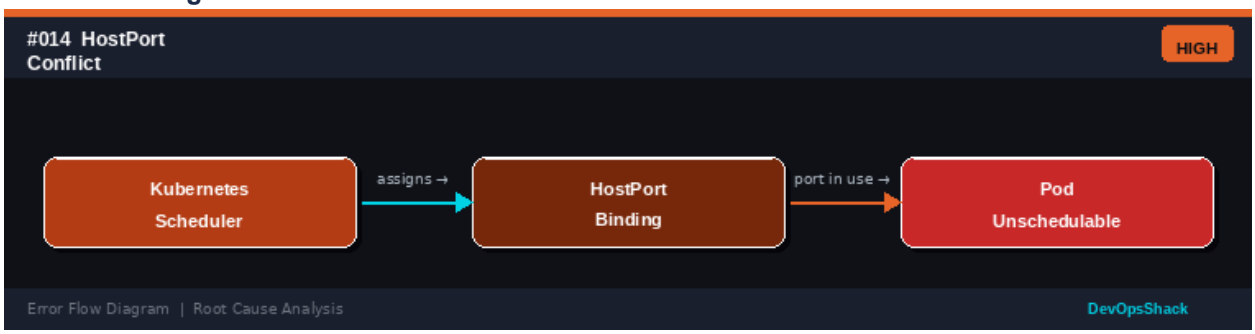
Error Flow Diagram



#014 HostPort Conflict *Pod Lifecycle* HIGH

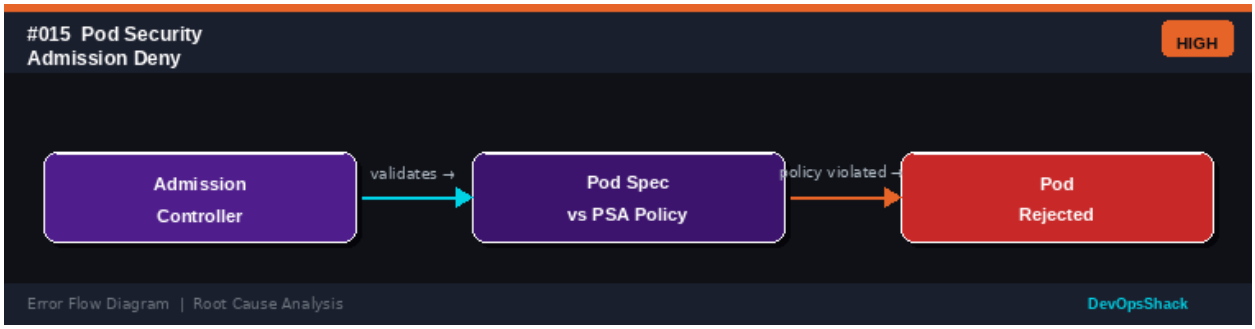
DESCRIPTION	A pod cannot be scheduled because it requests a specific HostPort that is already in use on all available nodes in the cluster.
ROOT CAUSE	When a pod uses hostPort, it binds directly to a port on the node's network interface. Only one pod can use a given hostPort per node. If all nodes already have that port occupied, the scheduler cannot place the pod anywhere.
SOLUTION	Avoid using hostPort when possible - use Services or Ingress instead. If hostPort is required, ensure pods are spread across nodes using PodAntiAffinity. Review existing pods using the same hostPort: kubectl get pods -o json | grep hostPort. Use NodePort services as an alternative to hostPort for external access.

Error Flow Diagram



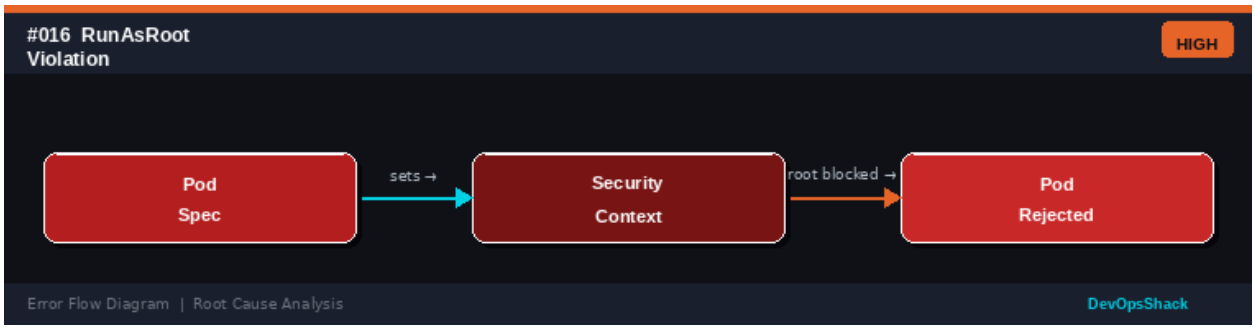
#015	Pod Security Admission Deny *Pod Lifecycle*	HIGH
DESCRIPTION	A pod creation was rejected by the Pod Security Admission (PSA) controller because its security configuration violates the namespace's security policy level (restricted, baseline, or privileged).	
ROOT CAUSE	Kubernetes enforces Pod Security Standards at the namespace level. The restricted profile prohibits running as root, using hostPath, privileged mode, and requires read-only root filesystem. Baseline prevents dangerous capabilities. Many existing workloads violate these without explicit configuration.	
SOLUTION	Check the namespace security label: kubectl get namespace <ns> --show-labels. Review what the error blocks: kubectl describe pod for the rejection message. Update the pod spec to comply: set runAsNonRoot: true, drop capabilities, avoid hostPath. Use the baseline profile for less strict enforcement. Label namespace with appropriate level: kubectl label ns <ns> pod-security.kubernetes.io/enforce=baseline.	

Error Flow Diagram



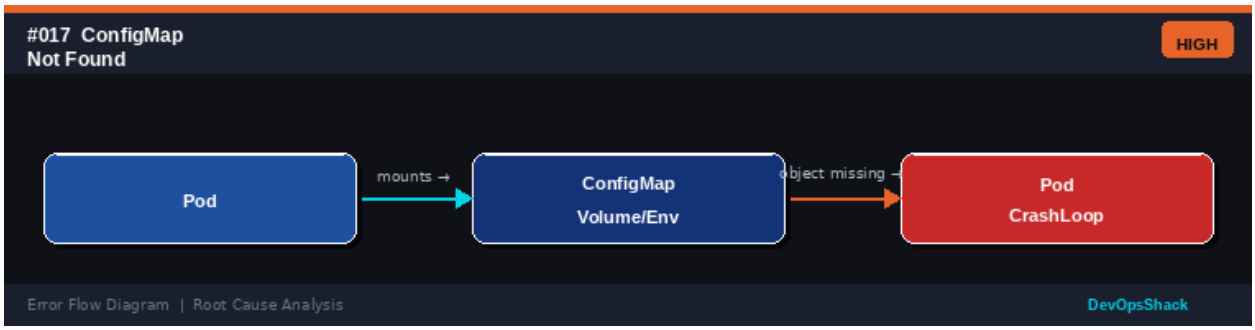
#016	RunAsRoot Violation *Pod Lifecycle*	HIGH
DESCRIPTION	The pod or container is configured to run as root (UID 0) but the namespace or cluster policy prohibits running containers as the root user.	
ROOT CAUSE	Running containers as root is a security risk. Pod Security Admission with restricted or baseline policy, OPA/Gatekeeper constraints, or PodSecurityPolicy (deprecated) can block root containers. The check applies when runAsNonRoot: true is set or enforced by policy.	
SOLUTION	Set runAsUser to a non-zero UID in the securityContext: runAsUser: 1000, runAsNonRoot: true. Rebuild the container image to use a non-root user (USER 1000 in Dockerfile). Ensure the application does not require root privileges. If root is truly required, use a privileged namespace with appropriate justification and controls.	

Error Flow Diagram



#017	ConfigMap Not Found *Pod Lifecycle*	HIGH
DESCRIPTION	The pod fails to start because it references a ConfigMap (as a volume or environment variable source) that does not exist in the namespace.	
ROOT CAUSE	Pod specs can reference ConfigMaps using envFrom, env.valueFrom.configMapKeyRef, or volumes.configMap. If the referenced ConfigMap is missing from the namespace, the pod will fail to start with a CreateContainerConfigError error.	
SOLUTION	Create the missing ConfigMap: kubectl create configmap <name> --from-literal=key=value. Verify ConfigMap exists in the correct namespace: kubectl get configmap -n <namespace>. Check pod spec for typos in ConfigMap name. Use optional: true in the ConfigMap reference if the ConfigMap may not always exist. Use Helm or GitOps to ensure ConfigMaps are deployed before pods.	

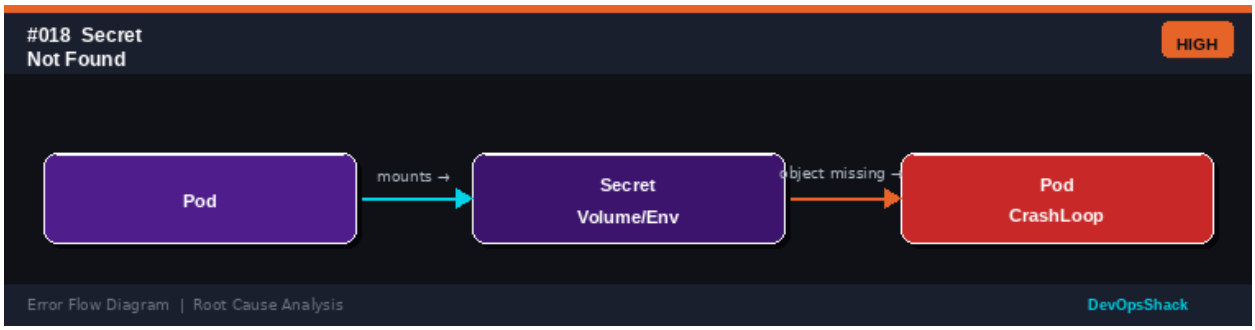
Error Flow Diagram



#018	Secret Not Found *Pod Lifecycle*	HIGH
DESCRIPTION	The pod fails to start because it references a Secret (as a volume, environment variable, or image pull secret) that does not exist in the namespace.	
ROOT CAUSE	Secrets are namespace-scoped. A pod referencing a Secret from another namespace or a Secret that was never created will fail. This is especially common when Secrets are managed externally (e.g., by an operator or CI/CD) and the pod is deployed before the Secret is available.	
SOLUTION	Create the missing Secret: kubectl create secret generic <name> --from-literal=key=value. Verify Secret exists: kubectl get secret -n <namespace>. Use External Secrets Operator or Sealed Secrets for GitOps-safe Secret	

	management. Add optional: true for non-critical Secrets. Ensure deployment ordering: use init containers or Helm hooks to wait for Secrets to be created.

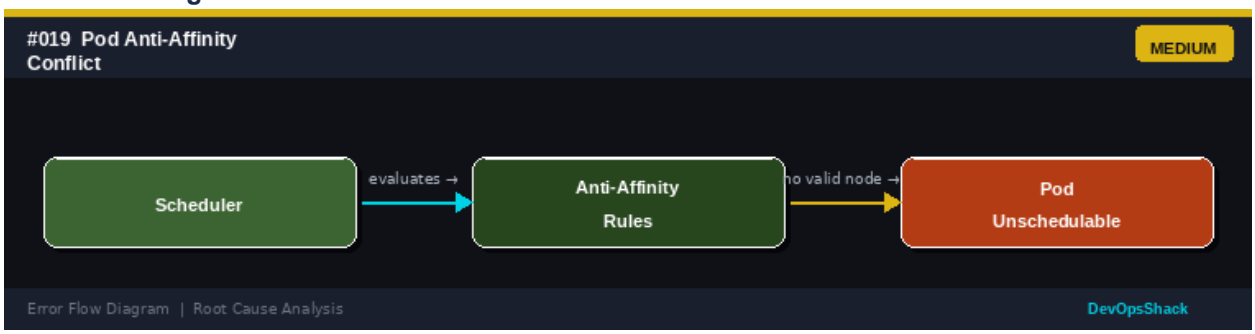
Error Flow Diagram



#019 Pod Anti-Affinity Conflict *Pod Lifecycle* — MEDIUM

DESCRIPTION	Pods cannot be scheduled because their anti-affinity rules prevent them from being placed on any available node (all nodes already host a pod matching the anti-affinity selector).
ROOT CAUSE	PodAntiAffinity rules prevent co-location of pods. With requiredDuringSchedulingIgnoredDuringExecution (hard rule), if every node already runs a matching pod, new pods are unschedulable. This commonly happens with single-node clusters or when the number of replicas exceeds the number of nodes.
SOLUTION	Use preferredDuringSchedulingIgnoredDuringExecution (soft rule) instead of required when strict separation is not mandatory. Reduce the number of replicas to match available nodes. Add more nodes to the cluster. Review the anti-affinity topology key - topologyKey: kubernetes.io/hostname is node-level. Consider using topologySpreadConstraints for more flexible distribution.

Error Flow Diagram

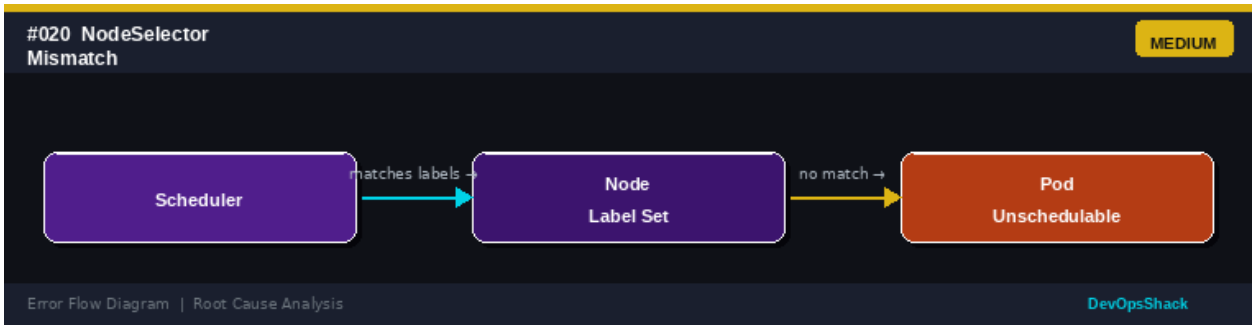


#020 NodeSelector Mismatch *Pod Lifecycle* — MEDIUM

DESCRIPTION	The pod has a nodeSelector that does not match any node labels in the cluster. The pod remains unschedulable.
ROOT CAUSE	nodeSelector constrains pods to nodes with specific labels. If no node carries the required label, the pod cannot be placed. This happens when node labels are

	removed, nodes are replaced without re-labeling, or nodeSelector contains a typo.
SOLUTION	Check available node labels: kubectl get nodes --show-labels. Verify the nodeSelector in the pod spec matches actual labels. Add the required label to a node: kubectl label node <node-name> <key>=<value>. Consider using nodeAffinity with preferred rules for more flexible scheduling. Check if a Node Feature Discovery (NFD) operator should be providing the labels automatically.

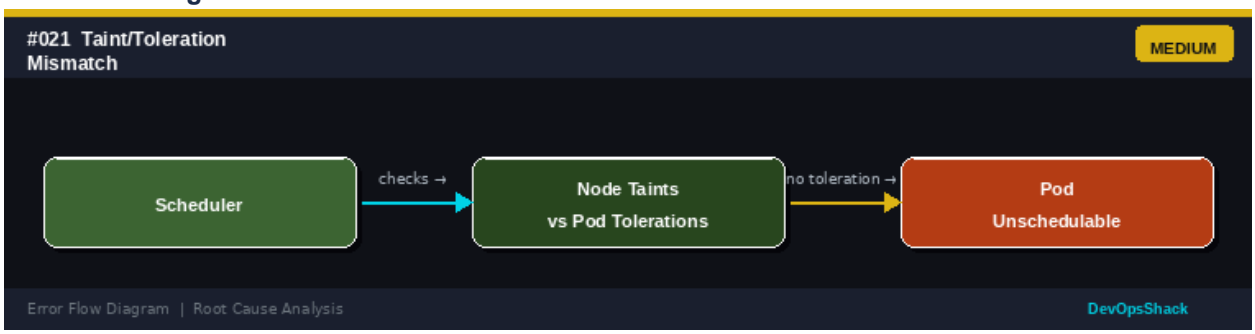
Error Flow Diagram



#021 Taint/Toleration Mismatch *Pod Lifecycle* — MEDIUM

DESCRIPTION	The pod is unschedulable because all nodes have taints that the pod does not tolerate. The scheduler cannot find a compatible node.
ROOT CAUSE	Node taints repel pods that do not have matching tolerations. Taints are used to mark nodes for special workloads (e.g., GPU nodes, control plane nodes, nodes under maintenance). If all nodes are tainted and the pod has no matching tolerations, it cannot be scheduled.
SOLUTION	Check node taints: kubectl describe nodes | grep Taint. Add the appropriate toleration to the pod spec. To run on all nodes including control plane: add toleration for node-role.kubernetes.io/control-plane:NoSchedule. For node maintenance taints: either add toleration or wait for maintenance to complete. Review and minimize unnecessary taints on nodes.

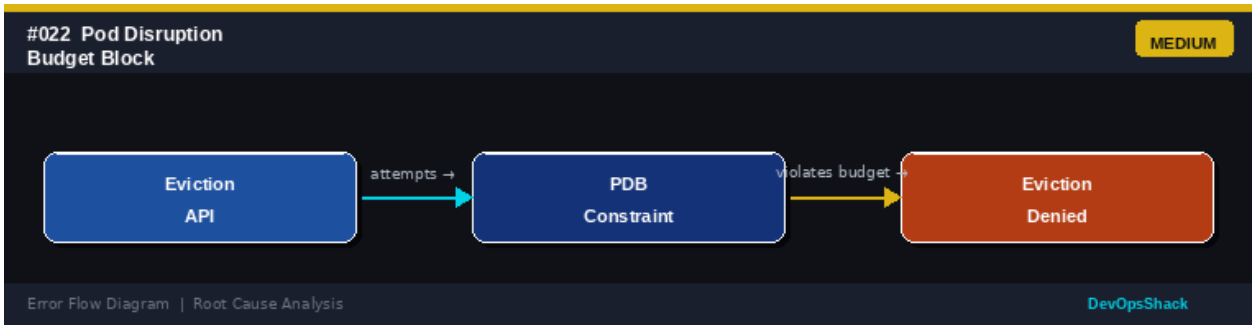
Error Flow Diagram



#022 Pod Disruption Budget Block *Pod Lifecycle* — MEDIUM

DESCRIPTION	A pod eviction request was denied because it would violate the PodDisruptionBudget (PDB), which defines the minimum number of pods that must remain available.
ROOT CAUSE	PDBs protect workloads from too many simultaneous disruptions. When kubectl drain, rolling updates, or manual deletions would bring available pod count below the PDB's minAvailable (or above maxUnavailable), the eviction is blocked. This is by design but can cause drain and update operations to hang.
SOLUTION	Check PDBs: kubectl get pdb -A. Check which pods are blocked: kubectl get pdb <name> -o yaml. If blocking is unintentional, review whether the PDB values are too strict. For cluster maintenance, temporarily delete the PDB if safe to do so. Fix underlying pod issues so more replicas become available, allowing the PDB to be satisfied. Ensure replicas > minAvailable.

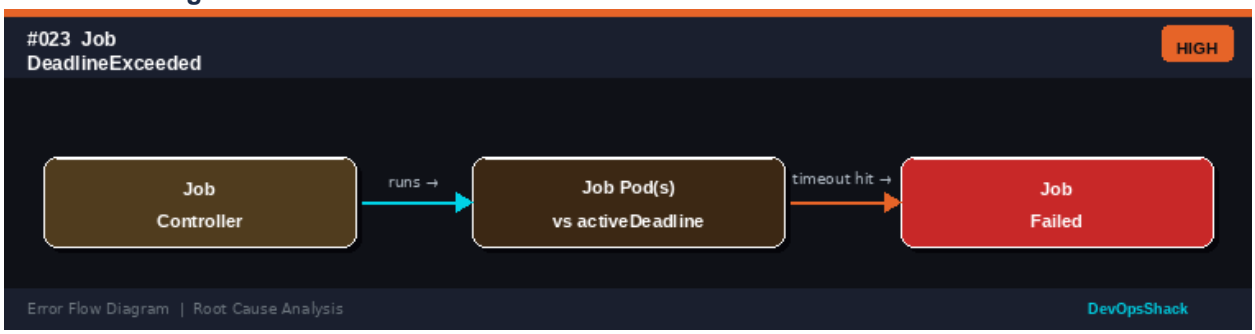
Error Flow Diagram



#023 Job DeadlineExceeded *Pod Lifecycle* HIGH

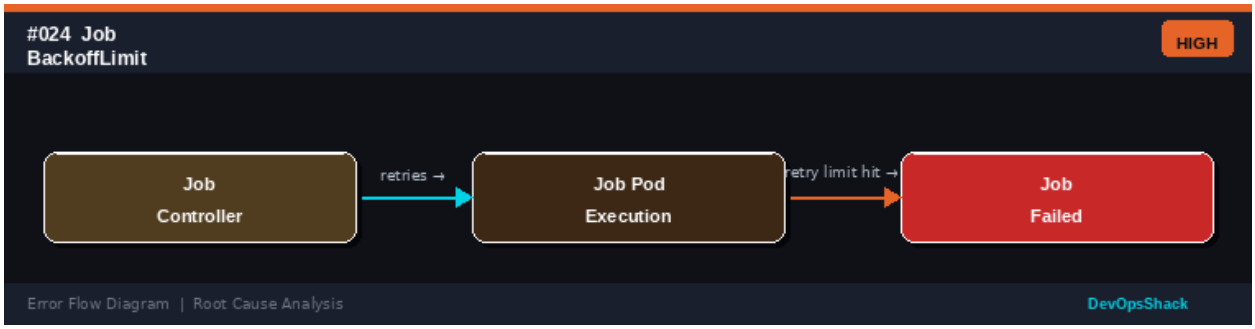
DESCRIPTION	A Kubernetes Job failed because it could not complete within the time specified by the activeDeadlineSeconds field.
ROOT CAUSE	When activeDeadlineSeconds is set on a Job, Kubernetes terminates all running pods and marks the Job as failed if completion is not reached within that time. Causes: job takes longer than expected, external service dependency is slow, pods keep failing and consuming retry time, and incorrect deadline estimation.
SOLUTION	Increase activeDeadlineSeconds to a realistic value for the workload. Optimize the job to complete faster. Check pod logs to understand what is causing slowness: kubectl logs <job-pod>. Set appropriate resource requests so pods are not CPU-throttled. If the job duration is variable, remove the deadline or make it very generous. Break large jobs into smaller batches.

Error Flow Diagram



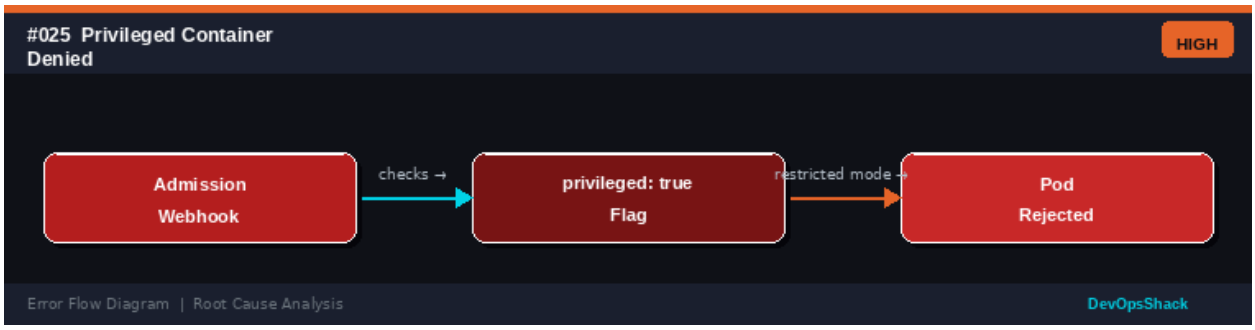
#024	Job BackoffLimit Exceeded *Pod Lifecycle*	HIGH
DESCRIPTION	A Kubernetes Job has exhausted its retry attempts (backoffLimit) because the pods keep failing. The Job is marked as Failed.	
ROOT CAUSE	The Job's pods are exiting with non-zero exit codes and the number of failures has reached backoffLimit (default 6). Each pod failure increments the failure counter. Root causes: application bug causing crash, missing configuration or secrets, dependent service unavailable, and incorrect command or arguments.	
SOLUTION	Check pod logs from failed attempts: kubectl logs <job-pod> --previous. Describe the job for events: kubectl describe job <name>. Fix the root cause of pod failure before re-running the job. Increase backoffLimit if failures are transient. Set appropriate restartPolicy (Never for debugging, OnFailure for transient errors). Use activeDeadlineSeconds alongside backoffLimit.	

Error Flow Diagram



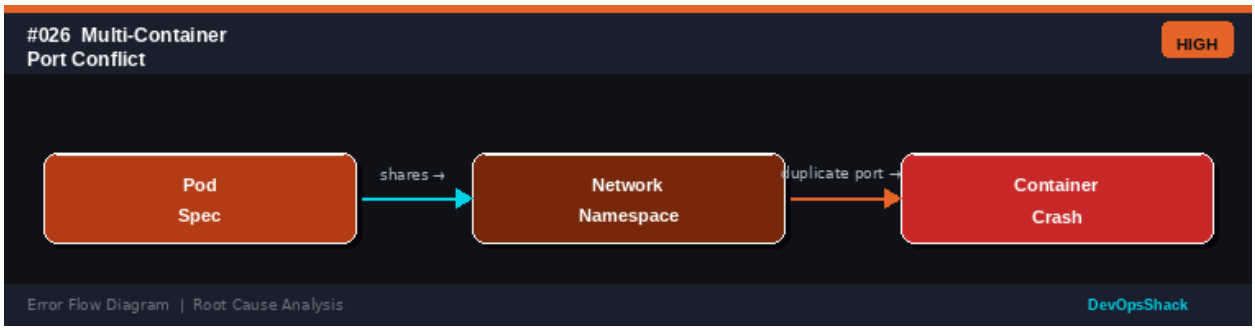
#025	Privileged Container Denied *Pod Lifecycle*	HIGH
DESCRIPTION	A pod requesting privileged: true in the security context was rejected by the admission controller. Privileged containers have near-root access to the host.	
ROOT CAUSE	Running privileged containers is a major security risk as they can break out of container isolation. Pod Security Admission with baseline or restricted policy, OPA/Gatekeeper, or PSP (deprecated) blocks privileged containers. This is enforced to prevent privilege escalation attacks.	
SOLUTION	Avoid privileged: true whenever possible. Use specific Linux capabilities instead (e.g., NET_ADMIN, SYS_PTRACE) via securityContext.capabilities.add. For system-level workloads (e.g., CNI, storage drivers), use a dedicated privileged namespace with appropriate controls. If truly needed, ensure the pod runs in a privileged namespace and the deployment is reviewed by the security team.	

Error Flow Diagram



#026	Multi-Container Port Conflict *Pod Lifecycle*	HIGH
DESCRIPTION	Two or more containers within the same pod are trying to bind to the same port, causing one or both containers to crash since pods share a network namespace.	
ROOT CAUSE	All containers in a pod share the same network namespace and therefore the same set of ports. If two containers both try to listen on port 8080, the second one will fail to bind. This often happens when sidecar containers (e.g., proxies, agents) use the same port as the main application.	
SOLUTION	Review all container port definitions in the pod spec. Change conflicting ports in the application configuration or sidecar configuration. Explicitly define containerPort in the pod spec for visibility. Common conflicts: port 8080 (app vs proxy), port 9090 (metrics). Use different ports for each container and document port assignments clearly.	

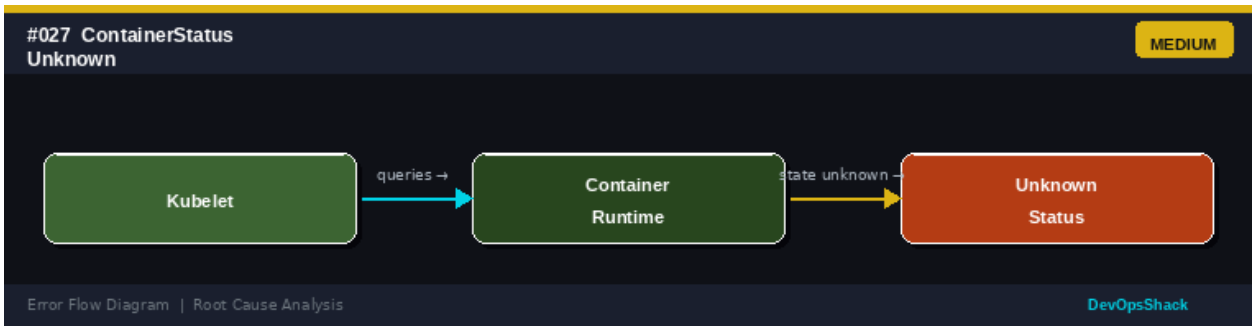
Error Flow Diagram



#027	ContainerStatus Unknown *Pod Lifecycle*	MEDIUM
DESCRIPTION	A container's status shows as Unknown, meaning Kubernetes cannot determine whether the container is running, stopped, or in what state it is.	
ROOT CAUSE	The kubelet cannot get container status from the container runtime. Causes: container runtime is unresponsive or crashed, the CRI (Container Runtime Interface) socket is broken, node communication issues, and a timing issue during rapid container state transitions.	
SOLUTION	Check the container runtime on the affected node: systemctl status containerd. Restart containerd if it is unresponsive: sudo systemctl restart containerd. Check kubelet logs: journalctl -u kubelet. If the node itself is problematic, drain and	

	replace it. Force-delete the pod if it is truly stuck: kubectl delete pod <pod> --force --grace-period=0.

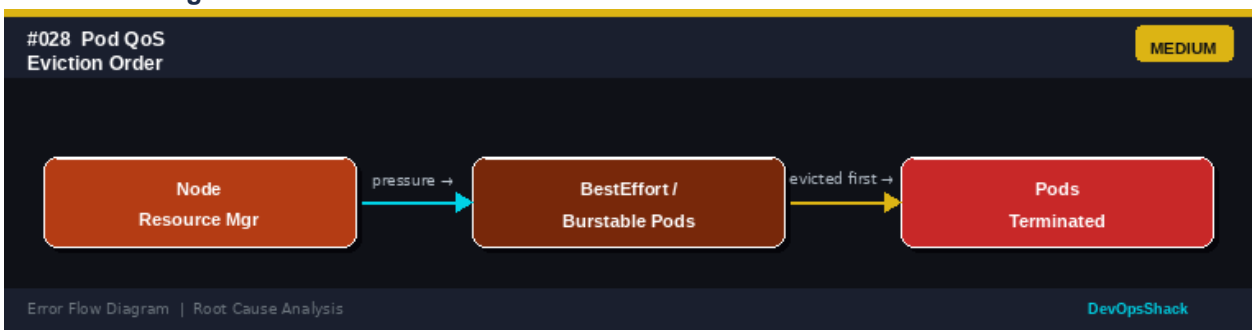
Error Flow Diagram



#028 Pod QoS Eviction Order *Pod Lifecycle* — MEDIUM

DESCRIPTION	Pods are being unexpectedly evicted during node resource pressure. Lower-priority pods (BestEffort or Burstable QoS) are evicted first when the node is under pressure.
ROOT CAUSE	Kubernetes assigns QoS classes based on resource specification: Guaranteed (requests=limits for all containers), Burstable (requests < limits or only some containers have limits), BestEffort (no requests or limits). Under memory/disk pressure, kubelet evicts BestEffort first, then Burstable, protecting Guaranteed pods.
SOLUTION	Set resource requests and limits on all containers to achieve Guaranteed QoS for critical pods: set requests equal to limits. Review QoS classes: kubectl get pod -o jsonpath='{.status.qosClass}'. For non-critical workloads, BestEffort QoS is acceptable. Investigate and resolve the root cause of node pressure (disk, memory). Use namespace resource quotas to prevent resource starvation.

Error Flow Diagram

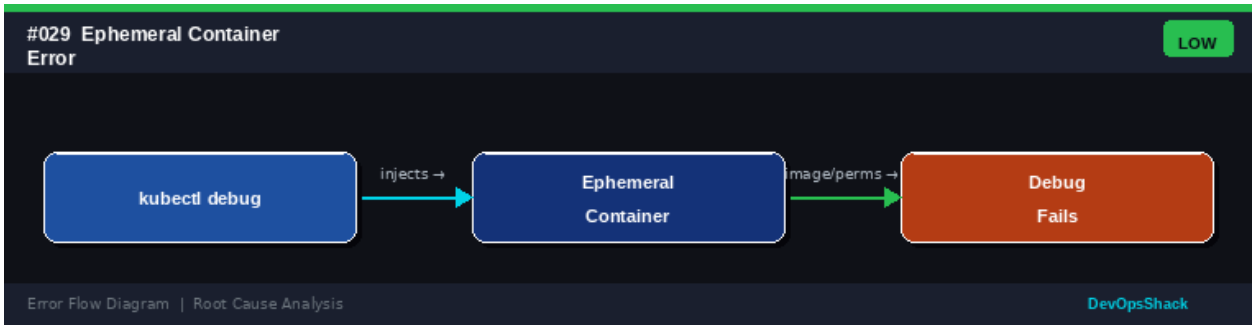


#029 Ephemeral Container Error *Pod Lifecycle* — LOW

DESCRIPTION	kubectl debug fails to inject an ephemeral container into a running pod, or the ephemeral container crashes immediately after injection.
ROOT CAUSE	Ephemeral containers are used for live debugging (kubectl debug). Failures occur when: the debug image cannot be pulled, the feature gate is disabled

	(Kubernetes <1.23), the pod is in a terminal state, the process namespace sharing is not enabled, and insufficient permissions to attach to the pod.
SOLUTION	Ensure Kubernetes version is 1.23+ (ephemeral containers are GA). Check feature gates in API server config. Use a lightweight debug image: kubectl debug -it <pod> --image=busybox --target=<container>. Ensure the user has RBAC permission to create ephemeral containers. Check that the pod is in Running state before debugging. Enable shareProcessNamespace: true for cross-container debugging.

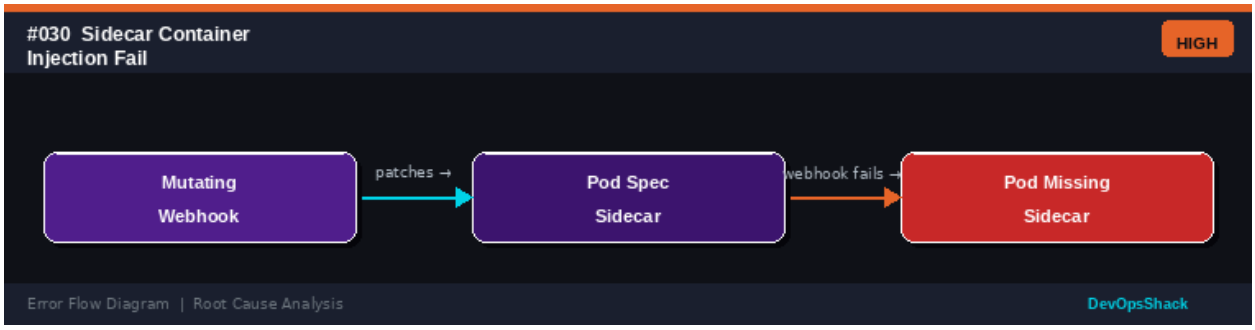
Error Flow Diagram



#030 Sidecar Container Injection Fail *Pod Lifecycle* — HIGH

DESCRIPTION	A sidecar container (e.g., Istio Envoy proxy, log forwarder) was not injected into the pod, causing service mesh or logging functionality to be missing.
ROOT CAUSE	Sidecar injection is performed by a MutatingAdmissionWebhook. Injection fails when: the webhook is not running or unhealthy, the namespace does not have the injection label (istio-injection: enabled), the pod has the injection disabled annotation, the webhook certificate is expired, and the webhook endpoint is unreachable from the API server.
SOLUTION	Check webhook status: kubectl get mutatingwebhookconfigurations. Ensure the namespace has the injection label: kubectl label namespace <ns> istio-injection=enabled. Check the webhook pod is running (e.g., istiod). Verify webhook TLS certificates are valid. Check for injection disabled annotation on the pod: sidecar.istio.io/inject: 'false'. Restart the webhook pod if it is unhealthy.

Error Flow Diagram



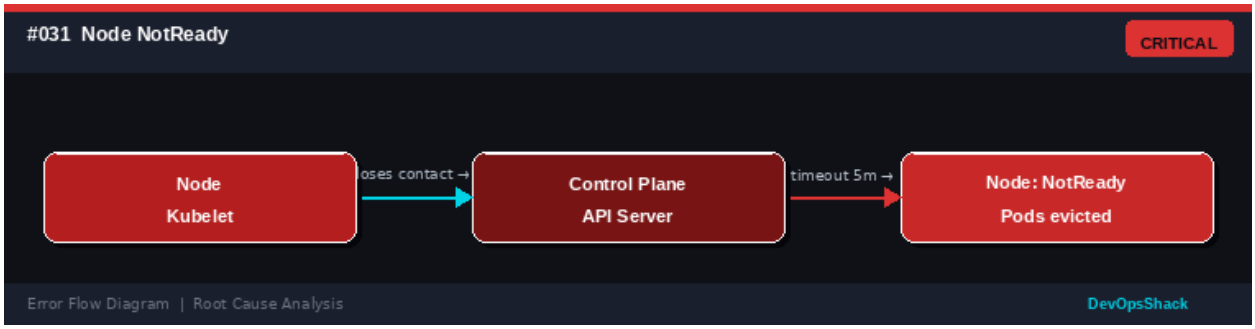
Node Errors

Errors #031 – #055

Node-level errors affect the compute infrastructure running your workloads. These include kubelet failures, resource pressures, network issues, and certificate problems that impact node availability and scheduling.

#031	Node NotReady *Node*	CRITICAL
DESCRIPTION	One or more nodes show a NotReady status in kubectl get nodes. Pods on the affected node may be evicted or become unreachable.	
ROOT CAUSE	The node has lost communication with the control plane or the kubelet has stopped functioning. After the node-monitor-grace-period (default 40s), the node controller marks it NotReady. After pod-eviction-timeout (default 5 minutes), pods are evicted. Causes: network partition, kubelet crash, node hardware failure, and resource exhaustion.	
SOLUTION	SSH to the node if possible and check kubelet: systemctl status kubelet. Restart kubelet: sudo systemctl restart kubelet. Check node events: kubectl describe node <node-name>. Verify network connectivity from node to API server. Check disk and memory: df -h, free -m. For cloud nodes, check the cloud provider console for instance health. Replace unrecoverable nodes and reschedule workloads.	

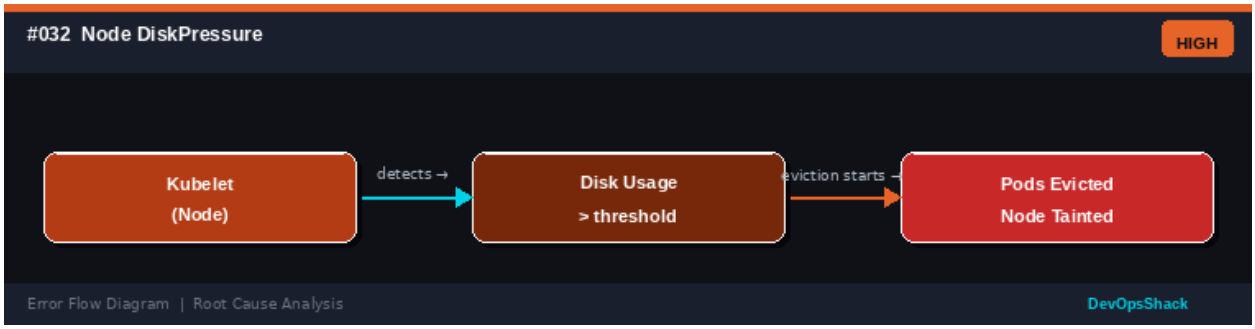
Error Flow Diagram



#032	Node DiskPressure *Node*	HIGH
DESCRIPTION	A node has the DiskPressure condition set to True, indicating that available disk space has fallen below the eviction threshold. Pods may be evicted.	
ROOT CAUSE	The kubelet monitors disk usage on the node. When available space or inodes fall below the evictionHardThreshold, the kubelet sets DiskPressure and begins evicting pods. Common causes: container log accumulation, large container images not being cleaned up, application writing excessive data to emptyDir or hostPath volumes.	
SOLUTION	Check disk usage on the node: df -h. Clean up unused container images: crictl rmi --prune or docker system prune. Set up log rotation and limit container log	

	size in kubelet config (containerLogMaxSize, containerLogMaxFiles). Configure emptyDir sizeLimit. Add a larger disk or a separate disk for container runtime data. Configure kubelet eviction thresholds appropriately.

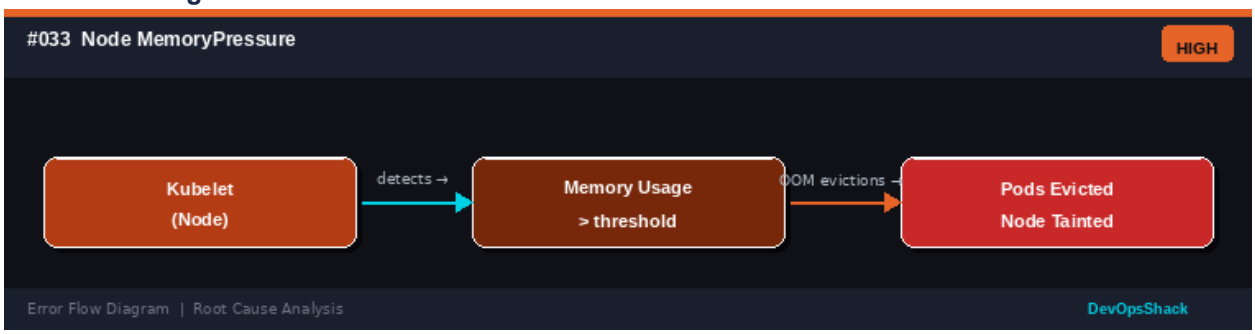
Error Flow Diagram



#033 Node MemoryPressure *Node* — HIGH

DESCRIPTION	A node has the MemoryPressure condition set to True. Available memory has dropped below the eviction threshold and the kubelet has started evicting pods.
ROOT CAUSE	When node memory available drops below the evictionHard threshold (default memory.available < 100Mi), kubelet sets MemoryPressure and begins evicting BestEffort and Burstable pods to reclaim memory. Causes: pods exceeding their memory limits (not being OOMKilled at pod level), system processes consuming memory, and memory leak in the container runtime.
SOLUTION	Identify memory-consuming processes: top or kubectl top pods --all-namespaces --sort-by memory. Set appropriate memory limits on all pods to trigger OOM kills at pod level rather than node level. Add more memory to the node. Tune kubelet eviction thresholds. Check for memory leaks in applications. Scale out to distribute memory load across more nodes.

Error Flow Diagram

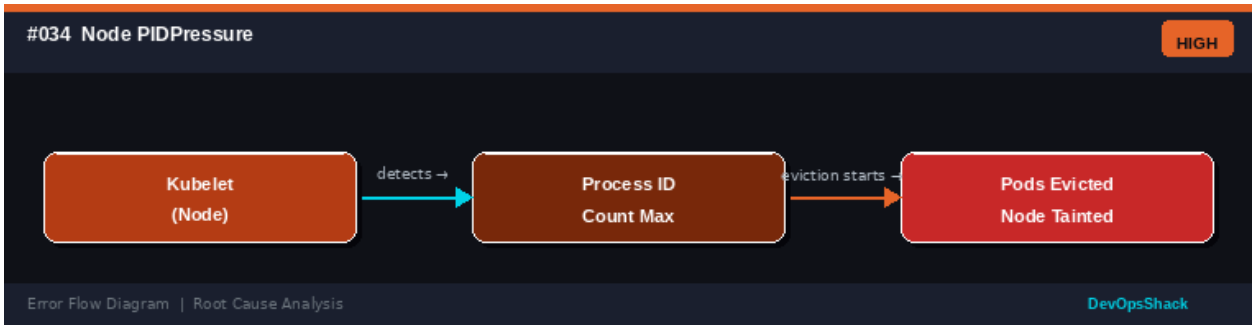


#034 Node PIDPressure *Node* — HIGH

DESCRIPTION	A node shows PIDPressure condition True because the number of running processes has reached the configured limit, threatening stability.
ROOT CAUSE	Each container process counts toward the system's PID limit. The kubelet monitors PID availability and sets PIDPressure when PID count falls below the

	eviction threshold. Causes: applications spawning excessive child processes, PID namespace not properly isolated, runaway processes (fork bombs), and pid.available eviction threshold too high.
SOLUTION	Check PID usage: cat /proc/sys/kernel/pid_max and ps aux | wc -l. Identify pods with high PID counts. Set PID limits in pod spec using resources.limits with pid resources. Increase the system PID limit: sysctl -w kernel.pid_max=<value>. Configure kubelet with --system-reserved='pid=1000' and --kube-reserved='pid=500'. Fix application processes that spawn excessive children.

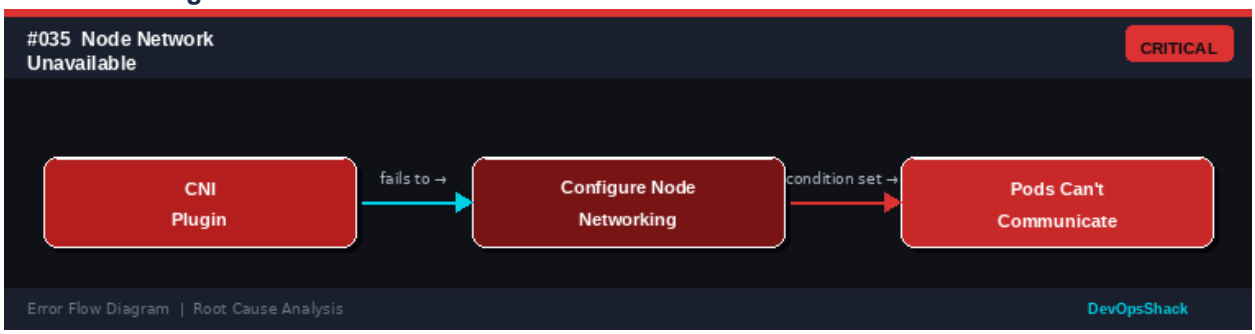
Error Flow Diagram



#035 Node Network Unavailable *Node* — CRITICAL

DESCRIPTION	A node has the NetworkUnavailable condition set to True, meaning the node networking is not correctly configured. Pods on this node cannot communicate.
ROOT CAUSE	The CNI plugin failed to configure the node's network properly. This sets the NetworkUnavailable condition to True. Causes: CNI plugin pod is not running on the node, CNI binary or config file is missing, network interface configuration failed, and IP address conflicts on the node network.
SOLUTION	Check CNI pod status on the affected node: kubectl get pods -n kube-system -o wide. Check CNI plugin logs (e.g., calico-node, flannel). Verify CNI binary is present: ls /opt/cni/bin/. Check CNI config: ls /etc/cni/net.d/. Restart the CNI DaemonSet pod on the node. Verify node network interfaces: ip addr show. Re-add the node if CNI configuration is unrecoverable.

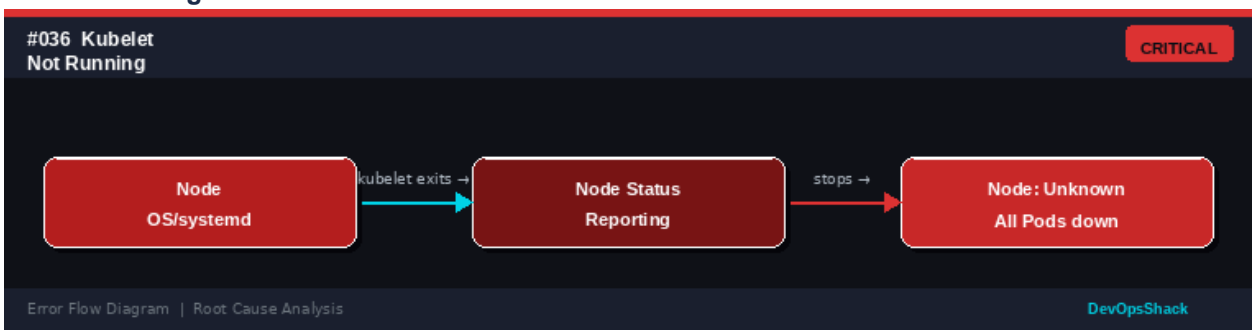
Error Flow Diagram



#036 Kubelet Not Running *Node* — CRITICAL

DESCRIPTION	The kubelet process is not running on a node. The node becomes unreachable and no new pods can be scheduled or started on that node.
ROOT CAUSE	The kubelet is the primary node agent and must run continuously. It stops due to: service crash (OOM, bug), manual stop during maintenance, systemd failure, configuration error preventing restart, and kubelet version incompatibility after upgrade.
SOLUTION	SSH to the node and check kubelet status: systemctl status kubelet. View kubelet logs for the crash reason: journalctl -u kubelet -n 100. Fix configuration errors in /var/lib/kubelet/config.yaml. Restart kubelet: sudo systemctl restart kubelet. Enable kubelet to auto-restart: sudo systemctl enable kubelet. For persistent failures, check OS resources (disk, memory) and kubelet binary integrity.

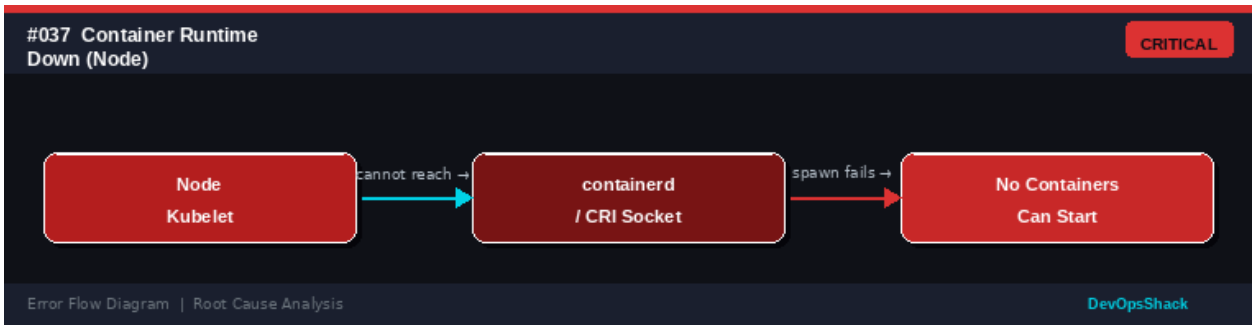
Error Flow Diagram



#037 Container Runtime Down *Node* — CRITICAL

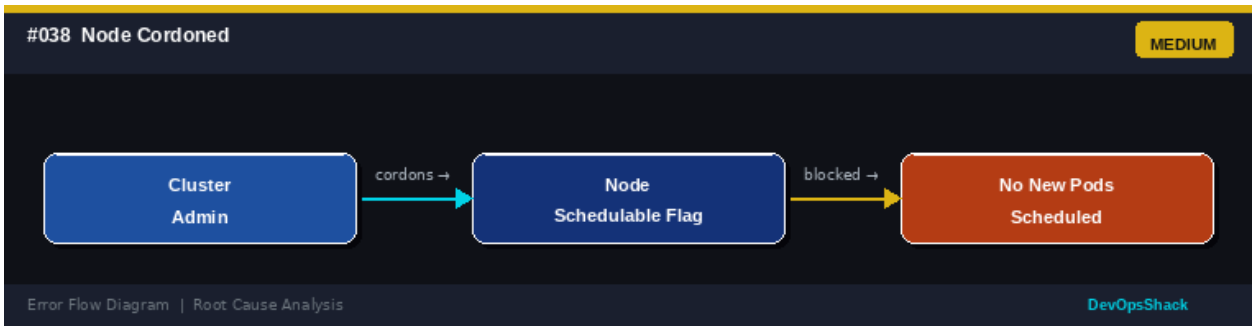
DESCRIPTION	The container runtime (containerd or CRI-O) has stopped functioning on a node. The kubelet cannot create, start, or inspect containers. All existing pods on the node are unreachable.
ROOT CAUSE	The container runtime is a critical dependency of the kubelet. If containerd or CRI-O crashes, the kubelet cannot perform any container operations. Causes: runtime OOM kill, containerd daemon crash, CRI socket file deleted or permission changed, runtime storage corruption, and kernel version incompatibility.
SOLUTION	SSH to the node. Check runtime status: systemctl status containerd. View runtime logs: journalctl -u containerd -n 200. Restart the runtime: sudo systemctl restart containerd. Check disk space for runtime data: df -h /var/lib/containerd. Verify socket exists: ls -la /run/containerd/containerd.sock. If runtime data is corrupt, consider wiping /var/lib/containerd and re-pulling images (drain node first).

Error Flow Diagram



#038	Node Cordoned *Node*	MEDIUM
DESCRIPTION	A node has been cordoned (kubectl cordon) and no new pods will be scheduled on it. Existing pods continue running but the node is marked as unschedulable.	
ROOT CAUSE	Cordoning is intentional but can be forgotten. Administrators cordon nodes before maintenance. If not uncordoned after maintenance, new pods continue to be excluded from this node, reducing cluster capacity. Caused by manual admin action or automation tools (e.g., node problem detector, cluster autoscaler).	
SOLUTION	Check cordoned nodes: kubectl get nodes. If maintenance is complete, uncordon: kubectl uncordon <node-name>. Review who/what cordoned the node using audit logs. Check if a node problem detector is automatically cordoning nodes due to persistent issues. Automate uncordoning in your maintenance scripts to prevent forgotten cordons.	

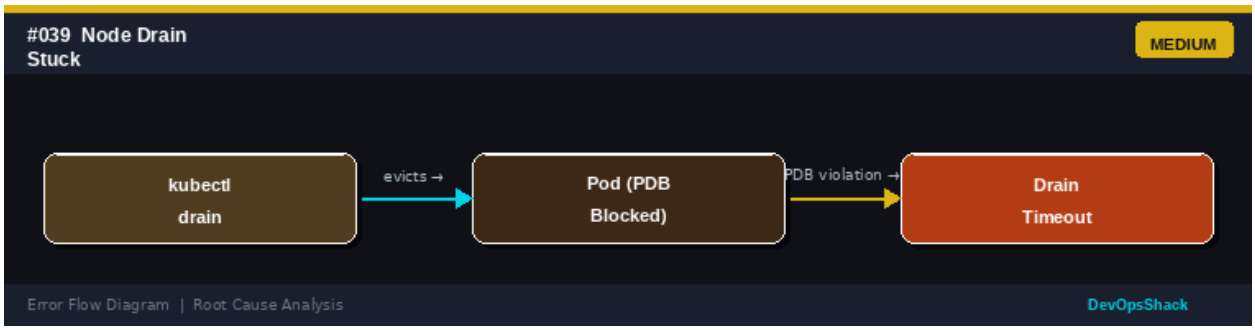
Error Flow Diagram



#039	Node Drain Stuck *Node*	MEDIUM
DESCRIPTION	kubectl drain is hanging and not completing. One or more pods are blocking eviction, preventing the drain from finishing.	
ROOT CAUSE	Node drain evicts all pods using the Eviction API. Drains get stuck when: a PodDisruptionBudget blocks eviction, a pod has no replication and --ignore-daemonsets is not set, a pod has a long terminationGracePeriodSeconds, a pod is stuck in Terminating state, and local storage pods are not allowed to be drained without --delete-emptydir-data.	
SOLUTION	Use kubectl drain <node> --ignore-daemonsets --delete-emptydir-data. Add --timeout=300s. If PDB is blocking: review PDB settings or temporarily delete it. Check for stuck Terminating pods and force delete them. Add --force to drain	

	pods without a controller. After drain, verify with kubectl get pods -o wide to ensure no pods remain on the node.

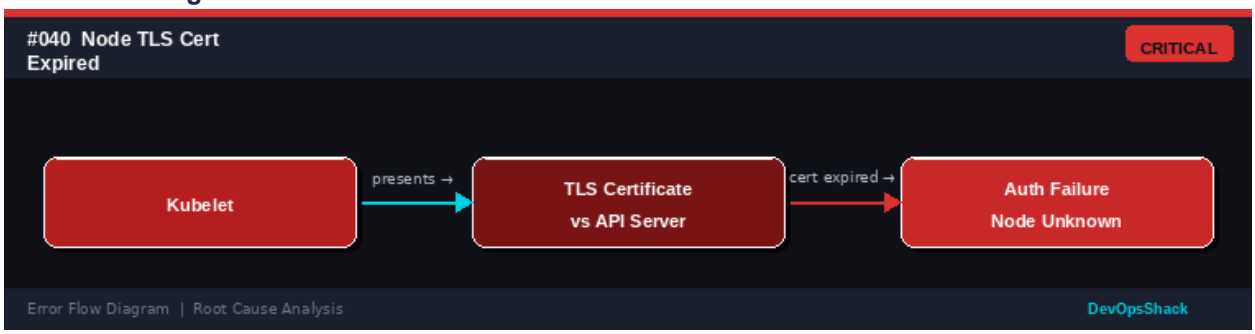
Error Flow Diagram



#040 Node TLS Certificate Expired *Node* — CRITICAL

DESCRIPTION	The kubelet certificate used to communicate with the API server has expired. The node shows Unknown or NotReady status and cannot rejoin the cluster.
ROOT CAUSE	Kubelet uses TLS certificates for secure communication. These certificates expire (typically 1 year). If auto-rotation is not configured or certificate rotation approval is not automated, certificates expire causing the kubelet to fail API server authentication. After expiry, the node cannot register or communicate.
SOLUTION	Check certificate expiration: openssl x509 -in /var/lib/kubelet/pki/kubelet-client-current.pem -noout -dates. Approve pending certificate rotation requests: kubectl get csr and kubectl certificate approve <csr-name>. Configure automatic CSR approval with kube-controller-manager flags. Enable kubelet certificate rotation: rotateCertificates: true in kubelet config. Renew expired certificates manually using kubeadm certs renew.

Error Flow Diagram

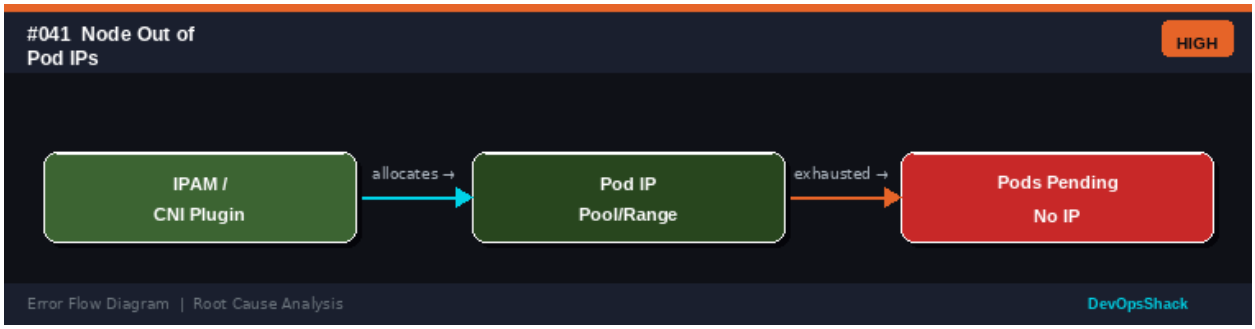


#041 Node Out of Pod IPs *Node* — HIGH

DESCRIPTION	New pods on a node cannot get an IP address because the node's Pod IP pool (CIDR) is exhausted. Pods remain in ContainerCreating state.
ROOT CAUSE	Each node is assigned a podCIDR range (e.g., 10.244.0.0/24 = 256 IPs). If more pods than available IPs are scheduled to the node, new pods cannot get IPs.

	This is common in small CIDR ranges with large pod density and when IPs from terminated pods are not released quickly.
SOLUTION	Check pod CIDR: kubectl get node <node> -o jsonpath='{.spec.podCIDR}'. Count pods on node: kubectl get pods --all-namespaces -o wide | grep <node-name> | wc -l. Reduce pods per node or increase node CIDR size (requires cluster recreate for many CNIs). Configure CNI with larger IP pools. Enable IPv6 dual-stack for more addresses. Use a CNI supporting IP reuse acceleration.

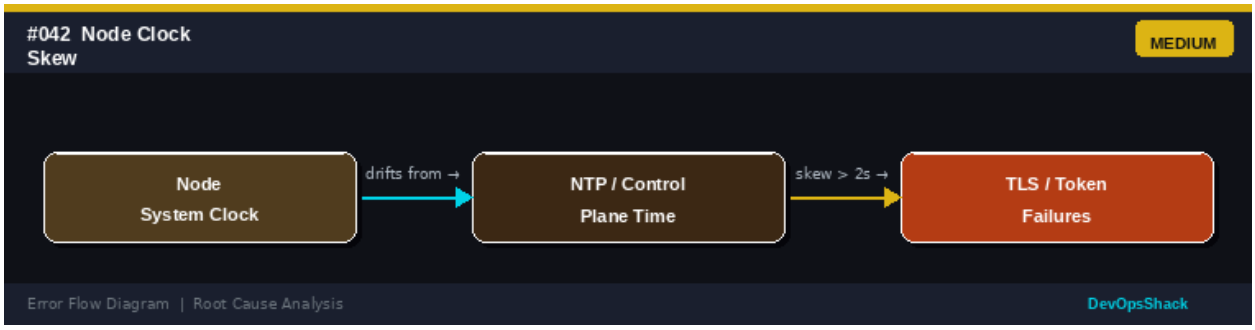
Error Flow Diagram



#042 Node Clock Skew *Node* — MEDIUM

DESCRIPTION	The node's system clock has drifted significantly from the control plane time, causing authentication failures, certificate validation errors, and token expiry issues.
ROOT CAUSE	TLS certificates and JWT tokens are time-sensitive. If a node's clock is skewed by more than a few seconds, the API server rejects requests with certificate not valid yet or token has expired errors. Causes: NTP not configured, NTP server unreachable, VM time drift after suspend/resume, and incorrect timezone settings.
SOLUTION	Check node time: date. Check NTP sync status: timedatectl status or chronyc tracking. Enable NTP: timedatectl set-ntp true. Configure NTP server in /etc/chrony.conf or /etc/systemd/timesyncd.conf. For cloud VMs, use the cloud provider's NTP endpoint. Set a cluster-wide monitoring alert for clock skew. Kubernetes recommends clock skew < 2 seconds.

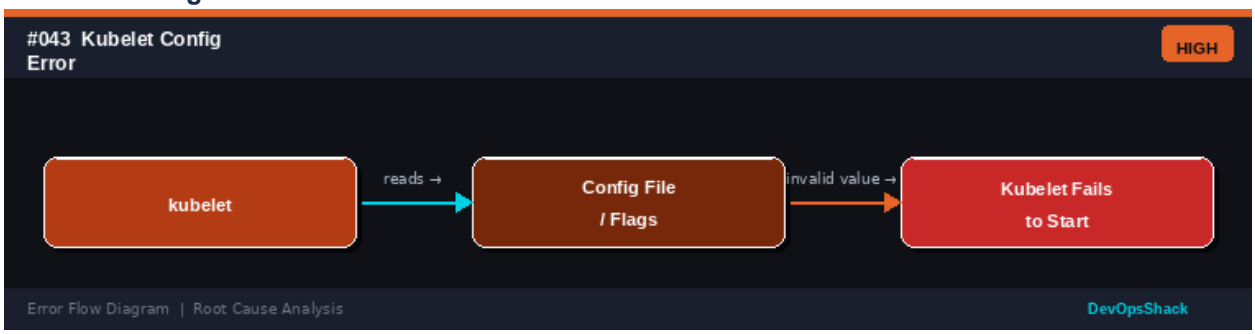
Error Flow Diagram



#043 Kubelet Config Error *Node* — HIGH

DESCRIPTION	The kubelet fails to start because of an invalid value in the kubelet configuration file or command-line flags.
ROOT CAUSE	The kubelet reads its configuration from /var/lib/kubelet/config.yaml (kubeadm clusters) or command-line flags. Invalid YAML syntax, unsupported feature gate names, incorrect value types, and references to non-existent files cause the kubelet to fail on startup.
SOLUTION	Check kubelet logs immediately after startup: journalctl -u kubelet -n 50. The error message usually identifies the problematic config field. Validate the config file: kubelet --config=/var/lib/kubelet/config.yaml --dry-run. Compare against valid kubelet config schema. Reset to a known-good configuration using kubeadm init phase kubelet-start on the affected node.

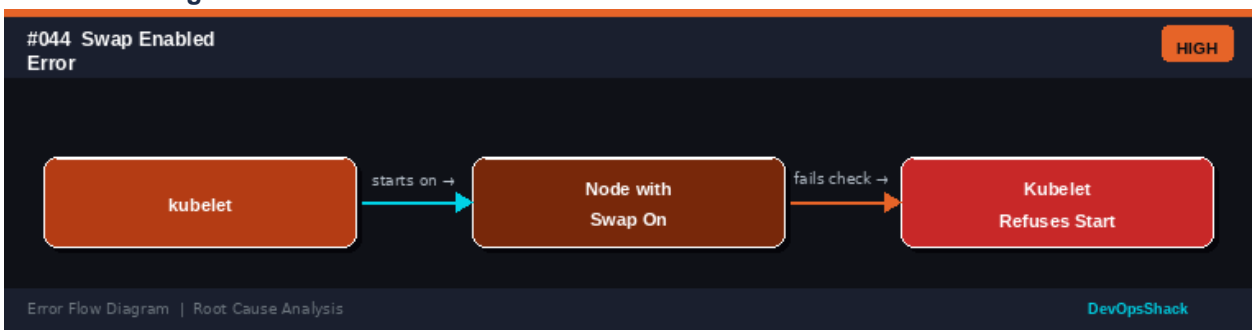
Error Flow Diagram



#044 Swap Enabled Error *Node* — HIGH

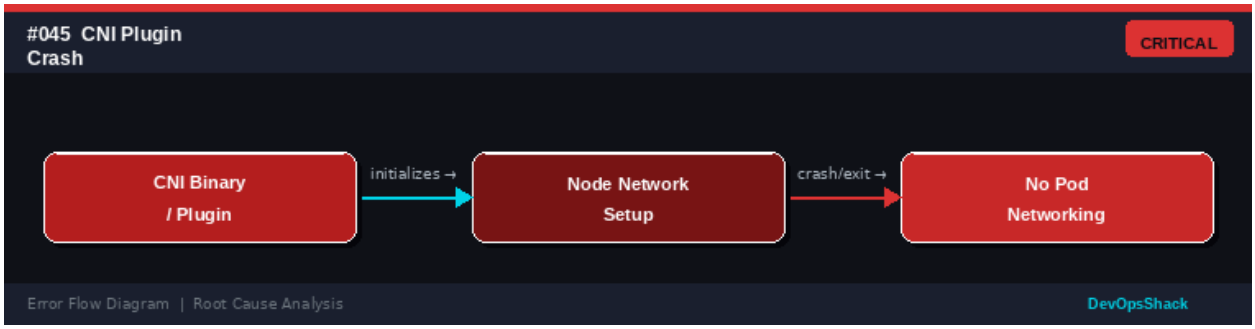
DESCRIPTION	The kubelet refuses to start because swap memory is enabled on the node. Kubernetes traditionally requires swap to be disabled.
ROOT CAUSE	By default, kubelet requires swap to be disabled (--fail-swap-on=true). Kubernetes memory management and QoS rely on cgroup memory limits, which do not interact well with swap. Since Kubernetes 1.28, swap support is beta (NodeSwap feature gate), but it requires explicit kubelet configuration.
SOLUTION	Disable swap: sudo swapoff -a. Make permanent by commenting out swap entries in /etc/fstab. Verify: free -h (swap should show 0). Restart kubelet. For Kubernetes 1.28+, if swap is needed, configure kubelet with failSwapOn: false and memorySwap.swapBehavior: LimitedSwap in the kubelet config with the NodeSwap feature gate enabled.

Error Flow Diagram



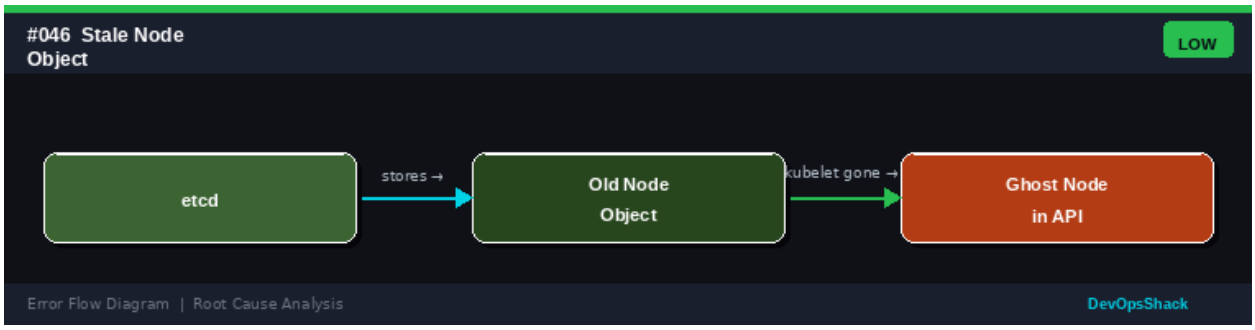
#045	CNI Plugin Crash *Node*	CRITICAL
DESCRIPTION	The CNI (Container Network Interface) plugin DaemonSet pod has crashed or is not running on one or more nodes, causing pod networking to fail on those nodes.	
ROOT CAUSE	CNI plugins (Calico, Flannel, Cilium, Weave) run as DaemonSets and configure pod networking. A crash causes all new pods on the affected node to fail network setup. Causes: CNI configuration error, missing kernel modules (e.g., ip_tables), insufficient permissions, network backend unavailability, and CNI binary corruption.	
SOLUTION	Check CNI pods: kubectl get pods -n kube-system -l <cni-selector> -o wide. View CNI pod logs: kubectl logs -n kube-system <cni-pod>. Verify kernel modules: modprobe ip_tables ip6tables. Check CNI config file: cat /etc/cni/net.d/*.conf. Re-apply CNI manifests: kubectl apply -f <cni-manifest.yaml>. Check CNI release notes for known issues with your Kubernetes version.	

Error Flow Diagram



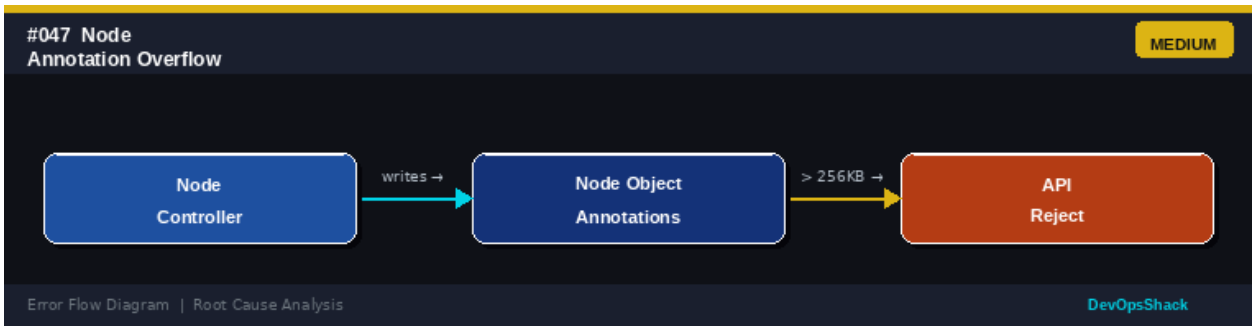
#046	Stale Node Object *Node*	LOW
DESCRIPTION	An old node object remains in the cluster API after the node is removed, showing as NotReady or Unknown. This is a ghost/phantom node.	
ROOT CAUSE	When nodes are removed without proper deregistration (cloud instances terminated without drain/delete), the node object persists in etcd. The garbage collection controller eventually removes it but may be slow. This causes confusion in monitoring and scheduling.	
SOLUTION	Delete stale node objects: kubectl delete node <node-name>. Automate node lifecycle: use cloud provider node auto-deregistration or configure the cloud controller manager. Enable node lifecycle controller to automatically remove nodes that have been NotReady too long. Check for Cluster Autoscaler configuration to clean up terminated instances.	

Error Flow Diagram



#047	Node Annotation Overflow *Node*	MEDIUM
DESCRIPTION	API Server rejects updates to the node object because the annotations have grown too large, exceeding Kubernetes object size limits.	
ROOT CAUSE	Kubernetes objects have a maximum size of ~1.5MB in etcd. Node annotations accumulate from multiple controllers, operators, and tools writing metadata. When the total object size exceeds limits, further updates fail with Request Entity Too Large.	
SOLUTION	Check node object size: kubectl get node <name> -o json | wc -c. Identify large annotations: kubectl get node <name> -o jsonpath='{.metadata.annotations}'. Remove unnecessary annotations: kubectl annotate node <name> <annotation-key>-. Configure controllers to use labels instead of large annotations. Implement annotation cleanup automation.	

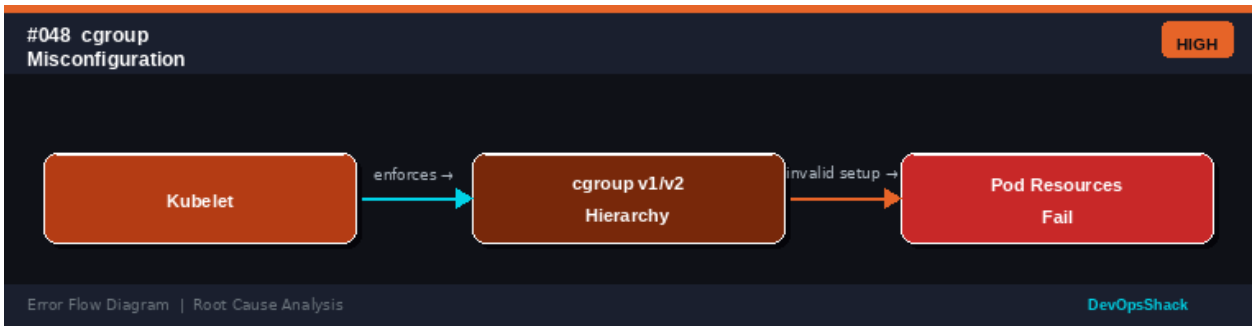
Error Flow Diagram



#048	cgroup Misconfiguration *Node*	HIGH
DESCRIPTION	Pods fail to start or experience incorrect resource enforcement because the cgroup hierarchy on the node is incorrectly configured.	
ROOT CAUSE	Kubernetes uses cgroups to enforce CPU and memory limits. Misconfiguration occurs when: systemd and kubelet use different cgroup drivers (must both use systemd or cgroupfs), cgroup v1/v2 mismatch, cgroup root path is incorrect, and after OS upgrades that change the default cgroup version.	
SOLUTION	Check cgroup driver consistency: kubelet config shows cgroupDriver and verify it matches: systemctl show containerd | grep Delegate. Check system cgroup version: stat -fc %T /sys/fs/cgroup/. For cgroup v2, ensure containerd and kubelet	

	both support it. Set cgroupDriver: systemd consistently in both kubelet config and containerd config.toml. Restart both services after changes.

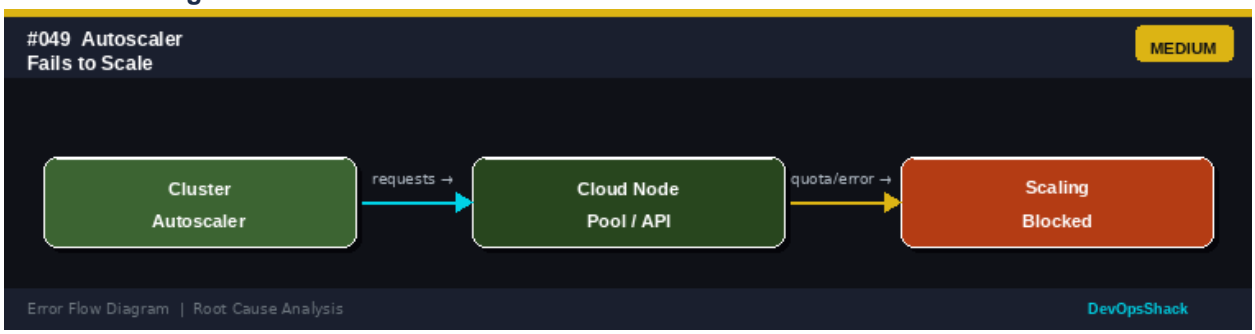
Error Flow Diagram



#049 Autoscaler Fails to Scale *Node* — MEDIUM

DESCRIPTION	The Cluster Autoscaler cannot provision new nodes when pods are unschedulable due to resource constraints.
ROOT CAUSE	The Cluster Autoscaler monitors unschedulable pods and requests new nodes from the cloud provider. Failures occur when: cloud quota is exhausted, IAM permissions are insufficient, the node group configuration is incorrect, scaling cooldown is active, and pod resource requests are too high for available instance types.
SOLUTION	Check Cluster Autoscaler logs: kubectl logs -n kube-system <autoscaler-pod> | grep -i error. Verify cloud quota in the provider console. Check IAM permissions for autoscaler service account. Review node group min/max configuration. Check for scale-down lock: kubectl describe cm cluster-autoscaler-status -n kube-system. Manually scale the node group to verify cloud provider integration is working.

Error Flow Diagram

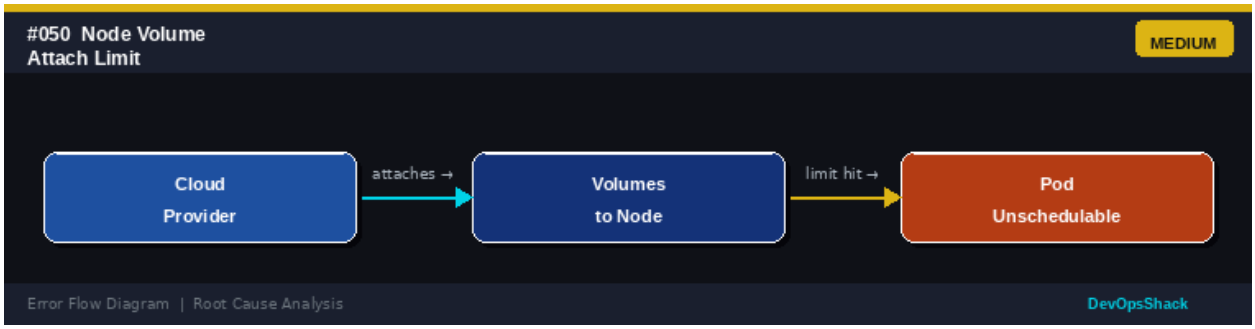


#050 Node Volume Attach Limit *Node* — MEDIUM

DESCRIPTION	Pods are stuck in Pending because the node has reached the maximum number of volumes that can be attached to a single instance (cloud provider limit).
ROOT CAUSE	Cloud providers limit the number of block volumes per VM instance (e.g., AWS EBS: 25-39 volumes per instance type, GCP: 128 volumes). When this limit is

	reached, no more PVCs can be attached to pods on that node. The scheduler should avoid such nodes but may not always do so correctly.
SOLUTION	Check volume attach count: kubectl get pods -o wide and count PVCs per node. Review cloud provider limits for your instance type. Distribute pods with PVCs across more nodes. Use node anti-affinity or topology spread constraints to limit pods with volumes per node. Upgrade to instance types with higher volume limits. Consider using shared filesystems (RWX PVCs) to reduce volume count.

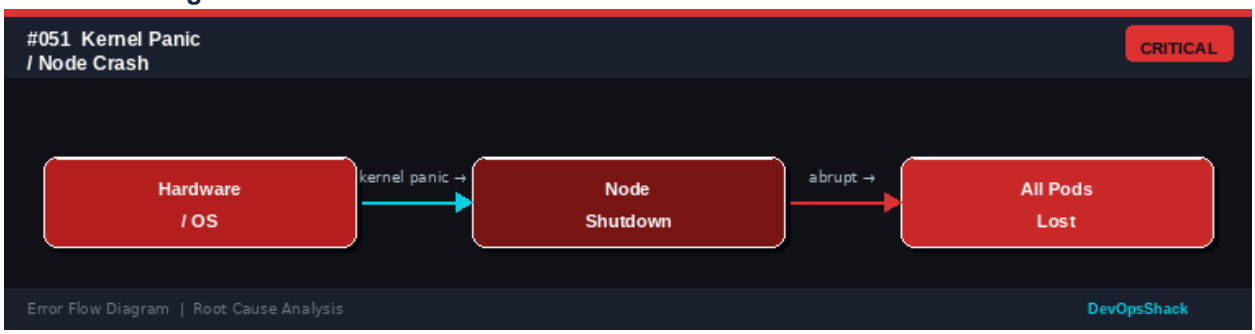
Error Flow Diagram



#051 Kernel Panic / Node Crash *Node* — CRITICAL

DESCRIPTION	A node has abruptly shut down or restarted due to a kernel panic, hardware failure, or OS-level crash. All pods on the node are lost.
ROOT CAUSE	Kernel panics are unrecoverable OS failures caused by: hardware faults (bad RAM, CPU errors), driver bugs, kernel bugs in heavily customized kernels, severe disk I/O errors, and software errors in kernel modules (e.g., from custom CNI kernel modules).
SOLUTION	Check kernel logs from before the crash: journalctl -k -b -1 (previous boot). Review /var/crash if kdump is configured. Check hardware health: dmesg | grep -i error. Enable and configure kdump for crash analysis. Use memory test tools (memtest86+) for RAM issues. Report kernel bugs upstream. Replace faulty hardware. Ensure node auto-recovery replaces crashed nodes.

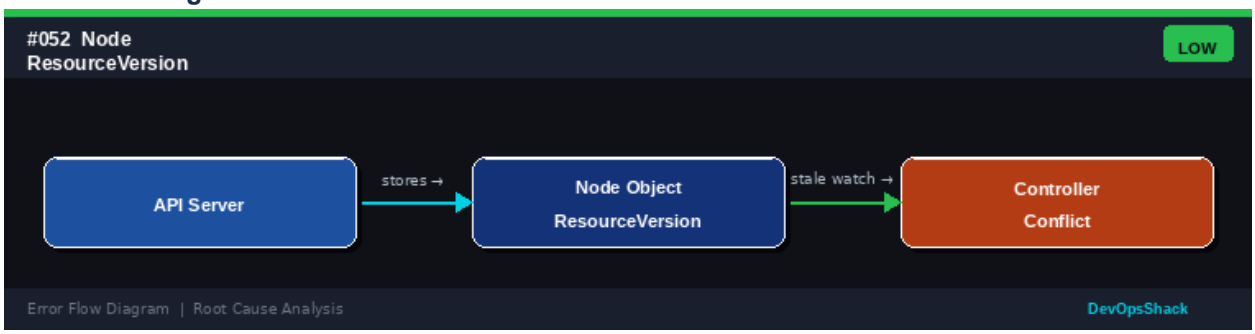
Error Flow Diagram



#052 Node ResourceVersion Conflict *Node* — LOW

DESCRIPTION	Controllers or operators receive 409 Conflict errors when updating node objects because multiple controllers are racing to update the same resource version.
ROOT CAUSE	Kubernetes uses optimistic concurrency via ResourceVersion. When multiple controllers read and update the same object simultaneously, only one update succeeds; others get 409 Conflict. This is usually transient but indicates high controller contention.
SOLUTION	This is typically self-healing - controllers retry on 409. Investigate if conflicts are frequent using audit logs. Ensure only one controller manages node objects for a given concern. Reduce update frequency in custom controllers. Use patch operations (kubectl patch) instead of full updates to minimize conflict windows. This is usually a monitoring/alerting concern rather than a critical error.

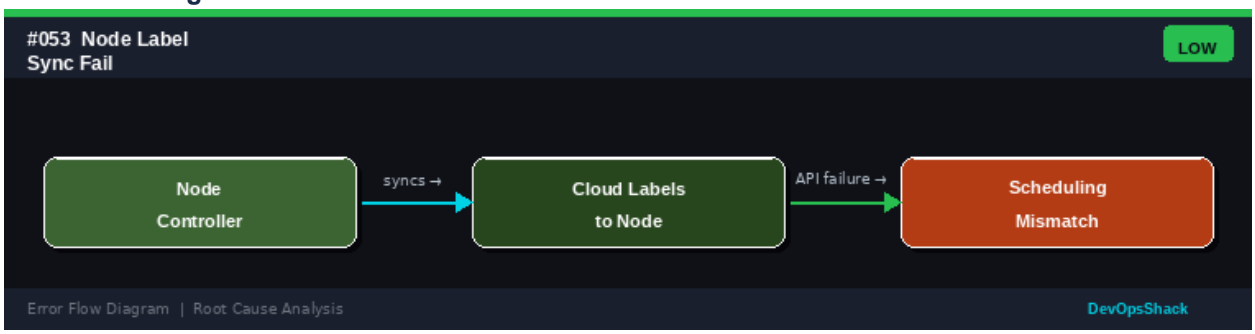
Error Flow Diagram



#053 Node Label Sync Failure *Node* — LOW

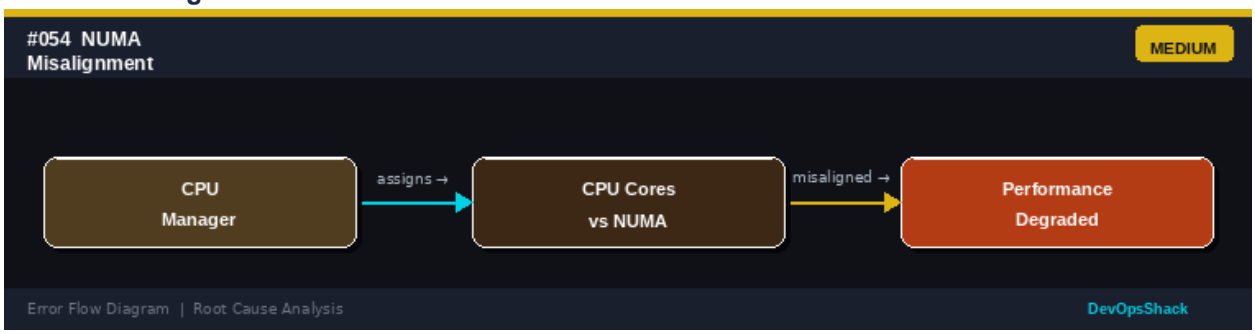
DESCRIPTION	Node labels from the cloud provider are not being synced to the Kubernetes node objects, causing scheduling decisions based on those labels to fail.
ROOT CAUSE	The cloud controller manager syncs cloud instance labels to Kubernetes node labels. Failures occur when: the cloud controller manager is not running or misconfigured, RBAC permissions are insufficient, cloud provider API is throttled, and the instance no longer exists.
SOLUTION	Check cloud controller manager: kubectl get pods -n kube-system | grep cloud-controller. Review cloud controller logs for API errors. Verify IAM/service account permissions to read cloud instance metadata. Manually label nodes as a workaround: kubectl label node <name> <key>=<value>. Ensure the cloud controller manager version matches your cloud provider's K8s integration requirements.

Error Flow Diagram



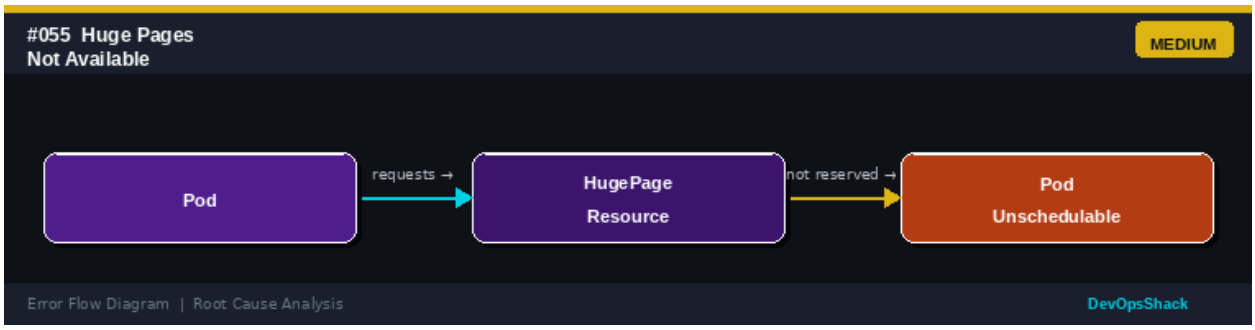
#054	NUMA Misalignment *Node*	MEDIUM
DESCRIPTION	CPU-intensive pods experience worse performance than expected because CPUs assigned to the container are not NUMA-aligned with the memory and I/O they are using.	
ROOT CAUSE	Non-Uniform Memory Access (NUMA) architectures have multiple memory domains. When a container's CPUs are from different NUMA nodes than its memory, memory latency increases. Kubernetes CPU Manager in static policy can handle NUMA topology, but requires careful configuration.	
SOLUTION	Enable CPU Manager in static policy: cpuManagerPolicy: static in kubelet config. Enable Topology Manager: topologyManagerPolicy: best-effort or single-numa-node. Request whole CPUs (integer CPU limits) in the pod spec. Set requests equal to limits for Guaranteed QoS. Monitor NUMA performance with tools like numad or perf. Use hardware with NUMA balancing enabled in the kernel.	

Error Flow Diagram



#055	Huge Pages Not Available *Node*	MEDIUM
DESCRIPTION	A pod requesting HugePages resources cannot be scheduled because the nodes do not have HugePages pre-allocated.	
ROOT CAUSE	HugePages (2Mi or 1Gi) must be pre-allocated on the OS before kubelet starts. If the node does not have enough pre-allocated HugePages matching the pod request, the pod is unschedulable. HugePages are consumed and released like regular resources but require OS-level setup.	
SOLUTION	Pre-allocate HugePages on nodes: echo 512 > /proc/sys/vm/nr_hugepages or set vm.nr_hugepages=512 in /etc/sysctl.conf. Restart kubelet after allocating HugePages so it picks them up. Verify HugePages are visible to kubelet: kubectl describe node | grep hugepages. Request specific size in pod: resources.requests.hugepages-2Mi: 512Mi. Add node labels to identify HugePage-capable nodes for scheduling.	

Error Flow Diagram



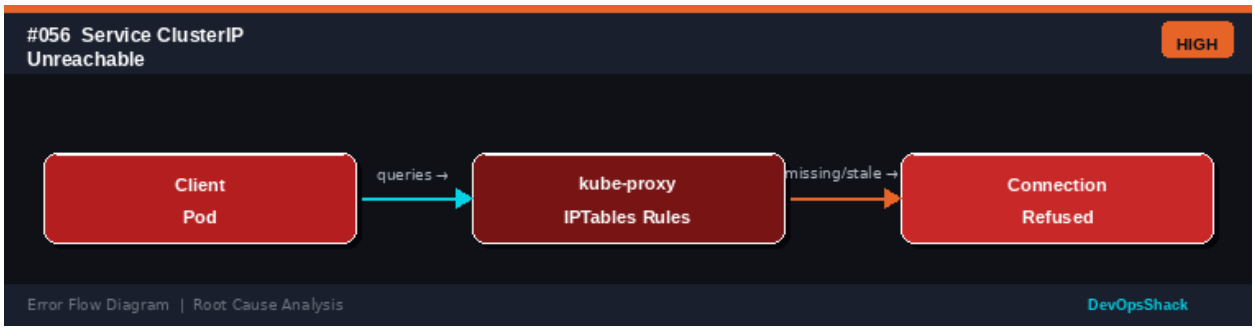
Networking Errors

Errors #056 – #080

Networking errors in Kubernetes span Service discovery, DNS resolution, Ingress routing, CNI plugin failures, and network policy issues. These errors often manifest as connectivity failures between pods, services, or external clients.

#056	Service ClusterIP Unreachable *Networking*	HIGH
DESCRIPTION	Pods cannot connect to a Kubernetes Service by its ClusterIP address. Connections are refused or time out even though the target pods appear to be running.	
ROOT CAUSE	Service routing relies on kube-proxy maintaining iptables or IPVS rules. ClusterIP becomes unreachable when: kube-proxy is not running or crashed, iptables rules are corrupted or incomplete, IPVS mode has stale entries, and the Service's endpoint list is empty because no pods match the selector.	
SOLUTION	Check kube-proxy: kubectl get pods -n kube-system | grep kube-proxy. Check Service endpoints: kubectl get endpoints <service-name>. If endpoints are empty, the selector likely doesn't match any pod labels. From a pod, test: curl <clusterIP>:<port>. Restart kube-proxy DaemonSet to regenerate iptables rules. Check for iptables conflicts with other firewall tools (e.g., UFW, firewalld).	

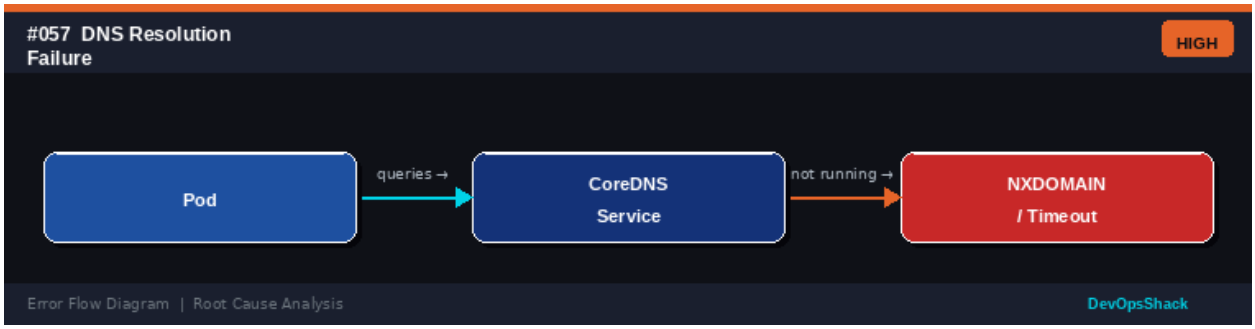
Error Flow Diagram



#057	DNS Resolution Failure *Networking*	HIGH
DESCRIPTION	Pods cannot resolve domain names. DNS lookups return NXDOMAIN or time out. The cluster DNS service (CoreDNS) may be unavailable.	
ROOT CAUSE	Kubernetes cluster DNS is provided by CoreDNS, running as pods in the kube-system namespace. DNS resolution fails when: CoreDNS pods are down or crashing, the kube-dns Service has no endpoints, the pod's /etc/resolv.conf points to wrong nameserver, and NetworkPolicy blocks DNS traffic (UDP/TCP port 53).	
SOLUTION	Check CoreDNS pods: kubectl get pods -n kube-system -l k8s-app=kube-dns. Check CoreDNS endpoints: kubectl get endpoints kube-dns -n kube-system. From a pod, test DNS: kubectl exec -it <pod> -- nslookup kubernetes. Check	

	/etc/resolv.conf in the pod. Review CoreDNS ConfigMap for syntax errors: kubectl get cm coredns -n kube-system -o yaml. Verify NetworkPolicy allows DNS traffic on port 53.

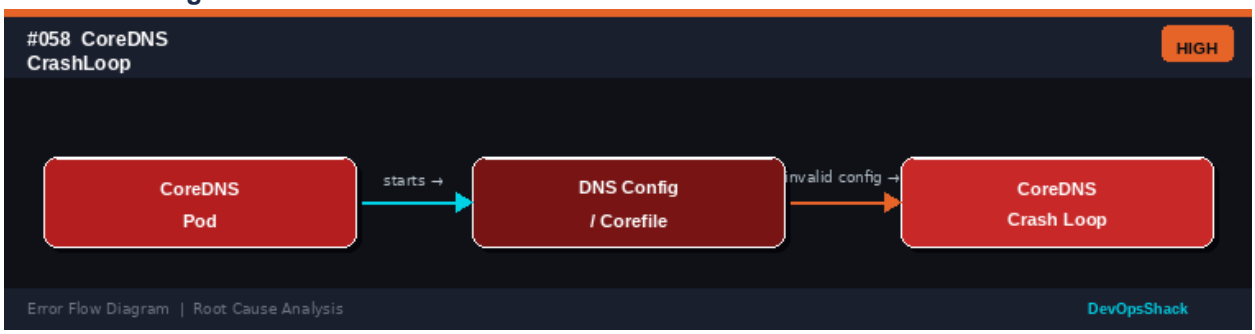
Error Flow Diagram



#058 CoreDNS CrashLoop *Networking* — HIGH

DESCRIPTION	CoreDNS pods are in CrashLoopBackOff, causing cluster-wide DNS resolution failures and breaking most inter-service communication.
ROOT CAUSE	CoreDNS crashes due to: invalid Corefile configuration syntax, loop detection when forwarding to upstream DNS that itself queries CoreDNS, insufficient memory limits (CoreDNS OOMKilled), and incompatible CoreDNS plugin configuration.
SOLUTION	Check CoreDNS logs: kubectl logs -n kube-system <coredns-pod>. Common fix for loop detection: add loop disable plugin to Corefile or forward to a different DNS server. Check the CoreDNS ConfigMap: kubectl edit cm coredns -n kube-system. Increase CoreDNS memory limits if OOMKilled. Validate Corefile syntax before applying changes. Rollback to a working CoreDNS configuration.

Error Flow Diagram

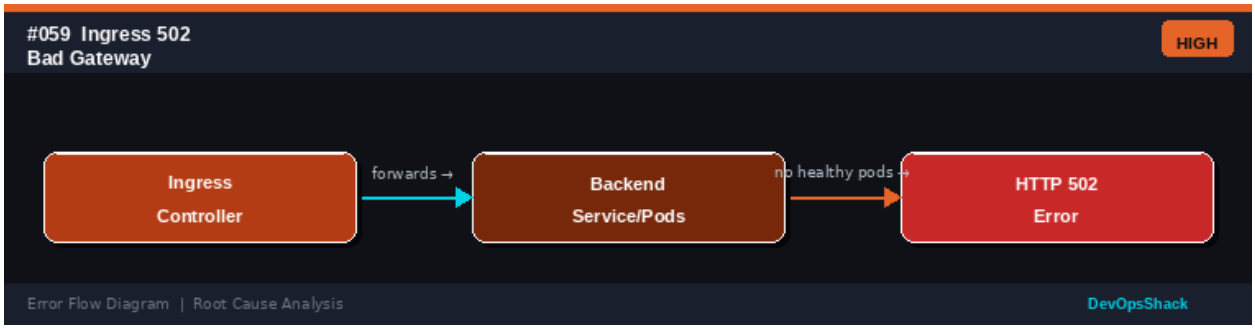


#059 Ingress 502 Bad Gateway *Networking* — HIGH

DESCRIPTION	The Ingress controller returns HTTP 502 Bad Gateway when clients try to access services through Ingress. The controller cannot reach the backend service.
ROOT CAUSE	The Ingress controller forwards requests to backend Services, which route to pods. A 502 occurs when: no pods are running or ready for the backend Service,

	the Service port does not match the pod's containerPort, pods are failing health checks, and the backend application is returning an invalid HTTP response.
SOLUTION	Check backend pod status: kubectl get pods -l <service-selector>. Check pod readiness: kubectl describe pod. Verify Service port mapping: kubectl get svc <name> -o yaml. Test direct pod access: kubectl exec -it <debug-pod> -- curl <pod-ip>:<port>. Check Ingress controller logs: kubectl logs -n <ingress-ns> <ingress-pod>. Verify Ingress backend service name and port match exactly.

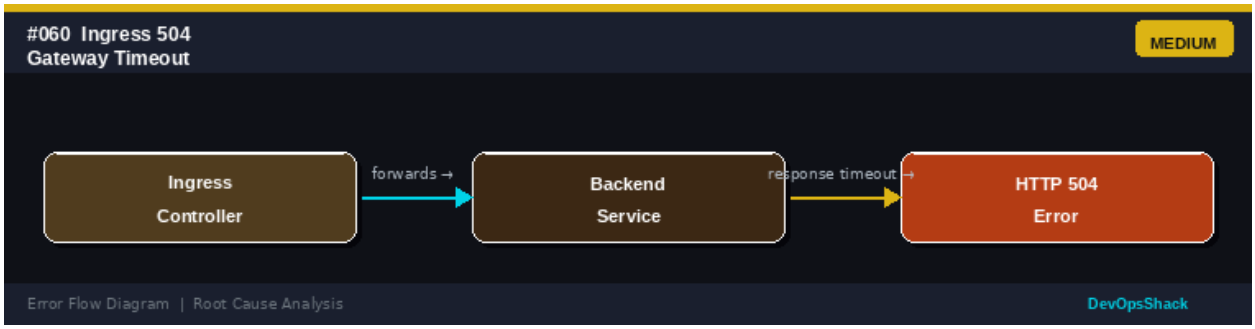
Error Flow Diagram



#060 Ingress 504 Gateway Timeout *Networking* — MEDIUM

DESCRIPTION	The Ingress controller returns HTTP 504 Gateway Timeout. The backend service is reachable but is not responding within the configured timeout period.
ROOT CAUSE	504 errors indicate the backend is too slow, not that it is unavailable (that would be 502). Causes: application processing a slow query or operation, database contention, external API calls blocking the response, and insufficient timeout configured in the Ingress annotation.
SOLUTION	Check application performance with APM tools or pod resource usage: kubectl top pods. Increase Ingress timeout annotations: nginx.ingress.kubernetes.io/proxy-read-timeout: '120'. Check for slow database queries. Review application code for blocking operations. Scale up backend pods to handle load. Check network latency between Ingress controller and backend pods. Review backend application logs for slow operations.

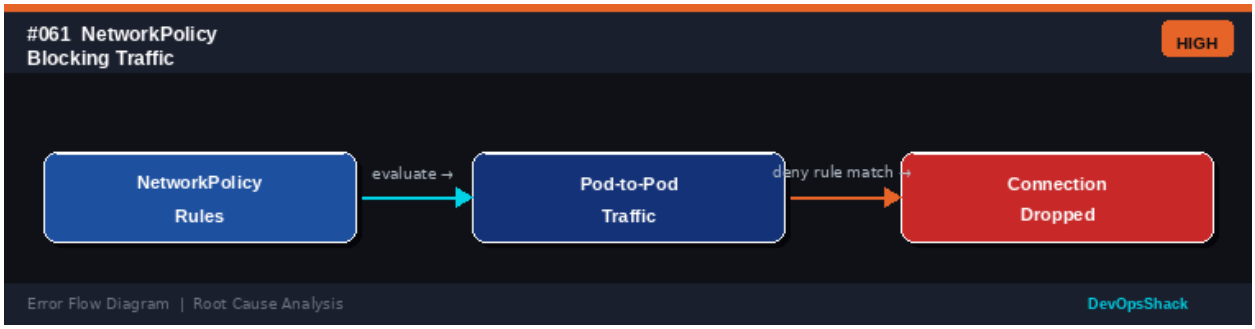
Error Flow Diagram



#061 NetworkPolicy Blocking Traffic *Networking* — HIGH

DESCRIPTION	Pod-to-pod or pod-to-external communication is blocked unexpectedly by a NetworkPolicy, causing connection timeouts or refused connections.
ROOT CAUSE	NetworkPolicies use a default-deny model: if any NetworkPolicy selects a pod, all traffic not explicitly allowed is denied. Traffic is blocked when: policies are too restrictive and do not allow needed communication, egress policies block DNS or external access, and policies in one namespace block cross-namespace traffic.
SOLUTION	Check policies: kubectl get networkpolicy -A. Test connectivity: kubectl exec -it <pod> -- curl <target>. Use a network policy viewer tool (e.g., Cilium network policy editor, calico-node policy inspect). Add specific allow rules for required traffic. For DNS: always allow egress to kube-dns namespace. Test with a temporary allow-all policy to confirm the policy is the cause. Remove overly broad deny-all policies that block legitimate traffic.

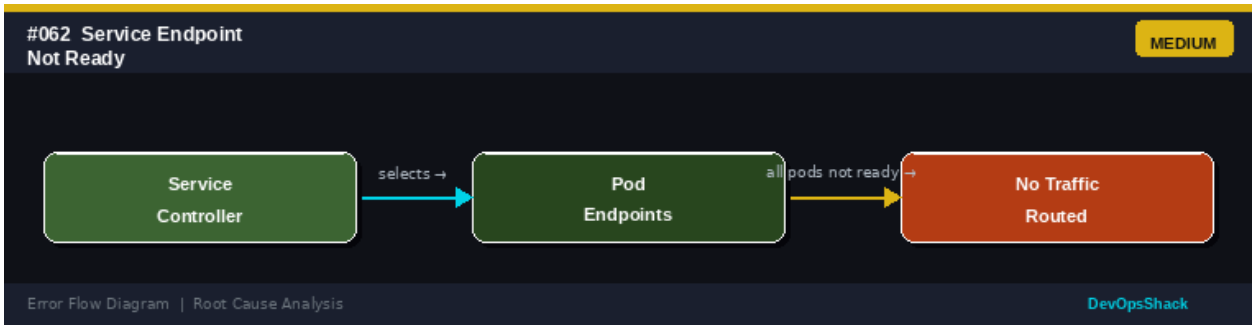
Error Flow Diagram



#062 Service Endpoint Not Ready *Networking* — MEDIUM

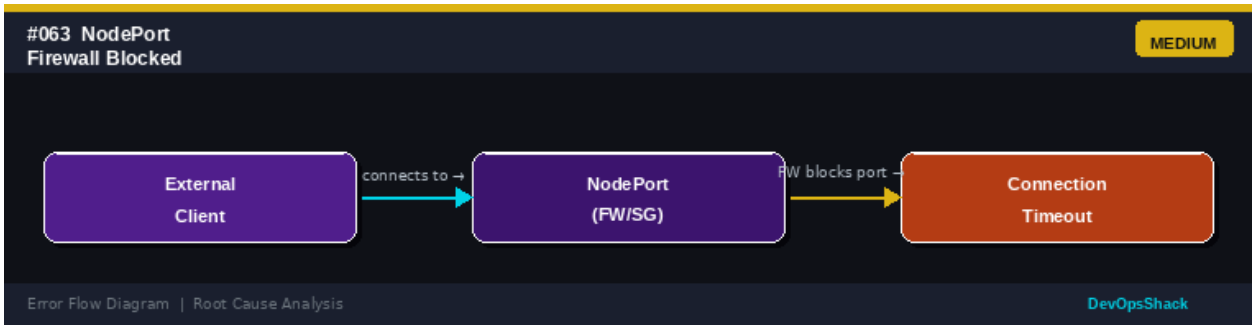
DESCRIPTION	A Service has endpoints listed but they show as NotReady, meaning traffic is not being sent to them. The service appears to work but returns errors.
ROOT CAUSE	Service endpoints are populated only for pods passing their readiness probe. If all pods fail readiness, the endpoint list is empty or all entries show as not ready. Applications can set publishNotReadyAddresses: true to route to not-ready pods, but this is unusual.
SOLUTION	Check endpoint readiness: kubectl get endpoints <service>. Describe the service: kubectl describe service <name>. Check pod readiness: kubectl describe pod <name> | grep Readiness. Fix the readiness probe or the application health check. Check if NetworkPolicy is blocking probe traffic from kubelet to pods. Review pod logs for startup errors. Verify the service selector matches pod labels exactly.

Error Flow Diagram



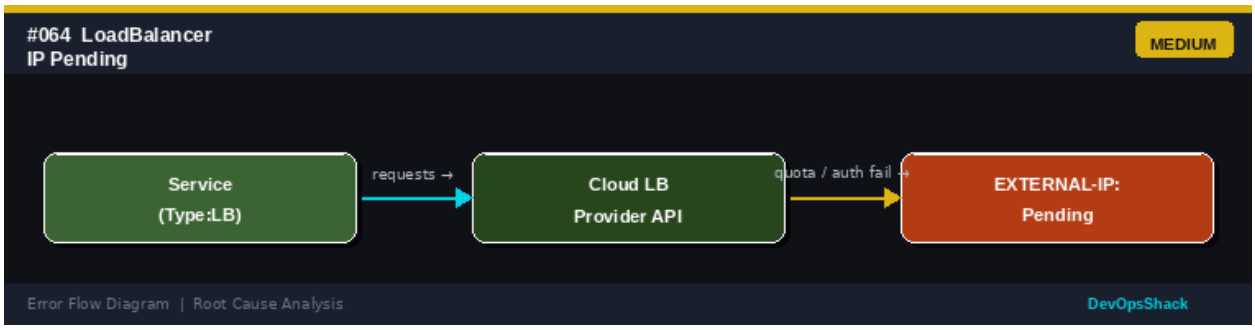
#063	NodePort Firewall Blocked *Networking*	MEDIUM
DESCRIPTION	External clients cannot connect to a NodePort service even though the service and pods are running correctly.	
ROOT CAUSE	NodePort services expose ports in the range 30000-32767 on every node's external IP. External traffic is blocked when: cloud security groups do not allow the NodePort range, host firewall (iptables, firewalld, UFW) blocks the port, and the NodePort falls outside the allowed range in kube-apiserver configuration.	
SOLUTION	Check cloud security group rules to allow the NodePort range (30000-32767). Verify host firewall: iptables -L -n | grep <port>. Test from within the cluster first: kubectl exec -it <pod> -- curl <node-ip>:<nodeport>. Check kube-proxy iptables rules: iptables -t nat -L KUBE-SERVICES. Use kubectl port-forward for debugging access without firewall concerns. Consider using Ingress or LoadBalancer instead of NodePort for production.	

Error Flow Diagram



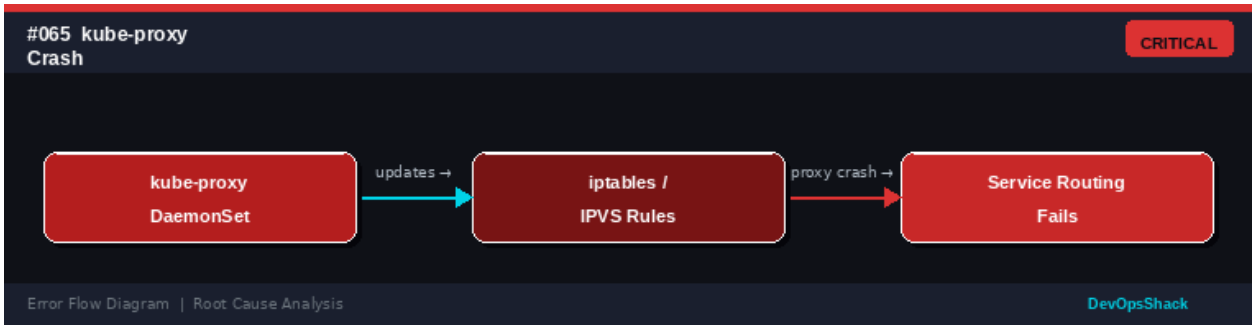
#064	LoadBalancer IP Pending *Networking*	MEDIUM
DESCRIPTION	A Service with type: LoadBalancer shows EXTERNAL-IP as <pending> indefinitely. No external IP is assigned.	
ROOT CAUSE	LoadBalancer services require cloud provider integration to provision an actual load balancer and assign an IP. This fails when: cloud provider integration (CCM) is not configured, cloud quota for load balancers is exhausted, IAM permissions are insufficient, and in bare-metal clusters without a load balancer controller (MetalLB, etc.).	
SOLUTION	Check cloud controller manager: kubectl get pods -n kube-system | grep cloud-controller. Review CCM logs for provisioning errors. Check cloud LB quota in provider console. For bare-metal, install MetalLB: kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/<version>/config/manifests/metallb-native.yaml and configure an IP pool. For testing, use NodePort or kubectl port-forward instead.	

Error Flow Diagram



#065	kube-proxy Crash *Networking*	CRITICAL
DESCRIPTION	kube-proxy is not running or crashing, causing Service routing to break. New Services are not reachable and existing iptables rules become stale.	
ROOT CAUSE	kube-proxy watches the API server for Service and Endpoint changes and configures iptables or IPVS rules accordingly. It crashes due to: OOM kill, API server connectivity loss, permission errors, kernel incompatibility with iptables mode, and race conditions in iptables rule management.	
SOLUTION	Check kube-proxy DaemonSet: kubectl get pods -n kube-system -l k8s-app=kube-proxy. Check logs: kubectl logs -n kube-system <kube-proxy-pod>. Restart kube-proxy: kubectl rollout restart ds/kube-proxy -n kube-system. Consider switching to IPVS mode for better performance and stability. Verify kernel modules: modprobe ip_vs ip_vs_rr ip_vs_wrr. Clean stale iptables rules after kube-proxy recovery.	

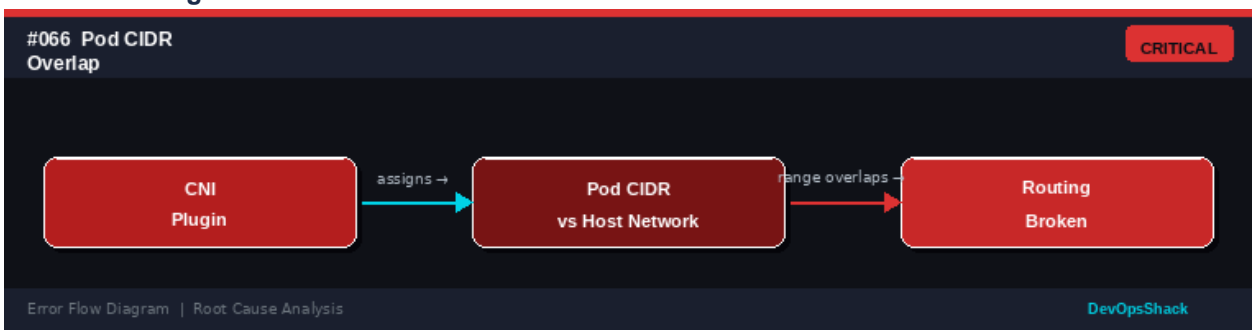
Error Flow Diagram



#066	Pod CIDR Overlap *Networking*	CRITICAL
DESCRIPTION	Pod-to-pod communication is broken because the pod network CIDR overlaps with the host network or another network, causing routing conflicts.	
ROOT CAUSE	The pod CIDR (e.g., 10.244.0.0/16) must not overlap with the node network, service CIDR, or any corporate network that nodes can reach. Overlap causes routing table conflicts where traffic meant for pods gets routed to the host network or vice versa.	
SOLUTION	Check pod CIDR: kubectl cluster-info dump | grep -i cidr. Check for overlaps with host network: ip route show. This is a cluster design issue requiring remediation at the network level. Short-term: update routing rules to prefer pod network	

	routes. Long-term: redesign CIDRs with non-overlapping ranges. This may require cluster recreation. Use a separate RFC1918 range or RFC4193 for pod networks.

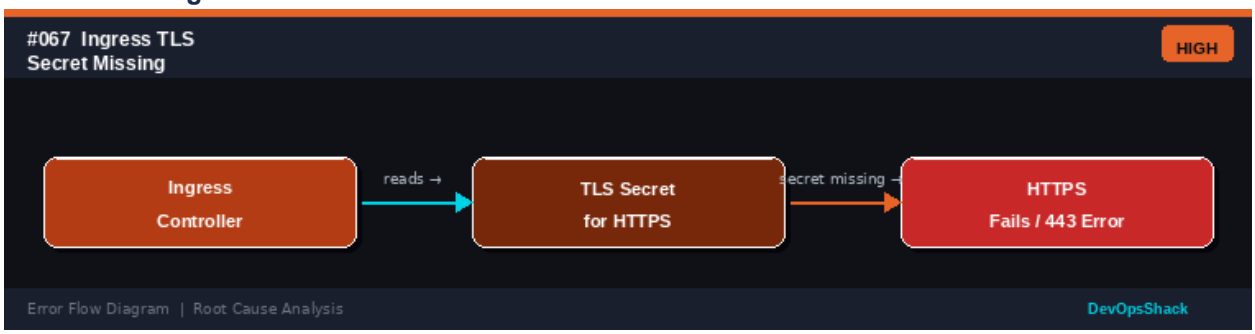
Error Flow Diagram



#067 Ingress TLS Secret Missing *Networking* — HIGH

DESCRIPTION	HTTPS traffic to an Ingress resource fails because the TLS Secret referenced in the Ingress spec does not exist in the namespace.
ROOT CAUSE	Ingress resources reference TLS Secrets for HTTPS termination: spec.tls[].secretName must point to a Secret with tls.crt and tls.key. If the Secret is missing, the Ingress controller uses its default certificate or returns SSL errors. The Secret must be in the same namespace as the Ingress.
SOLUTION	Create the TLS Secret: kubectl create secret tls <name> --cert=tls.crt --key=tls.key -n <namespace>. Use cert-manager to automate TLS certificate provisioning with Let's Encrypt. Verify Secret type is kubernetes.io/tls. Check that cert-manager Certificate resource is healthy and not in error state. Ensure the secret name matches exactly what is referenced in the Ingress spec.

Error Flow Diagram

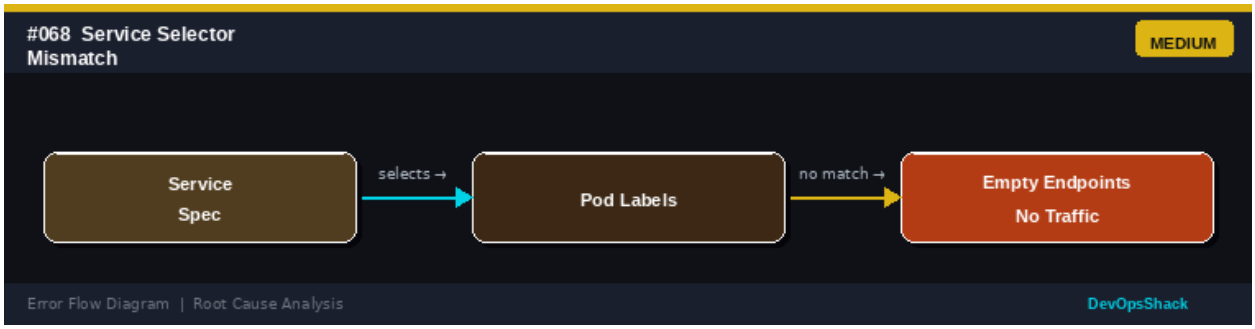


#068 Service Selector Mismatch *Networking* — MEDIUM

DESCRIPTION	A Service has no endpoints or is routing to wrong pods because the selector labels do not match any pod labels (or match unintended pods).
ROOT CAUSE	Service selectors use label matching to identify backend pods. Mismatches occur due to: typos in label keys or values, pod labels changed by a rollout without

	updating the Service selector, using different label schemas in different environments, and forgetting to add labels to pods.
SOLUTION	Compare service selector with pod labels: kubectl get svc <name> -o jsonpath='{.spec.selector}' and kubectl get pods --show-labels. Fix the selector in the Service or add/fix labels on pods. Run: kubectl get endpoints <service> to verify endpoints are populated. Use kubectl label pods --selector <old-label> <new-label>=<value> to bulk update pod labels. Adopt a consistent labeling strategy.

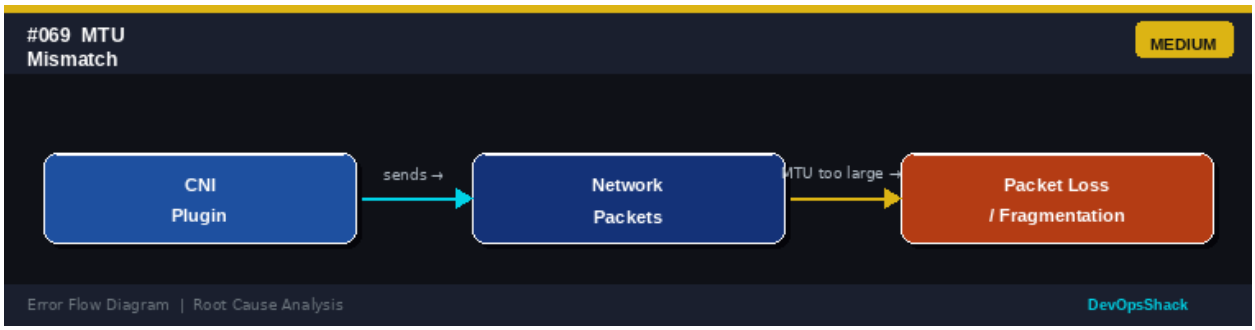
Error Flow Diagram



#069 MTU Mismatch *Networking* — MEDIUM

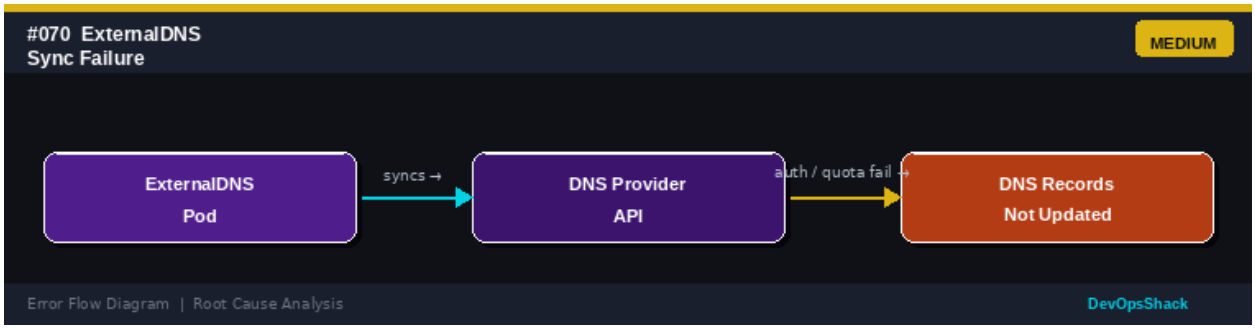
DESCRIPTION	Network communication between pods is unreliable, with random packet loss, connection hangs, or large request failures. Small requests work but large ones do not.
ROOT CAUSE	MTU (Maximum Transmission Unit) mismatch occurs when the CNI plugin uses a different MTU than the underlying network. Packets larger than the MTU are fragmented or dropped. This is common with overlay networks (VXLAN adds 50 bytes overhead) when nodes have MTU=1500. Symptoms: ICMP ping works, but large TCP transfers fail.
SOLUTION	Check node MTU: ip link show | grep mtu. Check CNI MTU configuration. Set CNI MTU to 50 bytes less than physical MTU for overlay networks: For Flannel, set --iface-mtu in config. For Calico, set mtu in installation spec. For Cilium, set --mtu flag. Enable jumbo frames on the underlying network (MTU 9000) to avoid overhead issues. Test with: ping -M do -s 1450 <pod-ip>.

Error Flow Diagram



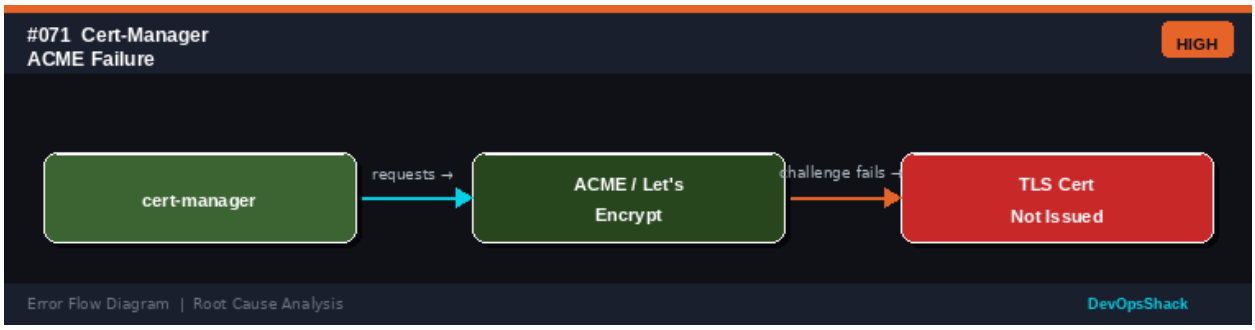
#070	ExternalDNS Sync Failure *Networking*	MEDIUM
DESCRIPTION	ExternalDNS is not updating DNS records for Services and Ingresses, causing external domain names to point to old IPs or not exist.	
ROOT CAUSE	ExternalDNS synchronizes Kubernetes Service/Ingress annotations with external DNS providers (Route53, CloudDNS, etc.). Sync fails when: cloud provider API credentials are invalid or expired, rate limiting from the DNS provider, IAM permissions insufficient for the DNS zone, and the DNS zone ID or domain filter is misconfigured.	
SOLUTION	Check ExternalDNS pod logs: kubectl logs -n <ns> <externaldns-pod>. Verify IAM permissions include DNS record management. Check the DNS provider API credentials Secret. Add the correct annotation to services: external-dns.alpha.kubernetes.io/hostname: myapp.example.com. Verify the zone ID and domain filter in ExternalDNS deployment args. Test with --dry-run=client first.	

Error Flow Diagram



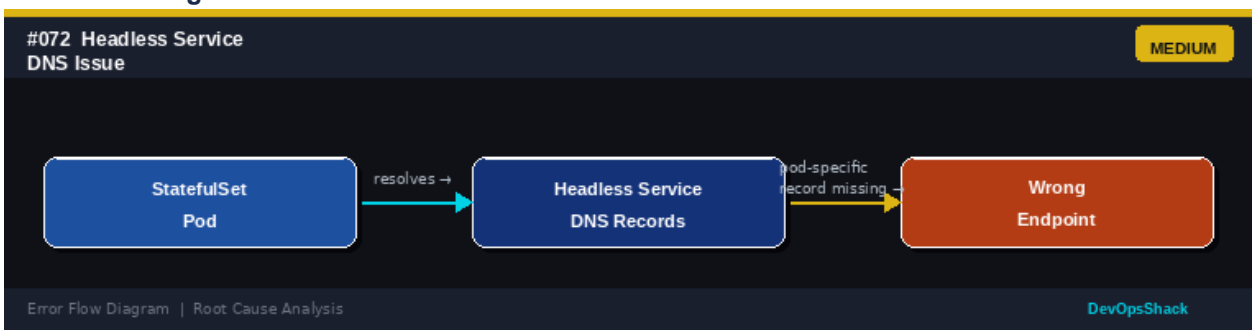
#071	Cert-Manager ACME Failure *Networking*	HIGH
DESCRIPTION	cert-manager cannot provision TLS certificates from Let's Encrypt or another ACME provider. Certificate resources show a Failed or Pending state.	
ROOT CAUSE	ACME certificate issuance requires completing a challenge to prove domain ownership. HTTP-01 challenges need the domain to be publicly accessible on port 80. DNS-01 challenges need DNS provider API access. Failures occur when: the domain is not publicly accessible, DNS-01 API credentials are invalid, challenge solver pod fails to start, and rate limits are exceeded.	
SOLUTION	Check Certificate status: kubectl describe certificate <name>. Check CertificateRequest and Order objects. For HTTP-01: ensure port 80 is accessible from the internet and the challenge ingress is created. For DNS-01: verify API credentials Secret and IAM permissions. Check for Let's Encrypt rate limits (5 failures per hour per domain). Use staging issuer for testing. Check cert-manager pod logs: kubectl logs -n cert-manager <pod>.	

Error Flow Diagram



#072	Headless Service DNS Issue *Networking*	MEDIUM
DESCRIPTION	StatefulSet pods or other workloads using headless Services cannot resolve individual pod DNS names, breaking stateful service discovery.	
ROOT CAUSE	Headless Services (clusterIP: None) return individual pod IPs via DNS rather than a ClusterIP. DNS records like pod-0.service.namespace.svc.cluster.local should resolve to pod-0's IP. Issues arise when: pods are not ready (readiness check failing), the pod hostname does not match the StatefulSet pod name pattern, and DNS TTL caching delays are causing stale records.	
SOLUTION	Check DNS resolution from a pod: nslookup pod-0.service.default.svc.cluster.local. Verify pod readiness. Check that StatefulSet pods have the correct subdomain matching the headless service name. Verify the Service selector matches pod labels. Check CoreDNS logs for resolution errors. Use publishNotReadyAddresses: true if pods need to discover each other before becoming ready (e.g., cluster formation).	

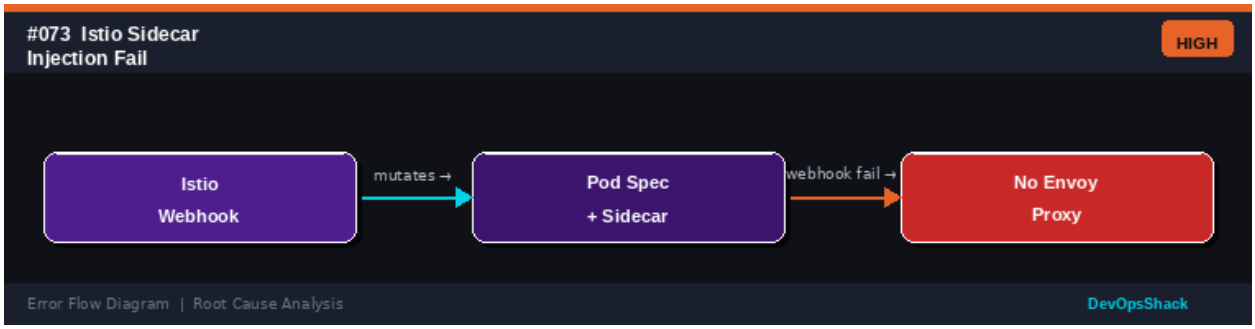
Error Flow Diagram



#073	Istio Sidecar Injection Failure *Networking*	HIGH
DESCRIPTION	Pods are deployed without the Istio Envoy sidecar proxy, breaking mTLS, traffic management, and observability features.	
ROOT CAUSE	Istio uses a MutatingAdmissionWebhook to inject the sidecar. Injection fails when: the istiod pod is not running, the namespace lacks the istio-injection: enabled label, the pod has the sidecar.istio.io/inject: 'false' annotation, the webhook certificate is expired, and the Istio revision mismatch between label and istiod version.	

SOLUTION	Verify namespace label: kubectl get ns <name> --show-labels | grep istio. Check istiod is running: kubectl get pods -n istio-system. Restart istiod to refresh webhook: kubectl rollout restart deployment/istiod -n istio-system. Check webhook config: kubectl get mutatingwebhookconfigurations. For revision-based install, use istio.io/rev=<revision> label. Check istiod TLS cert validity.

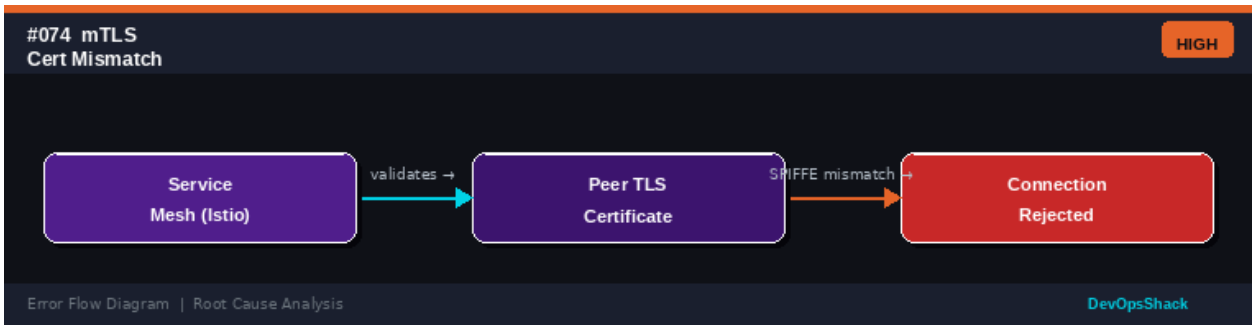
Error Flow Diagram



#074 mTLS Certificate Mismatch *Networking* — HIGH

DESCRIPTION	Service mesh mutual TLS authentication is failing between services. Connections are rejected with TLS handshake errors or RBAC denial from the mesh.
ROOT CAUSE	Mutual TLS in Istio/Linkerd uses SPIFFE identities (X.509 SVIDs). Mismatches occur when: pods from different trust domains try to communicate, workload certificates have expired (istiod rotation issue), PeerAuthentication policy requires STRICT mTLS but one side is not meshed, and certificate rotation is failing.
SOLUTION	Check Istio auth policy: kubectl get peerauthentication -A. Inspect certificates in the Envoy proxy: istioctl proxy-config secret <pod>. Ensure both communicating pods are in the mesh (have sidecars). Check istiod certificate rotation logs. Use istioctl analyze to detect mTLS policy issues. For migration, use PERMISSIVE mode temporarily. Verify trust domain configuration in IstioOperator or MeshConfig.

Error Flow Diagram

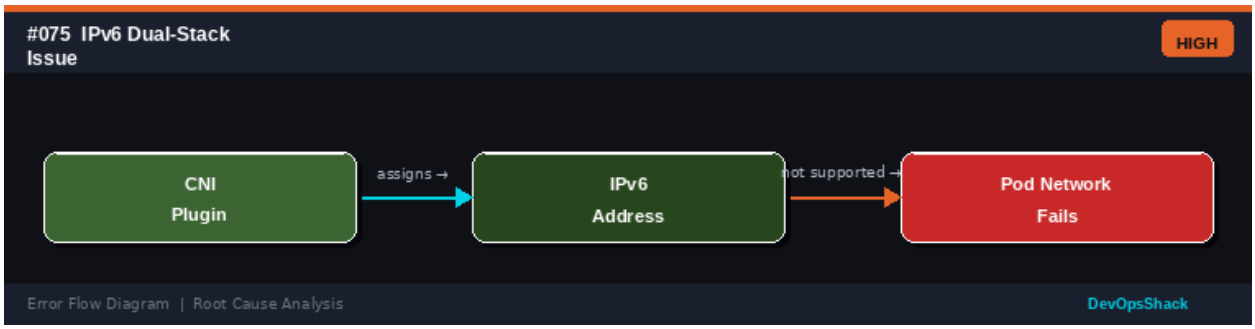


#075 IPv6 Dual-Stack Issue *Networking* — HIGH

DESCRIPTION	Pods or services cannot communicate over IPv6, or dual-stack is enabled but IP assignment is failing for one IP family.

ROOT CAUSE	Kubernetes dual-stack requires cluster-level configuration (--cluster-cidr, --service-cluster-ip-range for both families) and CNI support. Issues arise when: the CNI does not support dual-stack, IPv6 is disabled at the OS or cloud level, Service spec does not specify ipFamilies, and Pod CIDR allocation fails for IPv6.
SOLUTION	Verify dual-stack is configured: kubectl cluster-info dump | grep -i cidr. Check if IPv6 is enabled on nodes: sysctl net.ipv6.conf.all.disable_ipv6. Verify CNI supports dual-stack. Set Service ipFamilyPolicy: PreferDualStack or RequireDualStack. Check IPv6 routes on nodes. Test IPv6 connectivity: ping6 <ipv6-address>. Cloud providers require specific VPC/subnet configuration for IPv6.

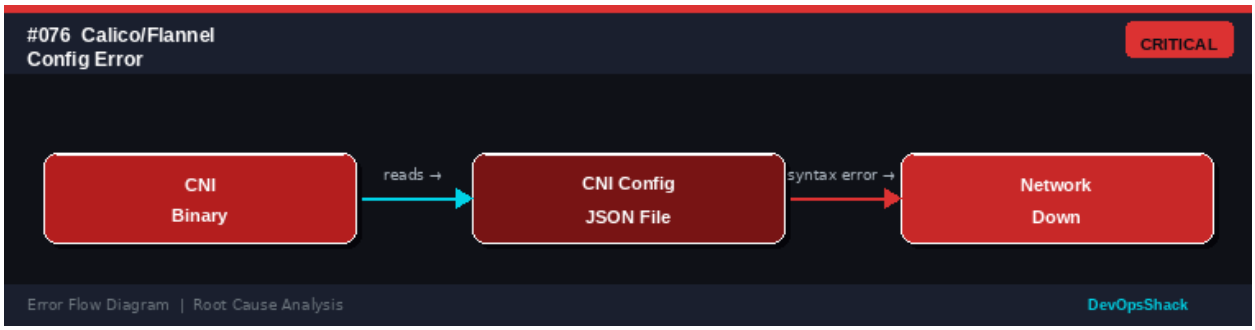
Error Flow Diagram



#076 Calico/Flannel Config Error *Networking* — CRITICAL

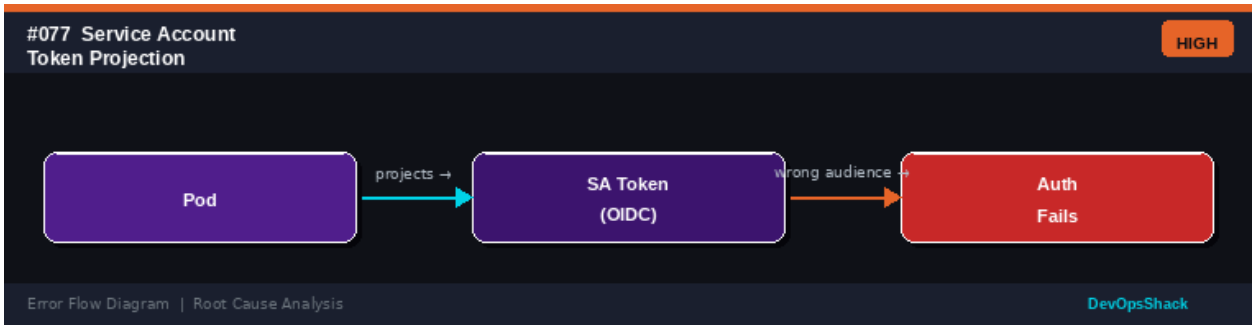
DESCRIPTION	The CNI plugin (Calico or Flannel) has a configuration error causing pod network setup to fail across the cluster or on specific nodes.
ROOT CAUSE	CNI plugin configuration is stored as JSON files in /etc/cni/net.d/. Syntax errors, incompatible CIDR settings, wrong backend type, and version mismatches between the CNI plugin and Kubernetes can cause network initialization failures.
SOLUTION	Check CNI config file: cat /etc/cni/net.d/*.conflist. Validate JSON syntax. For Calico, check the installation resource: kubectl get installation default -o yaml. For Flannel, check the ConfigMap: kubectl get cm kube-flannel-cfg -n kube-flannel. View CNI pod logs on the failing node. Re-apply CNI manifests after fixing configuration. Check the CNI spec version compatibility with your Kubernetes version.

Error Flow Diagram



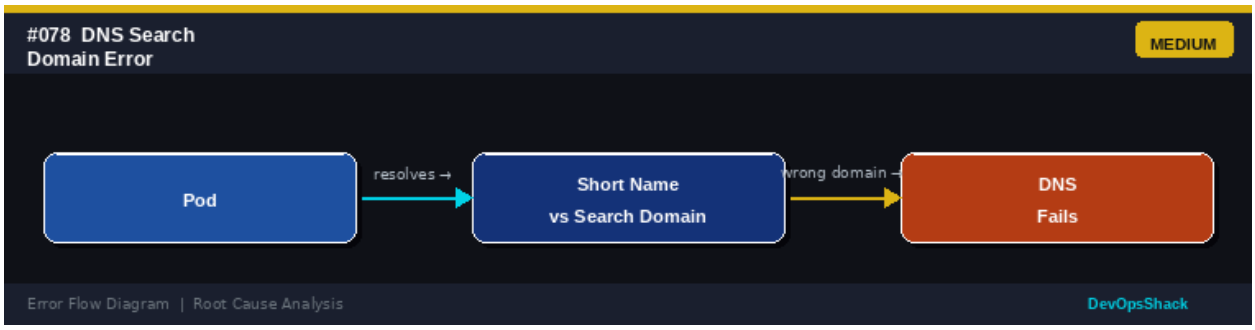
#077	Service Account Token Projection Error *Networking*	HIGH
DESCRIPTION	A pod using projected service account tokens for cloud provider authentication (IRSA on AWS, Workload Identity on GCP) fails with invalid audience or token errors.	
ROOT CAUSE	Projected Service Account Tokens (PSAT) include audience and expiry. Cloud provider IAM systems validate the audience must match the expected value. Failures occur when: the ServiceAccount is not annotated with the cloud IAM role, the OIDC provider is not configured, the token audience does not match the cloud provider's expected value, and the token has expired.	
SOLUTION	For AWS IRSA: annotate the ServiceAccount: eks.amazonaws.com/role-arn: <role-arn>. Verify the OIDC provider is configured in IAM. For GCP Workload Identity: annotate with iam.gke.io/gcp-service-account. Verify projected volume mount path and audience: spec.volumes.projected.sources.serviceAccountToken.audience. Check token: kubectl exec -it <pod> -- cat /var/run/secrets/kubernetes.io/serviceaccount/token | jwt decode.	

Error Flow Diagram



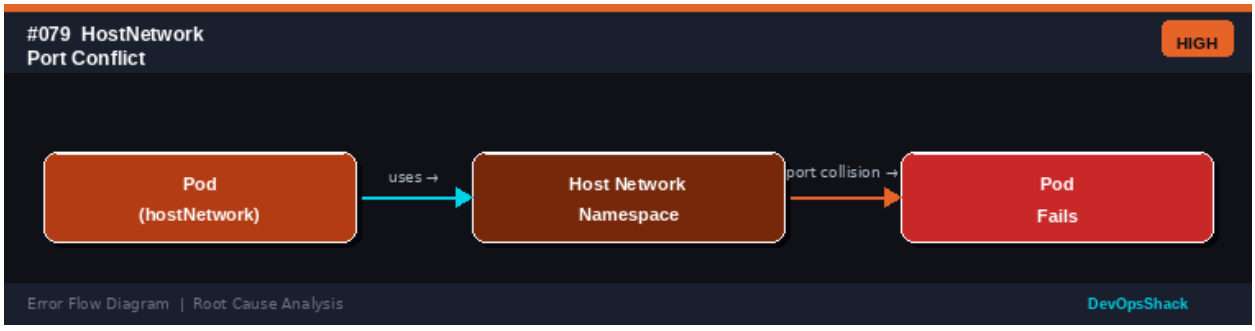
#078	DNS Search Domain Error *Networking*	MEDIUM
DESCRIPTION	Pods fail to resolve short service names (e.g., myservice instead of myservice.mynamespace.svc.cluster.local) because the DNS search domains in /etc/resolv.conf are incorrect.	
ROOT CAUSE	Kubernetes configures /etc/resolv.conf with search domains: <namespace>.svc.cluster.local, svc.cluster.local, cluster.local. If the cluster domain is not set correctly, or dnsConfig in the pod overrides search domains incorrectly, short-name resolution fails.	
SOLUTION	Check DNS config in a pod: kubectl exec -it <pod> -- cat /etc/resolv.conf. Verify cluster domain matches what CoreDNS is configured with. Check CoreDNS Corefile: kubectl get cm coredns -n kube-system -o yaml. Fix pod dnsConfig if it incorrectly overrides search domains. Use fully qualified domain names (FQDN) as a workaround. Ensure kubelet --cluster-domain matches CoreDNS domain zone.	

Error Flow Diagram



#079	HostNetwork Port Conflict *Networking*	HIGH
DESCRIPTION	A pod using hostNetwork: true fails to start because another process or pod on the host is already using the same port.	
ROOT CAUSE	When hostNetwork: true is set, the pod shares the node's network namespace and can bind to host ports directly. Only one process can own a port at a time. Conflicts occur with other hostNetwork pods, node-level system services (kubelet, containerd metrics), and previously-crashed pods that didn't release ports.	
SOLUTION	Identify port conflicts: ss -tulpn | grep <port> on the affected node. Check for other hostNetwork pods: kubectl get pods --all-namespaces -o json | jq '.items[] | select(.spec.hostNetwork==true)'. Change the conflicting application to use a different port. Avoid hostNetwork when possible, use Services instead. Use DaemonSet with specific node affinity to ensure only one instance per node.	

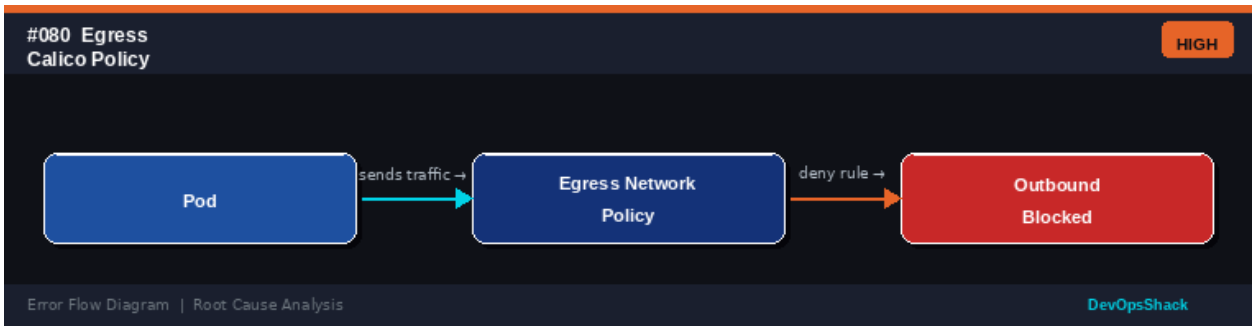
Error Flow Diagram



#080	Egress Calico NetworkPolicy Block *Networking*	HIGH
DESCRIPTION	Pods cannot reach external services, databases, or the internet due to overly restrictive Calico NetworkPolicy egress rules.	
ROOT CAUSE	Calico (and Kubernetes NetworkPolicies) implement a default-deny model for pods selected by a policy. Egress policies that do not explicitly allow DNS (port 53), required external endpoints, or all egress traffic will silently drop outbound connections.	
SOLUTION	Check policies: kubectl get networkpolicy -A. Identify which policy selects the pod. Add explicit egress rules. Always add DNS egress allowance: to kube-system pods with k8s-app: kube-dns, port 53. Use Calico GlobalNetworkPolicy	

for cluster-wide rules. Test with a temporary allow-all policy. Use calicoctl or cilium CLI to trace packet paths. Enable Calico flow logs for visibility.

Error Flow Diagram



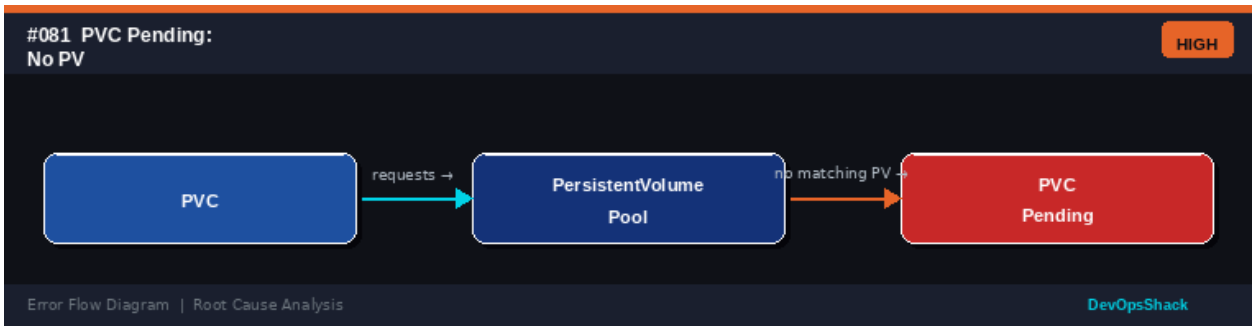
Storage Errors

Errors #081 – #100

Storage errors involve PersistentVolumes, PersistentVolumeClaims, StorageClasses, and CSI drivers. These issues can prevent pods from starting, cause data access problems, or lead to data loss if not handled correctly.

#081	PVC Pending - No PV *Storage*	HIGH
DESCRIPTION	A PersistentVolumeClaim (PVC) remains in Pending state because no available PersistentVolume (PV) matches the claim's requirements.	
ROOT CAUSE	Static provisioning requires manually creating PVs that match the PVC's storageClassName, accessModes, and size. Dynamic provisioning requires a working StorageClass with a provisioner. Failures occur when: no matching PV exists, the storageClassName does not exist, provisioner pods are not running, and cloud storage quotas are exceeded.	
SOLUTION	Check PVC status: kubectl describe pvc <name>. For static provisioning, create a PV that matches the PVC's storage class, access mode, and size. For dynamic provisioning, verify the StorageClass exists and the provisioner is running. Check provisioner logs. Verify cloud storage quota. For immediate binding vs. WaitForFirstConsumer: set volumeBindingMode appropriately in StorageClass.	

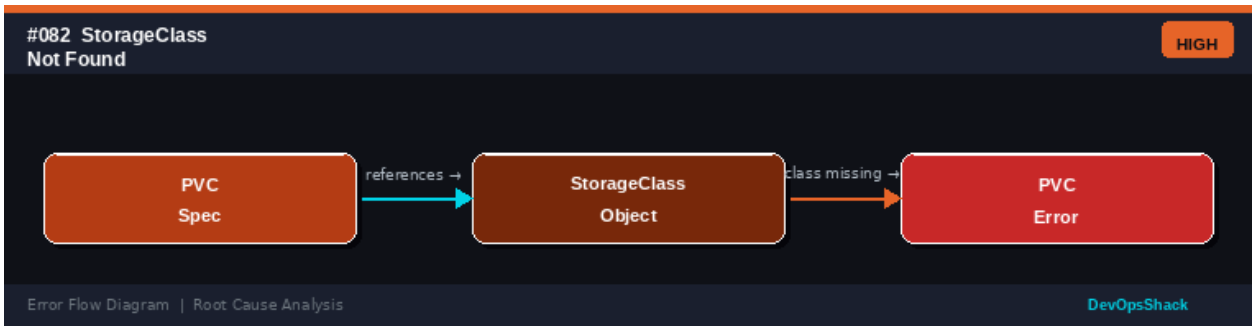
Error Flow Diagram



#082	StorageClass Not Found *Storage*	HIGH
DESCRIPTION	A PVC references a StorageClass that does not exist in the cluster. The PVC remains in Pending state with a ProvisioningFailed event.	
ROOT CAUSE	StorageClasses define how storage is dynamically provisioned. A missing StorageClass occurs when: the cluster was not set up with the required StorageClass, the name was misspelled in the PVC, the StorageClass was accidentally deleted, and a different cluster (dev/prod) has different StorageClasses.	
SOLUTION	Check available StorageClasses: kubectl get storageclass. Create the required StorageClass if it is missing. Verify the provisioner binary is installed (e.g., EBS CSI driver for AWS). Set a default StorageClass: kubectl patch storageclass	

	<name> -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'. Fix PVC manifests to use the correct StorageClass name.

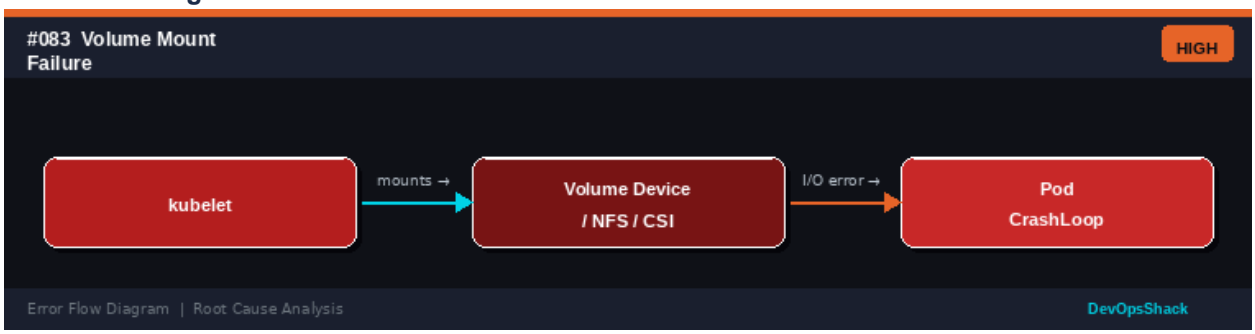
Error Flow Diagram



#083 Volume Mount Failure *Storage* — HIGH

DESCRIPTION	A pod is stuck in ContainerCreating state because a volume cannot be mounted. The pod events show FailedMount errors.
ROOT CAUSE	Volume mount failures occur when: the volume device is not attached to the node, the filesystem has errors requiring fsck, the mount target directory does not exist, SELinux or AppArmor is blocking the mount, NFS share is inaccessible, and the CSI driver is not installed or malfunctioning.
SOLUTION	Check pod events: kubectl describe pod <name> | grep -A20 Events. For cloud volumes, check attach status: kubectl describe volumeattachment. Check node logs for mount errors: journalctl -u kubelet | grep mount. Run fsck on the volume if filesystem errors are suspected. Verify NFS server is running and accessible. Re-install CSI driver if it is the cause. Force-detach volumes stuck in attaching state.

Error Flow Diagram

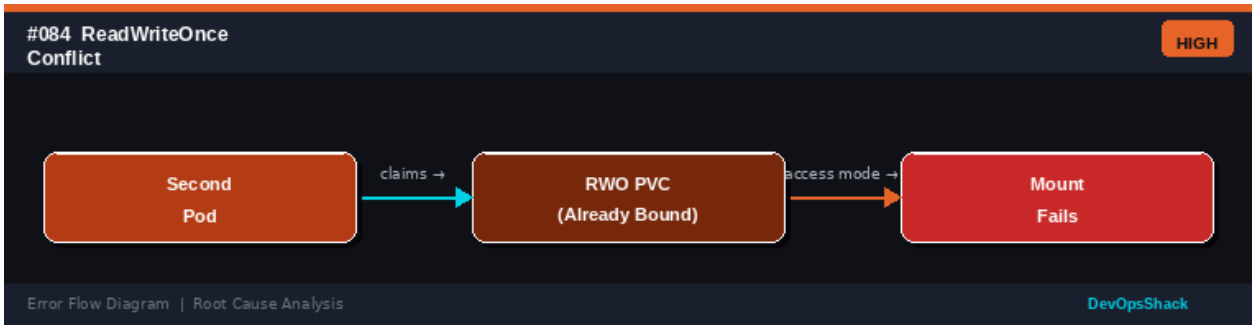


#084 ReadWriteOnce PVC Conflict *Storage* — HIGH

DESCRIPTION	A pod fails to start because the PVC it needs is already bound and mounted as ReadWriteOnce (RWO) to another node. Only one node can mount an RWO volume at a time.
ROOT CAUSE	RWO PVCs can only be mounted on a single node. This is enforced by the cloud storage layer (e.g., EBS, GCE PD). When a Deployment with RWO PVCs scales

	or moves to a different node, the old node must release the volume before the new pod can mount it. This causes a race condition during pod migrations.
SOLUTION	For workloads needing multi-node access, use ReadWriteMany (RWX) storage class (e.g., EFS, NFS, Ceph). For RWO workloads, use StatefulSets to ensure one pod per PVC. Check for stuck volume detach: kubectl describe pv <name>. Enable StorageClass volumeBindingMode: WaitForFirstConsumer to bind PVC to the correct node first. Force-detach the volume from the old node if it is not in use.

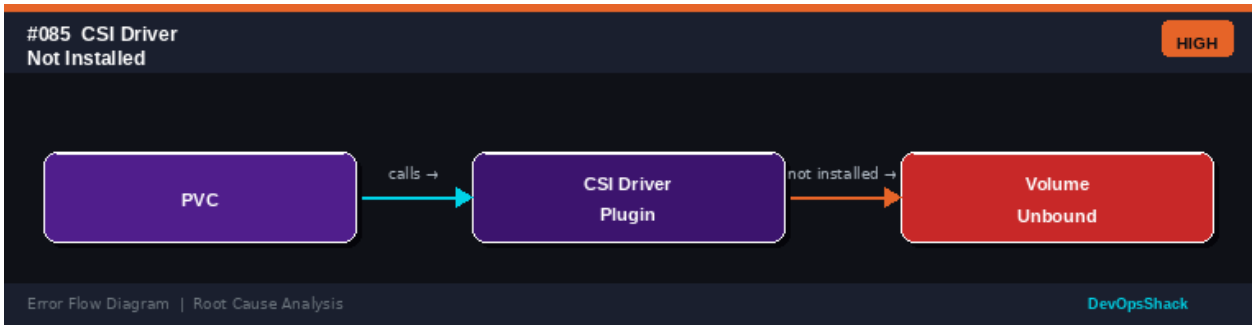
Error Flow Diagram



#085 CSI Driver Not Installed *Storage* — HIGH

DESCRIPTION	Volumes using a CSI (Container Storage Interface) driver cannot be provisioned or attached because the CSI driver is not installed in the cluster.
ROOT CAUSE	CSI drivers are separate installations (typically DaemonSets and Deployments) that implement the CSI spec for specific storage backends. If the CSI driver is not installed, all volume operations fail. This is common when: the cluster was newly created, the CSI driver was removed, and when migrating from in-tree plugins to CSI.
SOLUTION	Check CSI drivers: kubectl get csidrivers. Install the appropriate CSI driver for your storage backend (e.g., aws-ebs-csi-driver, gcp-pd-csi-driver, csi-nfs-driver). Follow the driver's installation guide. Verify CSI controller and node pods are running. Check feature gate CSIMigration if migrating from in-tree plugins. Review StorageClass provisioner field to match the installed CSI driver name.

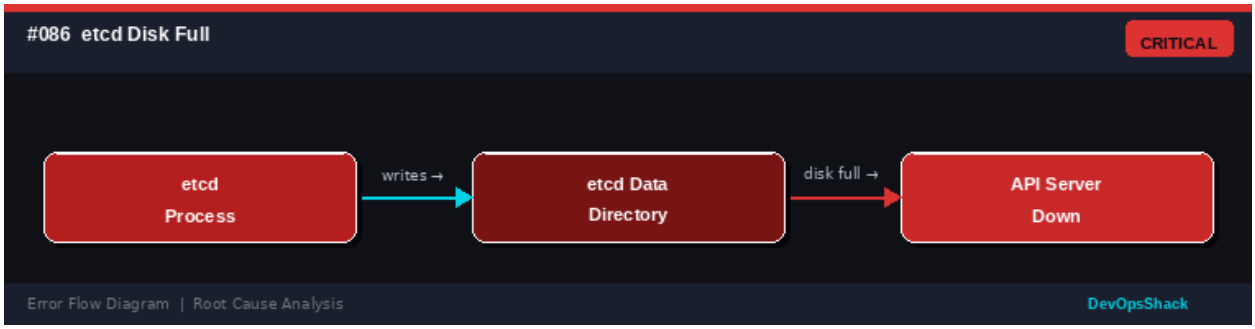
Error Flow Diagram



#086 etcd Disk Full *Storage* — CRITICAL

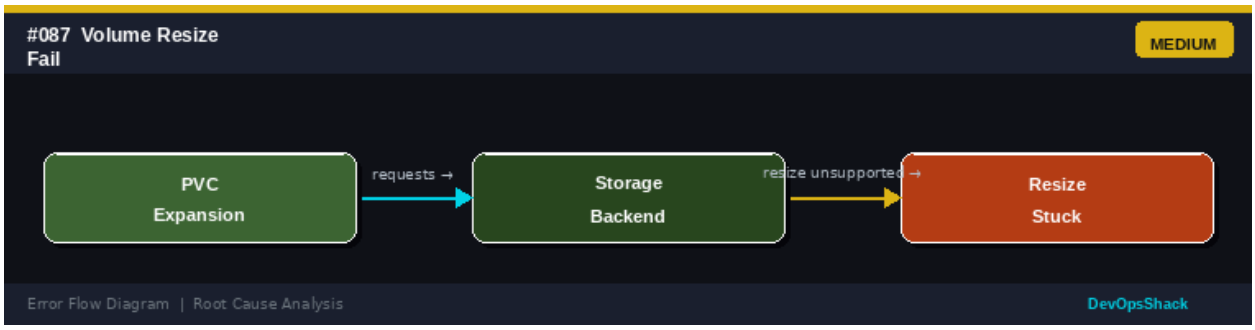
DESCRIPTION	The etcd cluster runs out of disk space, causing the API server to become unavailable. No reads or writes to the cluster are possible.
ROOT CAUSE	etcd stores all cluster state in a key-value store on disk. etcd data grows over time due to: accumulation of Kubernetes object history (Secrets, ConfigMaps), Watch events not being compacted, large objects (large ConfigMaps or Secrets), and high churn of ephemeral objects. Default etcd quota is 2GB.
SOLUTION	Defragment etcd immediately: ETCDCTL_API=3 etcdctl defrag --endpoints=<endpoint>. Compact the revision history: etcdctl compact <latest-rev>. Increase etcd quota: --quota-backend-bytes=8589934592 (8GB). Set up regular etcd compaction and defragmentation (etcd does auto-compaction but may need tuning). Monitor etcd disk usage with Prometheus. Move etcd to a larger disk. Delete unused Kubernetes objects.

Error Flow Diagram



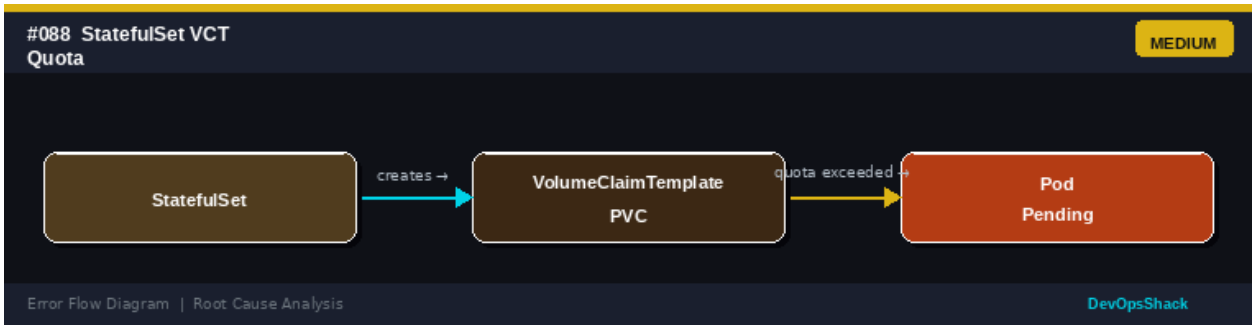
#087	Volume Resize Failure *Storage*	MEDIUM
DESCRIPTION	Expanding a PVC to a larger size does not take effect or returns an error. The volume remains at the original size.	
ROOT CAUSE	Volume resizing requires: the StorageClass to have allowVolumeExpansion: true, the underlying storage backend to support online expansion, and the node to resize the filesystem after the block device is expanded. Cloud providers support different resize capabilities and may require pod restart.	
SOLUTION	Check StorageClass: kubectl get sc <name> -o yaml | grep allowVolumeExpansion. If false, create a new StorageClass with allowVolumeExpansion: true and migrate data. Edit the PVC to increase size: kubectl patch pvc <name> -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}'. Check PVC conditions: kubectl describe pvc | grep Conditions. Restart the pod to trigger filesystem resize. For pre-expansion filesystems, expand manually: resize2fs /dev/sdb.	

Error Flow Diagram



#088	StatefulSet VolumeClaimTemplate Quota *Storage*	MEDIUM
DESCRIPTION	A StatefulSet cannot create pods because the PVCs generated from its volumeClaimTemplates are being blocked by ResourceQuota limits on storage.	
ROOT CAUSE	StatefulSets create PVCs automatically using volumeClaimTemplates. Each replica creates a PVC. ResourceQuotas can limit the total storage requests in a namespace. If the quota is exceeded, new PVCs (and therefore new pods) cannot be created.	
SOLUTION	Check quota usage: kubectl describe resourcequota -n <namespace>. View storage usage: kubectl get pvc -n <namespace> | awk '{sum += $4} END {print sum}'. Increase the quota or delete old PVCs from scaled-down StatefulSets (they are retained by default). Note: StatefulSet scale-down does NOT delete PVCs automatically - delete them manually. Request a quota increase from the cluster admin.	

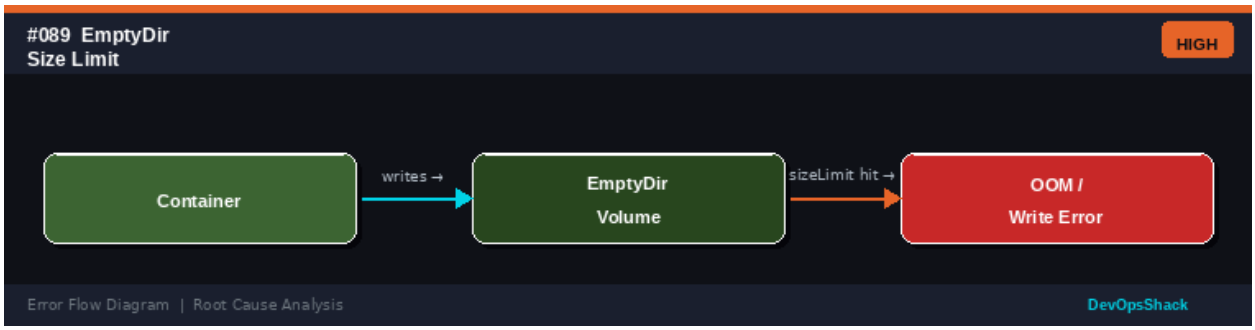
Error Flow Diagram



#089	EmptyDir Size Limit Exceeded *Storage*	HIGH
DESCRIPTION	A container writing to an emptyDir volume is being killed or failing because the written data exceeds the sizeLimit set on the volume.	
ROOT CAUSE	emptyDir volumes share the node's disk and are ephemeral. A sizeLimit enforces a maximum size for the emptyDir (backed by tmpfs in memory or disk). Exceeding the limit causes the kubelet to evict the pod. This is intended behavior when the application writes more temporary data than allocated.	
SOLUTION	Remove or increase the sizeLimit in the pod spec for the emptyDir volume. For tmpfs (in-memory emptyDir), ensure limit is within available node memory. Use a PVC with appropriate storage class for persistent or large temporary data.	

	Optimize the application to write less temporary data or clean up periodically. Monitor emptyDir usage with node metrics.

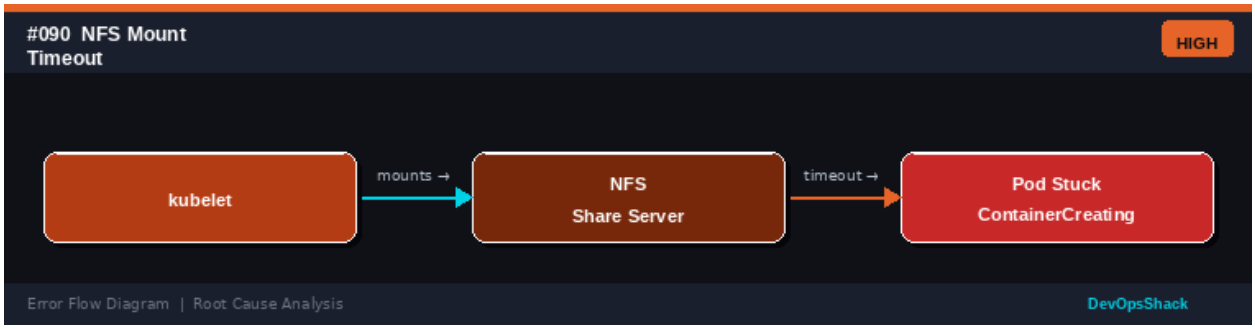
Error Flow Diagram



#090 NFS Mount Timeout *Storage* — HIGH

DESCRIPTION	Pods using NFS volumes are stuck in ContainerCreating state because the NFS mount is timing out. kubelet cannot mount the NFS share.
ROOT CAUSE	NFS mounts require network connectivity to the NFS server, correct export permissions, and the nfs-utils package on nodes. Timeouts occur when: the NFS server is down or overloaded, the NFS export path is incorrect, the node IP is not in the NFS export allow list, and firewall rules block NFS ports (2049/TCP+UDP).
SOLUTION	Test NFS connectivity from the node: showmount -e <nfs-server-ip>. Verify NFS exports are correct: /etc/exports on the NFS server. Ensure nodes can reach NFS on port 2049. Check kubelet NFS mount logs: journalctl -u kubelet | grep nfs. Add the correct NFS options in the PV spec (e.g., nfsvers=4, hard, intr). Consider using a CSI NFS driver instead of direct NFS PV for better management.

Error Flow Diagram

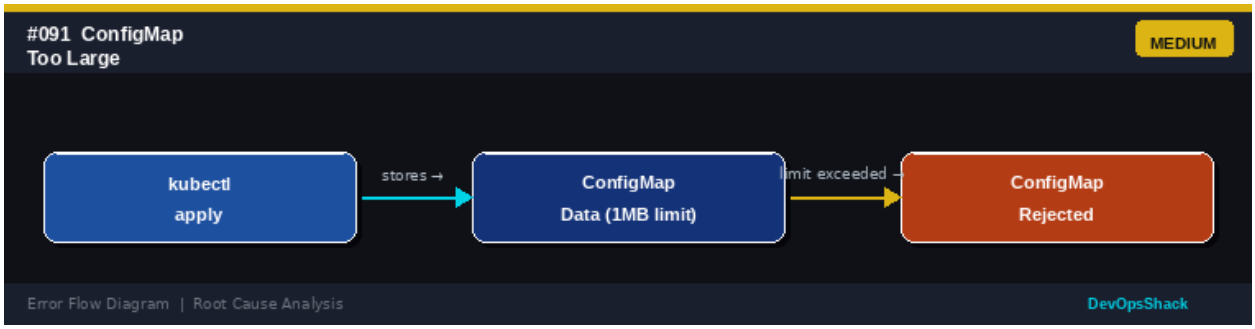


#091 ConfigMap Too Large *Storage* — MEDIUM

DESCRIPTION	A ConfigMap cannot be created or updated because it exceeds the 1MB size limit imposed by Kubernetes/etcd.
ROOT CAUSE	Kubernetes stores ConfigMaps in etcd and enforces a maximum object size. ConfigMaps become too large when: embedding large configuration files, binary data, or certificates, storing large files meant to be in container images,

	accumulating many configuration keys, and using ConfigMaps as a general-purpose object store.
SOLUTION	Check ConfigMap size: kubectl get cm <name> -o json | wc -c. Split large ConfigMaps into multiple smaller ones. Move large binary files to container images or object storage (S3, GCS). Use Secrets for sensitive data separately. Consider using an external configuration service (Vault, AWS SSM). For base64-encoded data, the overhead is significant: use binaryData field instead of data for binary content.

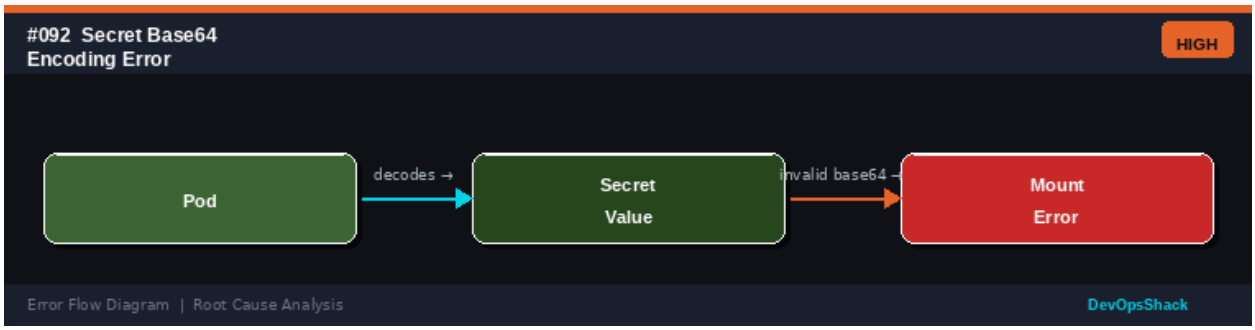
Error Flow Diagram



#092 Secret Base64 Encoding Error *Storage* — HIGH

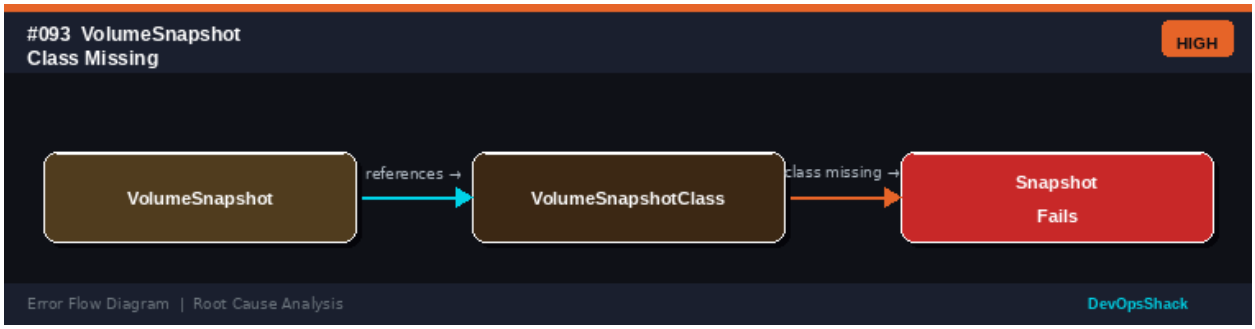
DESCRIPTION	A pod fails to start or a Secret cannot be decoded properly because the Secret value is not correctly base64-encoded.
ROOT CAUSE	Kubernetes Secrets store values as base64-encoded strings. When creating Secrets manually, incorrect encoding causes issues: using kubectl create secret --from-literal automatically encodes, but manually editing YAML may result in double-encoding or incorrect padding. Common mistake: echo <value> | base64 adds a newline that gets encoded.
SOLUTION	Create Secrets using kubectl: kubectl create secret generic <name> --from-literal=key=value (auto-encodes). For manual YAML: use echo -n <value> | base64 (the -n flag removes the trailing newline). Verify encoding: echo <base64-string> | base64 -d. Use stringData instead of data in the Secret YAML to write plain text (Kubernetes encodes automatically). Check the Secret: kubectl get secret <name> -o jsonpath='{.data.<key>}' | base64 -d.

Error Flow Diagram



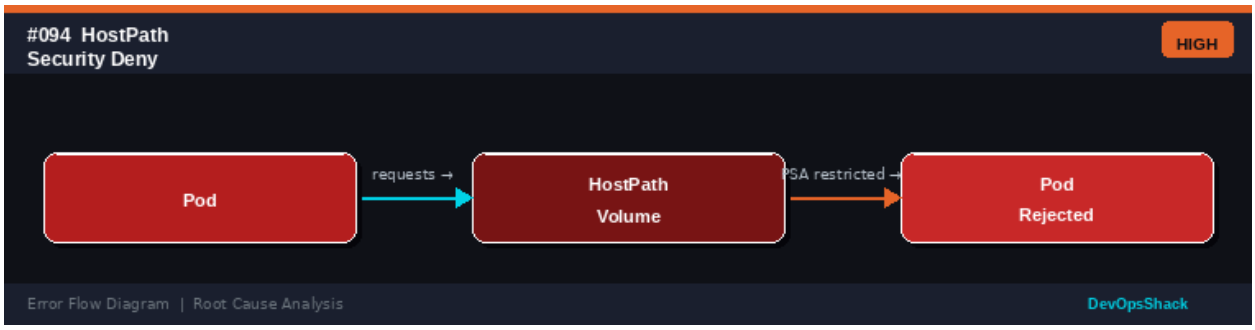
#093	VolumeSnapshot Class Missing *Storage*	HIGH
DESCRIPTION	A VolumeSnapshot cannot be created because the referenced VolumeSnapshotClass does not exist in the cluster.	
ROOT CAUSE	VolumeSnapshots require the VolumeSnapshot CRD (installed with the snapshot-controller) and a VolumeSnapshotClass that matches the CSI driver being used. Missing VolumeSnapshotClass is common when: the snapshot-controller is not installed, the StorageClass and VolumeSnapshotClass driver names do not match, and the feature gate is not enabled (Kubernetes <1.20).	
SOLUTION	Check VolumeSnapshotClasses: kubectl get volumesnapshotclass. Install snapshot-controller: kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/... Create the VolumeSnapshotClass with the correct driver: driver: ebs.csi.aws.com for AWS. Verify the CSI driver supports snapshots. Install snapshot CRDs: VolumeSnapshot, VolumeSnapshotContent, VolumeSnapshotClass.	

Error Flow Diagram



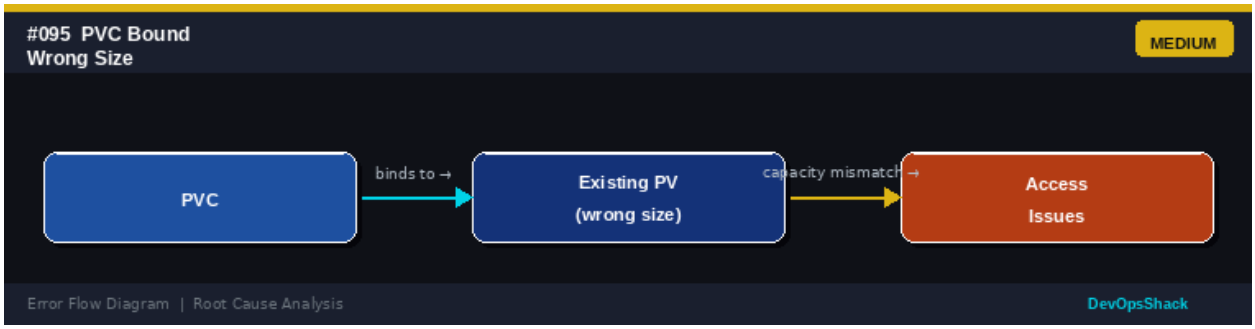
#094	HostPath Security Deny *Storage*	HIGH
DESCRIPTION	A pod requesting a hostPath volume is rejected by the admission controller due to security policy violations.	
ROOT CAUSE	hostPath volumes mount directories from the node's filesystem directly into the pod. This is a significant security risk as it can expose sensitive host files or allow container escape. Pod Security Admission with baseline/restricted profiles blocks hostPath mounts. OPA/Gatekeeper constraints can also block them.	
SOLUTION	Avoid hostPath wherever possible. Use PVCs with appropriate StorageClasses instead. If hostPath is required for system-level workloads (e.g., log collectors): use a privileged namespace, restrict the allowed paths using OPA policies, set readOnly: true. For log collection, use DaemonSets with specific host paths and appropriate tolerations and security contexts.	

Error Flow Diagram



#095	PVC Bound to Wrong Size PV *Storage*	MEDIUM
DESCRIPTION	A PVC is bound to a PersistentVolume that is larger or smaller than expected, causing capacity or billing issues.	
ROOT CAUSE	When multiple PVs are available, the scheduler binds a PVC to the smallest PV that satisfies all requirements. If a PVC requests 10Gi and only a 100Gi PV is available, the 100Gi PV is bound - wasting capacity. The PVC cannot shrink to match the PV's actual used capacity.	
SOLUTION	Check PV/PVC binding: kubectl get pv,pvc -A. Pre-create PVs of exact sizes needed. Use StorageClass with volumeBindingMode: WaitForFirstConsumer to prevent premature binding. Implement capacity planning to match PV sizes to actual needs. For over-provisioned PVs: volume resize (shrinking is generally not supported - must recreate). Use dynamic provisioning to always get exact-size volumes.	

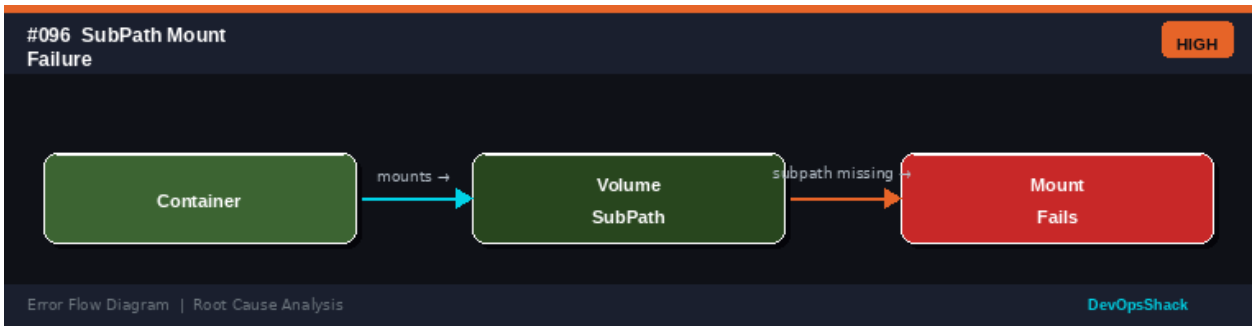
Error Flow Diagram



#096	SubPath Mount Failure *Storage*	HIGH
DESCRIPTION	A container fails to start because it mounts a specific sub-path within a volume that does not exist.	
ROOT CAUSE	Kubernetes allows mounting a subdirectory of a volume using subPath. If the subdirectory does not exist in the volume, the mount fails. This is common with: new volumes (empty), ConfigMap/Secret subPath mounts where the key doesn't exist, and race conditions where another pod creates the directory.	
SOLUTION	Ensure the subPath directory exists in the volume (create it during volume initialization with an init container). For ConfigMap/Secret subPaths, verify the key name matches exactly: the subPath for a ConfigMap key nginx.conf should	

	mount exactly that key. Use volumeMounts without subPath if the entire volume should be mounted. Check for case-sensitivity issues on the filesystem.

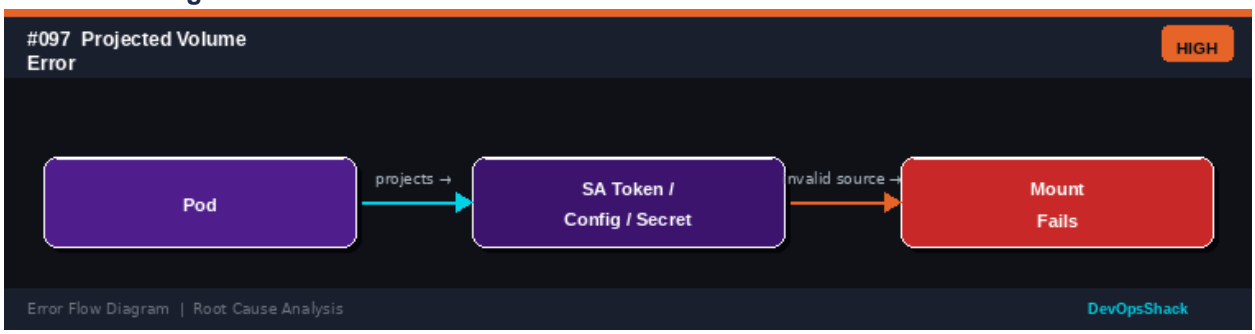
Error Flow Diagram



#097 Projected Volume Error *Storage* — HIGH

DESCRIPTION	A pod with a projected volume (combining ServiceAccount tokens, ConfigMaps, and Secrets into one volume) fails to mount correctly.
ROOT CAUSE	Projected volumes combine multiple volume sources into a single directory. Failures occur when: any referenced source (ConfigMap, Secret) does not exist, ServiceAccount token projection fails (expired OIDC config), the source key does not match, and version conflicts between projected volume feature support.
SOLUTION	Check each projected source independently: kubectl get cm,secret <names>. Verify ServiceAccount exists. Check if OIDC configuration is valid for token projection. Simplify by separating the projected volume into individual volume mounts to identify which source is failing. Check pod events: kubectl describe pod. Verify feature gates support projected volumes (GA since K8s 1.11).

Error Flow Diagram

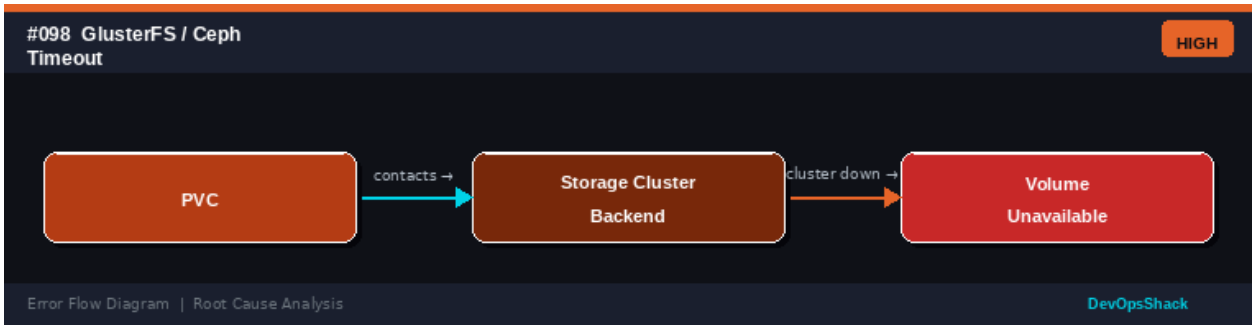


#098 GlusterFS/Ceph Volume Timeout *Storage* — HIGH

DESCRIPTION	Pods using GlusterFS or Ceph (Rook) volumes time out during mounting or I/O operations, causing pods to become unresponsive.
ROOT CAUSE	Distributed storage systems like GlusterFS and Ceph require the storage cluster to be healthy and reachable. Timeouts occur when: storage cluster nodes are down (loss of quorum), network congestion between compute and storage

	nodes, storage cluster is undersized for the I/O load, and rebalancing operations are causing performance degradation.
SOLUTION	Check storage cluster health: ceph status or gluster volume status. For Rook/Ceph: kubectl get cephcluster -n rook-ceph. Monitor storage cluster metrics. Add more OSDs for Ceph or bricks for GlusterFS to increase capacity. Set appropriate I/O timeout values in storage class parameters. Consider migrating to a cloud-native storage solution for better Kubernetes integration. Implement storage monitoring and alerting.

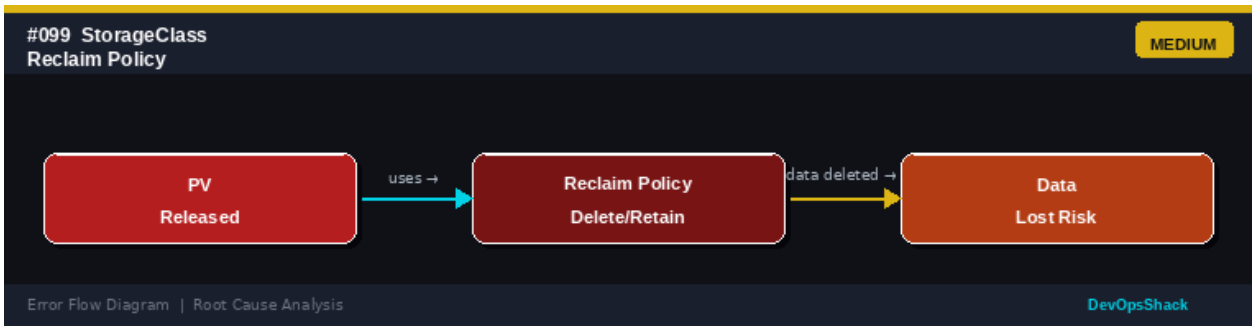
Error Flow Diagram



#099 StorageClass Reclaim Policy Risk *Storage* — MEDIUM

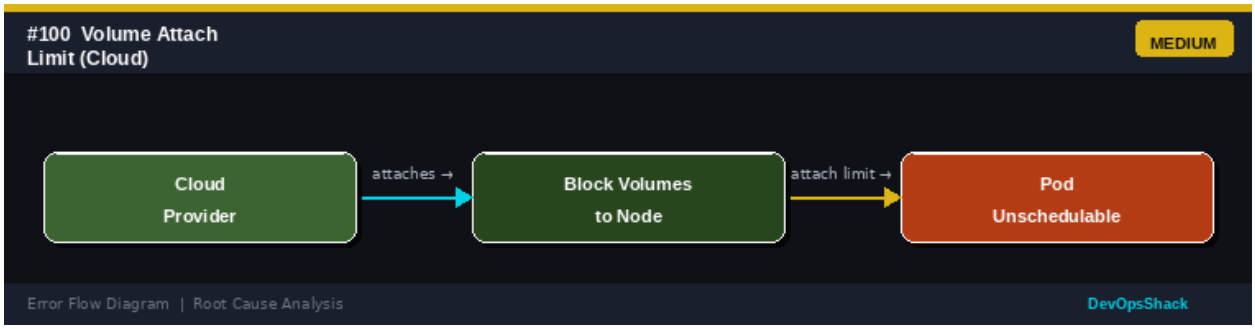
DESCRIPTION	When a PVC is deleted, the underlying PV and cloud disk are automatically deleted (Delete reclaim policy), causing permanent data loss.
ROOT CAUSE	StorageClass reclaim policy determines what happens to a PV when its bound PVC is deleted. Delete policy (default for dynamic provisioning) removes both the PV and the underlying storage resource. Retain policy keeps the storage. This becomes a risk when accidental PVC deletion occurs or when scaling down StatefulSets.
SOLUTION	Change the default StorageClass reclaim policy to Retain for production: kubectl patch storageclass <name> -p '{"reclaimPolicy":"Retain"}'. Note: this only affects newly created PVs. For existing PVs: kubectl patch pv <name> -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'. Implement backup solutions (Velero) independent of reclaim policy. Use RBAC to prevent unauthorized PVC deletion.

Error Flow Diagram



#100	Node Volume Attach Limit *Storage*	MEDIUM
DESCRIPTION	Pods with PVC requirements cannot be scheduled on nodes because those nodes have reached the cloud provider's maximum volume attachment limit.	
ROOT CAUSE	Cloud block storage volumes can only be attached to one VM at a time, and VMs have a maximum number of volumes they can attach. AWS: 25-39 EBS volumes depending on instance type. GCP: 128 persistent disks. Azure: varies by VM size. When the limit is reached, no more PVCs can be attached to pods on that node.	
SOLUTION	Check volume count per node: kubectl get pods -o wide and count PVCs. AWS: set --max-eben-volumes for the scheduler or set node limit via CSI driver. Use larger instance types with higher volume limits. Distribute pods with PVCs across more nodes using pod anti-affinity. Use shared filesystems (EFS, NFS, Ceph RWX) to reduce per-node volume count. Detach unused volumes from nodes.	

Error Flow Diagram



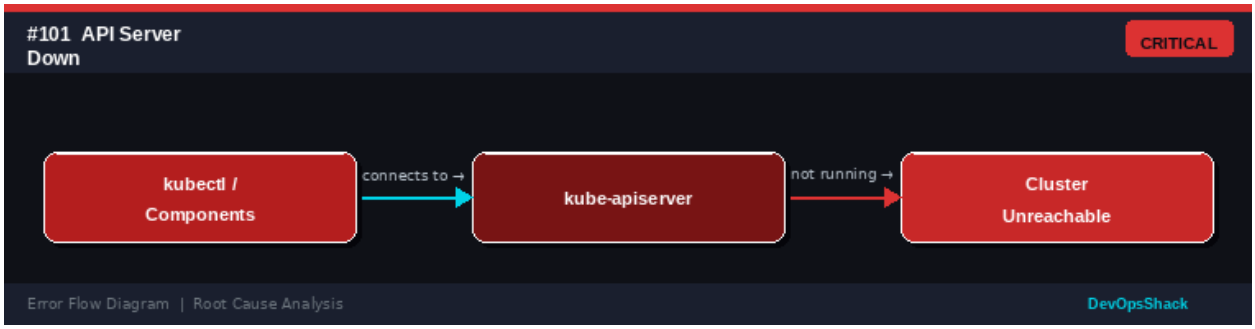
API & Control Plane Errors

Errors #101 – #120

Control plane errors affect the central management components of Kubernetes: the API server, etcd, controller manager, and scheduler. Issues here can have cluster-wide impact, making them the highest-priority errors to resolve.

#101	API Server Down *Control Plane*	CRITICAL
DESCRIPTION	The Kubernetes API server is not responding. kubectl commands fail with connection refused or timeout errors. The entire cluster management plane is unavailable.	
ROOT CAUSE	The kube-apiserver can go down due to: the process crashing (OOM, bug), the static pod manifest being invalid, etcd being unavailable, TLS certificate issues, and resource exhaustion (CPU/memory on control plane node). High-availability setups need all API servers down before total outage.	
SOLUTION	For single control plane: SSH to control plane node and check: systemctl status kube-apiserver or crictl ps | grep apiserver. Check static pod manifest: /etc/kubernetes/manifests/kube-apiserver.yaml. View logs: journalctl -u kubelet | grep apiserver. Check etcd health. For HA clusters, check load balancer health targeting API servers. Verify TLS certificates are valid. Check control plane node resources (disk, memory).	

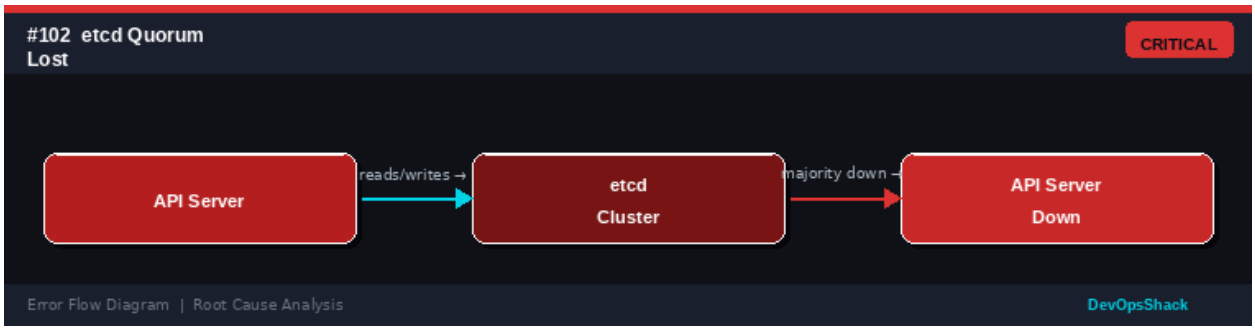
Error Flow Diagram



#102	etcd Quorum Lost *Control Plane*	CRITICAL
DESCRIPTION	The etcd cluster has lost quorum (majority of members are unavailable). The API server cannot read or write cluster state, making the entire cluster non-functional.	
ROOT CAUSE	etcd uses the Raft consensus protocol requiring a majority (quorum) of members to be available. For 3 nodes: 2 must be up. For 5 nodes: 3 must be up. Quorum is lost due to: multiple simultaneous node failures, network partition isolating majority of members, and all etcd members on a single host.	
SOLUTION	Restore from backup if quorum cannot be re-established. For recoverable scenarios: restart failed etcd members. Check etcd member list: etcdctl member list. For single-member recovery: start etcd with --force-new-cluster (dangerous -	

	only as last resort). Prevent by using 3 or 5 etcd members across availability zones. Set up regular etcd backups: etcdctl snapshot save backup.db.

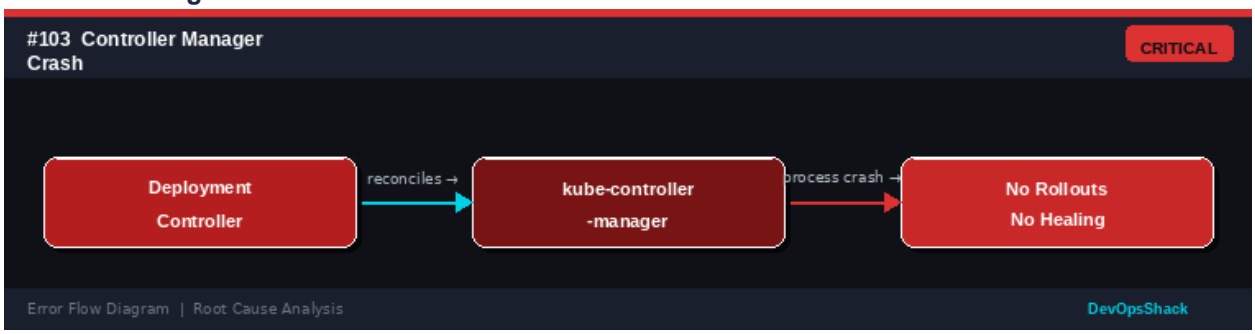
Error Flow Diagram



#103 Controller Manager Crash *Control Plane* — CRITICAL

DESCRIPTION	The kube-controller-manager has crashed or is not running. Deployments will not roll out new pods, failed pods will not be replaced, and resource management will stop working.
ROOT CAUSE	The controller manager runs many controllers in a single process: Deployment, ReplicaSet, Job, ServiceAccount, Node, etc. It crashes due to: OOM (running on a resource-constrained control plane), API server connectivity loss, certificate expiry, and bugs in admission webhooks that controllers trigger.
SOLUTION	Check controller manager status: crictl ps | grep kube-controller or kubectl get pods -n kube-system | grep kube-controller-manager. View logs: kubectl logs -n kube-system kube-controller-manager-<node>. For static pod deployments, check manifest: /etc/kubernetes/manifests/kube-controller-manager.yaml. Increase control plane node memory. Check for webhook timeouts that block controller operations.

Error Flow Diagram

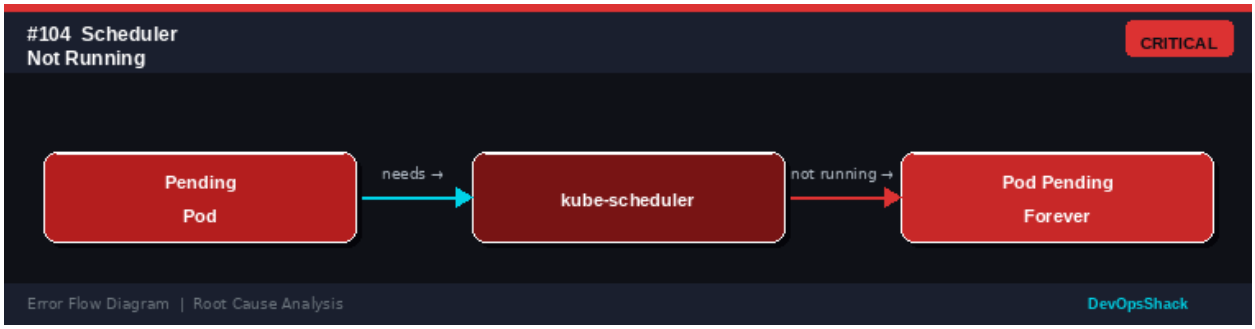


#104 Scheduler Not Running *Control Plane* — CRITICAL

DESCRIPTION	The kube-scheduler is not running. New pods remain in Pending state indefinitely since no component can assign them to nodes.
ROOT CAUSE	The scheduler watches for unschedulable pods and assigns them to appropriate nodes based on resource availability, affinity rules, and taints/tolerations. Without

	the scheduler: pods with nodeName set will still start (bypass scheduler), but all normally-created pods will stay Pending. Causes: scheduler pod crash, OOM, cert expiry, and leader election failure.
SOLUTION	Check scheduler: kubectl get pods -n kube-system | grep scheduler. View logs: kubectl logs -n kube-system kube-scheduler-<node>. For static pods: check /etc/kubernetes/manifests/kube-scheduler.yaml. Verify TLS certificates are valid. Check scheduler's leader election lease: kubectl get lease -n kube-system. Restart the scheduler pod or the kubelet on the control plane node if static pod is stuck.

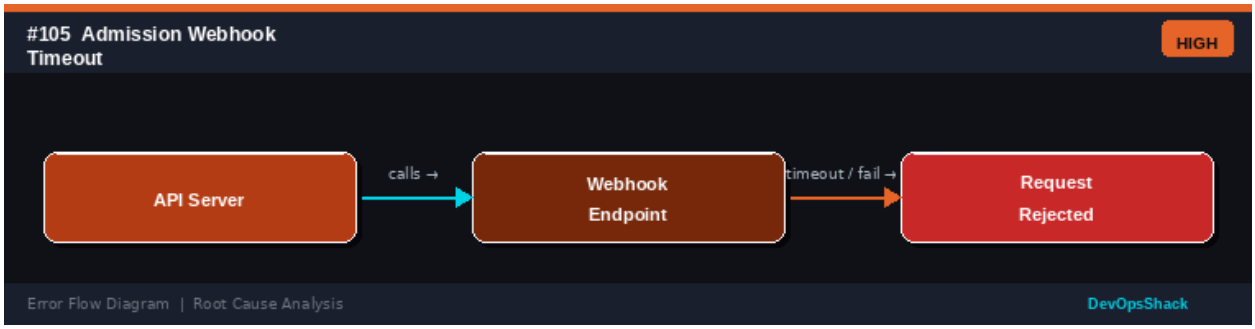
Error Flow Diagram



#105 Admission Webhook Timeout *Control Plane* — HIGH

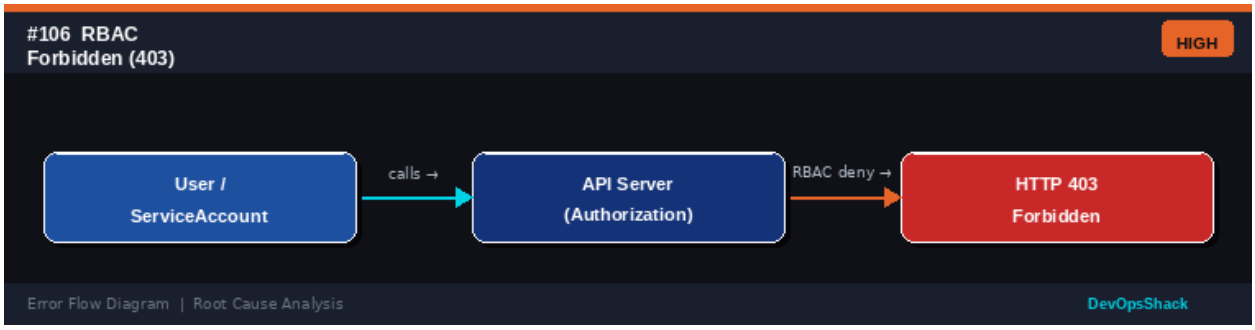
DESCRIPTION	API requests are being rejected with timeout errors because an admission webhook (validating or mutating) is not responding within the timeout period.
ROOT CAUSE	Admission webhooks intercept API requests for validation or mutation. If the webhook endpoint is slow or unavailable, API requests are rejected with a webhook call failure error. The default webhook timeout is 10 seconds. Causes: webhook pod is overloaded, network policy blocking API server to webhook traffic, and webhook service not having any ready endpoints.
SOLUTION	Check webhook pods are running: kubectl get pods in the webhook namespace. Check the webhook configuration: kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations. Set failurePolicy: Ignore on non-critical webhooks to prevent them from blocking cluster operations. Increase webhook timeout: timeoutSeconds: 30. Ensure the webhook service has endpoints: kubectl get endpoints <webhook-service>. Add the webhook pod to its own namespace's allow-list.

Error Flow Diagram



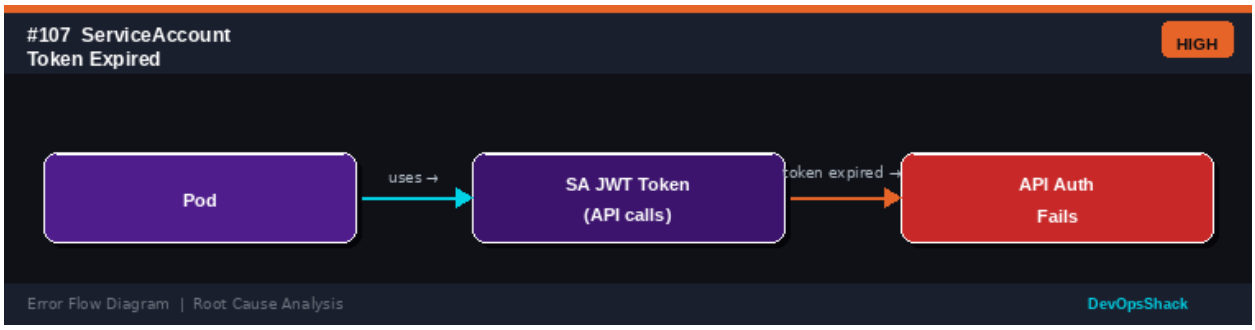
#106	RBAC Forbidden (403) *Control Plane*	HIGH
DESCRIPTION	A user or ServiceAccount receives HTTP 403 Forbidden when trying to perform an operation in the cluster. The API server rejects the request due to missing permissions.	
ROOT CAUSE	Kubernetes RBAC requires explicit permission grants. 403 errors occur when: a ClusterRole or Role binding is missing, the RoleBinding uses a wrong namespace, the ServiceAccount name is misspelled, and the Role does not include the required verb or resource. The default ServiceAccount has minimal permissions.	
SOLUTION	Check what permissions exist: kubectl auth can-i <verb> <resource> --as=<user>. List role bindings: kubectl get rolebindings,clusterrolebindings -A | grep <subject>. Create minimal RBAC: kubectl create role <name> --verb=get,list --resource=pods. Bind it: kubectl create rolebinding <name> --role=<role> --serviceaccount=<ns>:<sa>. Use kubectl auth reconcile to apply RBAC from files. Follow least-privilege principle.	

Error Flow Diagram



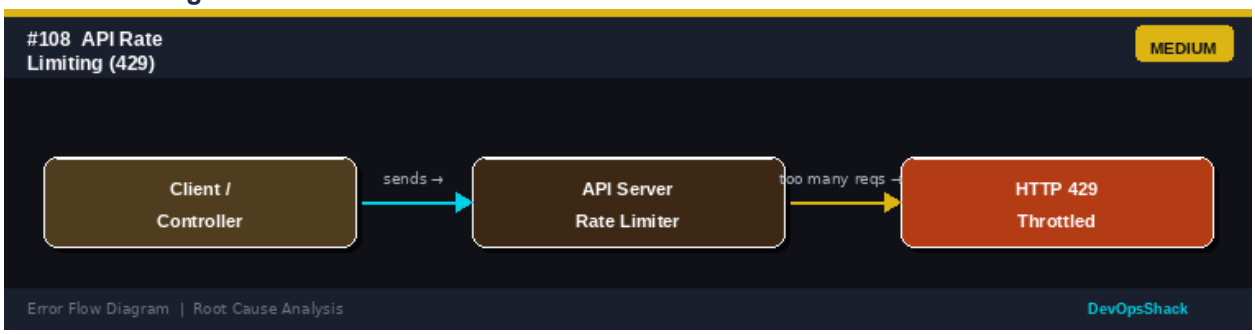
#107	ServiceAccount Token Expired *Control Plane*	HIGH
DESCRIPTION	A pod fails to authenticate with the Kubernetes API because its ServiceAccount token has expired. API calls from within the pod return 401 Unauthorized.	
ROOT CAUSE	Since Kubernetes 1.21, pods use time-limited projected ServiceAccount tokens (BoundServiceAccountToken feature). These tokens expire after 1 hour by default and are automatically rotated by the kubelet. Tokens in older pods (pre-1.21 format) or improperly projected volumes may not rotate. External use of tokens outside pods will expire.	
SOLUTION	For in-pod tokens: tokens are auto-rotated - restart the pod to refresh. Check if projected token is mounted correctly: kubectl exec -it <pod> -- cat /var/run/secrets/kubernetes.io/serviceaccount/token | jwt decode. Verify the kubelet is correctly rotating tokens. For external token use: generate new tokens with kubectl create token <serviceaccount> --duration=8760h. Avoid using long-lived static tokens.	

Error Flow Diagram



#108	API Rate Limiting (429) *Control Plane*	MEDIUM
DESCRIPTION	API server requests are being throttled with HTTP 429 Too Many Requests. Controllers, operators, or tools are receiving rate limit errors.	
ROOT CAUSE	The API server implements rate limiting (API Priority and Fairness or APF) to protect cluster stability. Too many requests per second from a single client or globally causes throttling. Common causes: controllers with tight retry loops, operators with excessive watches, kubectl commands in tight scripts, and CI/CD pipelines hammering the API.	
SOLUTION	Check API server audit logs for high-frequency clients. Implement exponential backoff in custom controllers. Enable API Priority and Fairness (APF) for granular rate control: configure FlowSchemas and PriorityLevelConfigurations. Reduce watch frequency in operators. Use informers with ListWatch instead of polling. Distribute API calls across time. Increase --max-requests-inflight and --max-mutating-requests-inflight API server flags.	

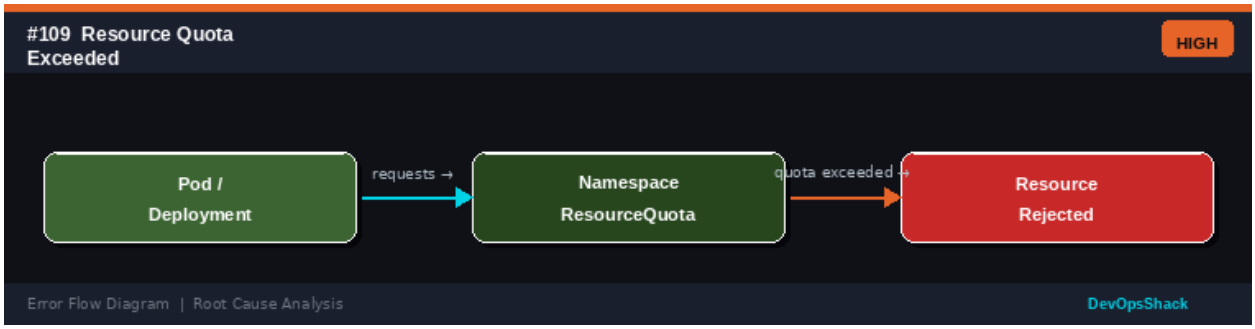
Error Flow Diagram



#109	Resource Quota Exceeded *Control Plane*	HIGH
DESCRIPTION	Creating a new pod, deployment, or other resource fails with exceeded quota error because the namespace's ResourceQuota has been reached.	
ROOT CAUSE	ResourceQuotas limit the total resources that can be consumed in a namespace. Quotas cover: CPU/memory requests and limits, number of pods, Services, Secrets, ConfigMaps, PVCs, and storage. Quotas are checked at admission time and prevent new resources from being created when limits are reached.	
SOLUTION	Check quota usage: kubectl describe resourcequota -n <namespace>. Identify what is consuming quota: kubectl get pods,pvc,svc -n <namespace>. Delete	

	unused resources. Request a quota increase from the cluster admin. Ensure completed Jobs have TTL set to clean up automatically. Review quota values for appropriateness of workload. Use limit ranges to prevent single pods from consuming all quota.

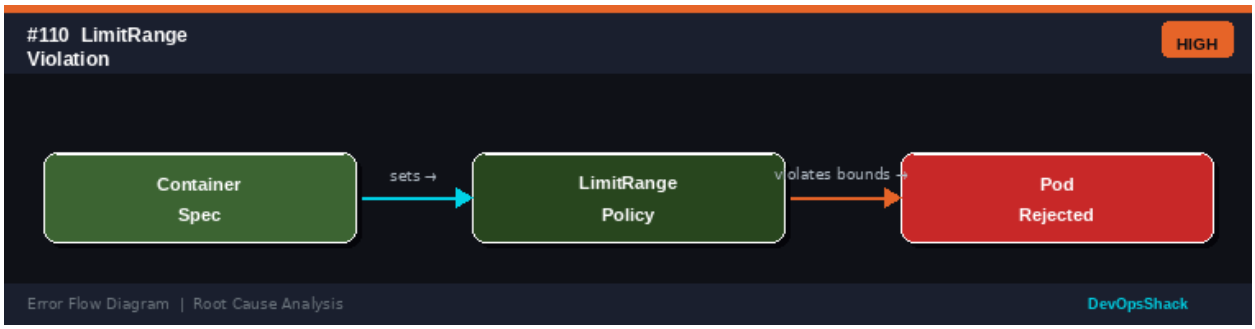
Error Flow Diagram



#110 LimitRange Violation *Control Plane* — HIGH

DESCRIPTION	A pod is rejected because its resource requests or limits violate the LimitRange constraints defined for the namespace.
ROOT CAUSE	LimitRanges set minimum and maximum resource requirements for containers and pods in a namespace. They can also set defaults when no requests/limits are specified. Violations occur when: container requests less than the LimitRange minimum (too low), container requests more than the maximum (too high), and limit-to-request ratio exceeds the maxLimitRequestRatio.
SOLUTION	Check LimitRange: kubectl describe limitrange -n <namespace>. View the specific violation in the pod rejection message. Adjust container resource requests and limits to fall within the LimitRange bounds. For default injection: LimitRange will inject defaults for containers with no resources set. Update LimitRange if it is too restrictive: kubectl edit limitrange <name>. Balance workload needs with namespace policy.

Error Flow Diagram

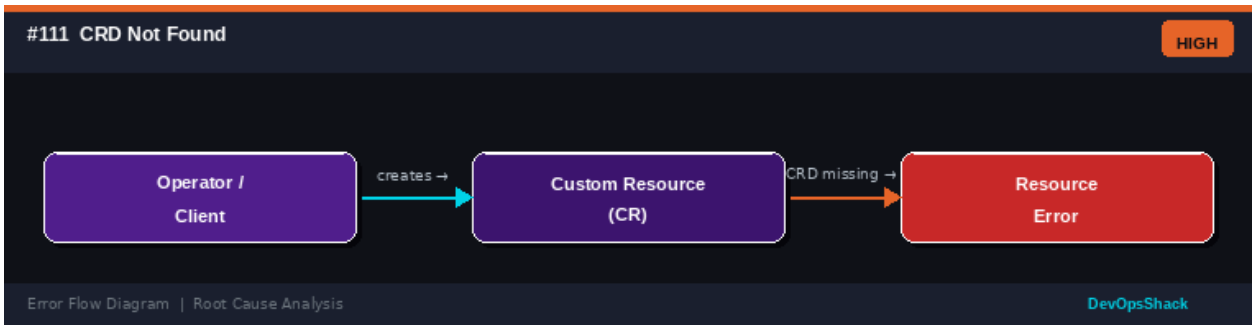


#111 CRD Not Found *Control Plane* — HIGH

DESCRIPTION	An operator or user tries to create a Custom Resource (CR) but gets a 404 error because the Custom Resource Definition (CRD) is not installed in the cluster.

ROOT CAUSE	Custom Resources extend Kubernetes and require their CRD to be installed first. CRDs define the schema, versions, and scope of custom resources. Missing CRDs occur when: an operator is uninstalled but CRs remain, CRD installation step was skipped, CRD was accidentally deleted, and upgrading an operator without upgrading CRDs first.
SOLUTION	Check CRDs: kubectl get crds | grep <name>. Install missing CRD from operator manifests: kubectl apply -f crds/. For Helm: helm install should include CRDs. If CRD was deleted with live CRs, reinstall CRD and CRs may reappear. Check Helm chart's crds/ directory. For Operator Lifecycle Manager: check CRD is included in the operator's CSV. Always install CRDs before operator pods.

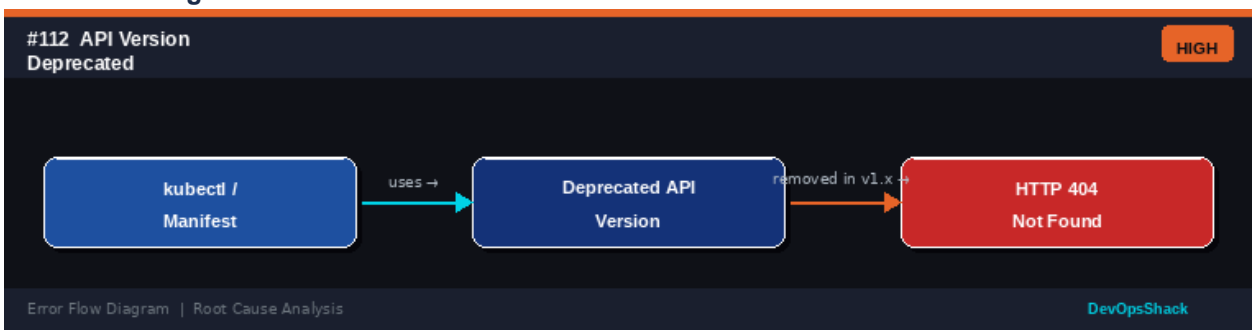
Error Flow Diagram



#112 API Version Deprecated/Removed *Control Plane* HIGH

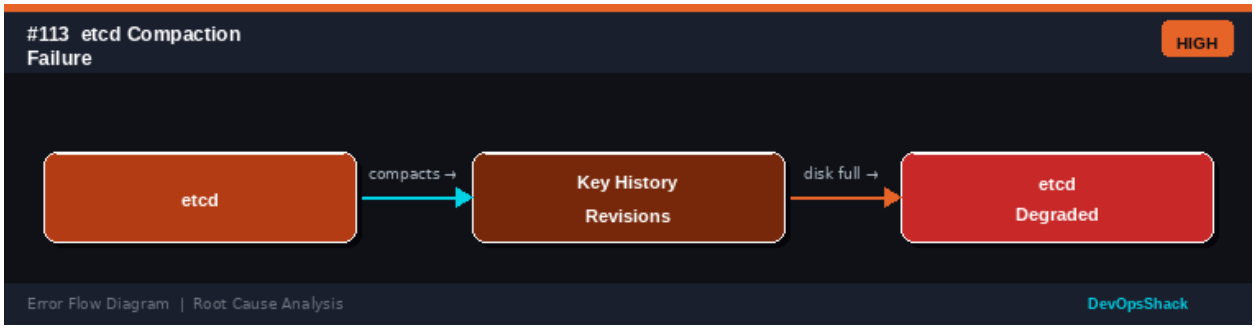
DESCRIPTION	An API resource fails with 404 Not Found or 400 Bad Request because the apiVersion specified in the manifest has been removed from the Kubernetes version being used.
ROOT CAUSE	Kubernetes periodically deprecates and removes API versions after a deprecation period. Examples: extensions/v1beta1 (removed in 1.16), networking.k8s.io/v1beta1 (removed in 1.22), policy/v1beta1 PodSecurityPolicy (removed in 1.25). Manifests using removed API versions fail immediately.
SOLUTION	Check which API versions are available: kubectl api-versions. Use kubectl convert to migrate manifests: kubectl convert -f old-manifest.yaml --output-version apps/v1. Run Pluto or kubent to detect deprecated API usage before upgrades: pluto detect-files -d <dir>. Update all manifests to use the current stable API version. Review Kubernetes deprecation guide before each upgrade.

Error Flow Diagram



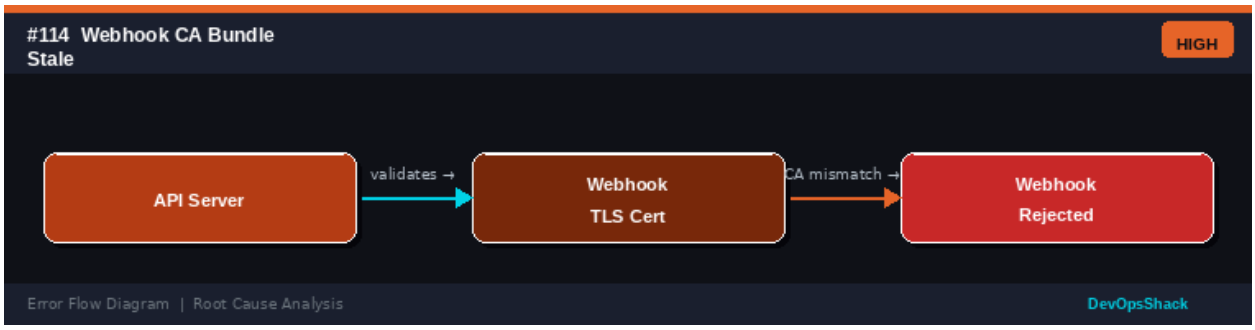
#113	etcd Compaction Failure *Control Plane*	HIGH
DESCRIPTION	etcd is becoming slow or unresponsive because its database is growing and revisions are not being compacted or defragmented.	
ROOT CAUSE	etcd keeps a history of all key revisions for MVCC (Multi-Version Concurrency Control). Without regular compaction, the database grows unboundedly. Without defragmentation, free space from deleted keys is not reclaimed. Both cause: slow etcd operations, increased memory usage, and eventual disk exhaustion.	
SOLUTION	Enable automatic compaction: --auto-compaction-retention=8 (hours) in etcd config. Perform manual compaction: etcdctl compact $(etcdctl endpoint status --write-out='json' | jq -r '.[].Status.header.revision'). Defragment: ETCDCTL_API=3 etcdctl defrag --endpoints=<all-endpoints>. Monitor etcd database size. Schedule regular compaction+defrag in a cron job. Set --quota-backend-bytes appropriately.	

Error Flow Diagram



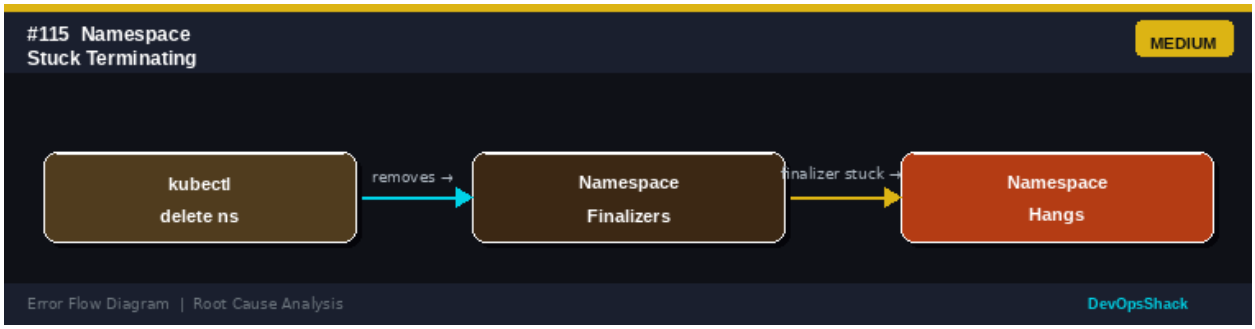
#114	Webhook CA Bundle Stale *Control Plane*	HIGH
DESCRIPTION	Admission webhooks are being rejected because the caBundle in the webhook configuration does not match the current TLS certificate of the webhook service.	
ROOT CAUSE	Admission webhooks use TLS and the webhook configuration includes a caBundle for certificate verification. When cert-manager rotates certificates, the caBundle in the WebhookConfiguration must also be updated. Stale CAs cause TLS verification failure and all webhook calls are rejected.	
SOLUTION	Check webhook configuration: kubectl get validatingwebhookconfigurations <name> -o yaml. If using cert-manager, annotate the webhook to auto-inject CA: cert-manager.io/inject-ca-from: <ns>/<cert-name>. Manually update caBundle with the current CA. Use the --auto-inject-ca flag in relevant operators. Check if cert-manager cainjector is running: kubectl get pods -n cert-manager. Ensure cert-manager has RBAC to update webhook configs.	

Error Flow Diagram



#115	Namespace Stuck Terminating *Control Plane*	MEDIUM
DESCRIPTION	A namespace shows Terminating status indefinitely and cannot be deleted. Resources within the namespace cannot be deleted either.	
ROOT CAUSE	Namespace deletion requires all resources within to be deleted first. It gets stuck when: custom resource finalizers are not cleaned up (the owning operator is gone), a custom controller is not running to process finalizers, and the namespace itself has a finalizer that nothing is processing.	
SOLUTION	Force-remove namespace finalizers: kubectl get namespace <name> -o json | python3 -c "import json,sys; ns=json.load(sys.stdin); ns['spec']['finalizers']=[]; print(json.dumps(ns))" | kubectl replace --raw /api/v1/namespaces/<name>/finalize -f -. Check for CRDs with resources in the namespace: kubectl api-resources --verbs=list --namespaced -o name | xargs -I{} kubectl get {} -n <ns>. Remove or patch finalizers on individual resources.	

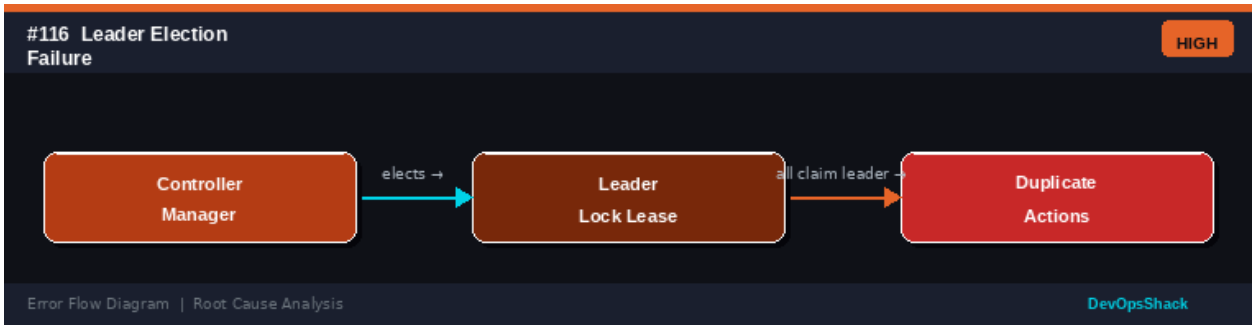
Error Flow Diagram



#116	Leader Election Failure *Control Plane*	HIGH
DESCRIPTION	Multiple instances of a controller (kube-controller-manager, kube-scheduler) are simultaneously acting as leader, causing duplicate operations or conflicts.	
ROOT CAUSE	Kubernetes controllers use lease-based leader election to ensure only one instance is active at a time. Failures occur when: the lease lock cannot be written to etcd (etcd issues), network partition causes two instances to think they are leaders, and the lease renewal fails silently, allowing another instance to take over while the first continues.	
SOLUTION	Check leader leases: kubectl get lease -n kube-system. Identify which instance holds the lease. Ensure etcd is healthy for reliable leader election. If two leaders	

	exist simultaneously, identify the root cause (network partition, etcd issue) and resolve it. For custom controllers, use the controller-runtime leader election library. Ensure lease duration, renew deadline, and retry period are properly tuned.

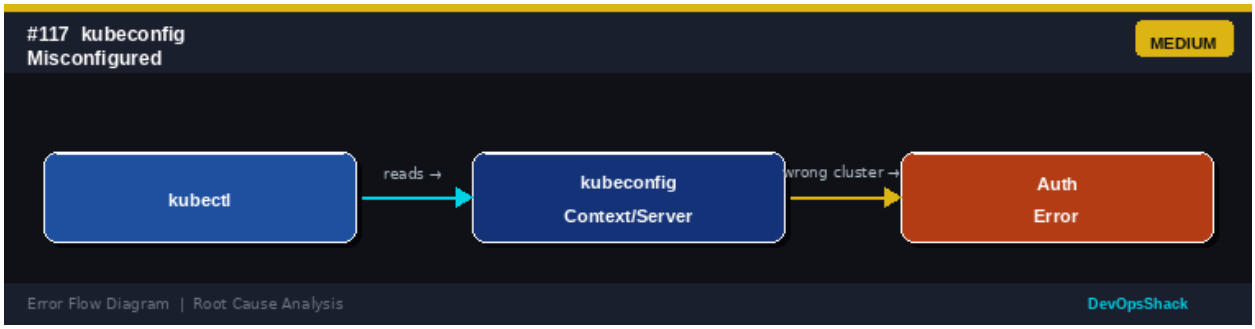
Error Flow Diagram



#117 kubeconfig Misconfigured *Control Plane* — MEDIUM

DESCRIPTION	kubectl commands fail with authentication or connection errors because the kubeconfig file points to the wrong cluster, uses invalid credentials, or has a malformed context.
ROOT CAUSE	kubeconfig contains cluster connection details, credentials, and contexts. Common issues: wrong server URL, expired certificate data, missing or incorrect token, context pointing to a deleted cluster, and namespace override in context masking actual namespace.
SOLUTION	Check kubeconfig: kubectl config view. Verify current context: kubectl config current-context. Switch context: kubectl config use-context <context-name>. Check connection: kubectl cluster-info. Regenerate kubeconfig with kubeadm: kubeadm kubeconfig user --client-name=<name>. For EKS: aws eks update-kubeconfig --name <cluster>. For GKE: gcloud container clusters get-credentials <cluster>. Merge multiple kubeconfigs: KUBECONFIG=file1:file2 kubectl config view --flatten.

Error Flow Diagram

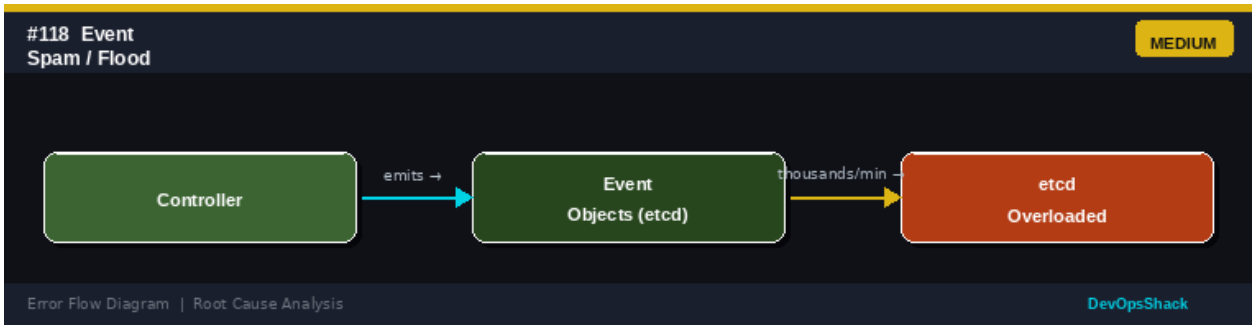


#118 Event Spam / Flood *Control Plane* — MEDIUM

DESCRIPTION	The cluster is generating thousands of events per minute, overloading etcd and making kubectl get events output unusable.

ROOT CAUSE	Event flooding occurs when controllers emit events in tight loops, such as: a misconfigured resource that keeps failing and retrying, a liveness probe failing repeatedly, a custom controller emitting events too aggressively, and resource conflicts causing constant update loops.
SOLUTION	Identify high-volume event sources: kubectl get events -A --sort-by='.metadata.creationTimestamp' | tail -50. Fix the underlying loop causing repeated events. Configure event compression and throttling in kubelet: --event-burst, --event-qps. Implement event deduplication in custom controllers (use recorder.Eventf with appropriate reason and message deduplication). Reduce event TTL in API server. Consider using structured logging instead of events for high-frequency status updates.

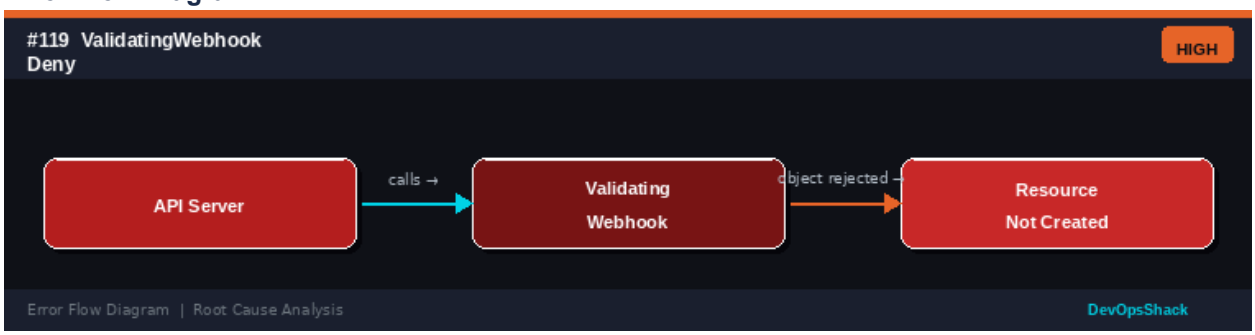
Error Flow Diagram



#119 ValidatingWebhook Denial *Control Plane* — HIGH

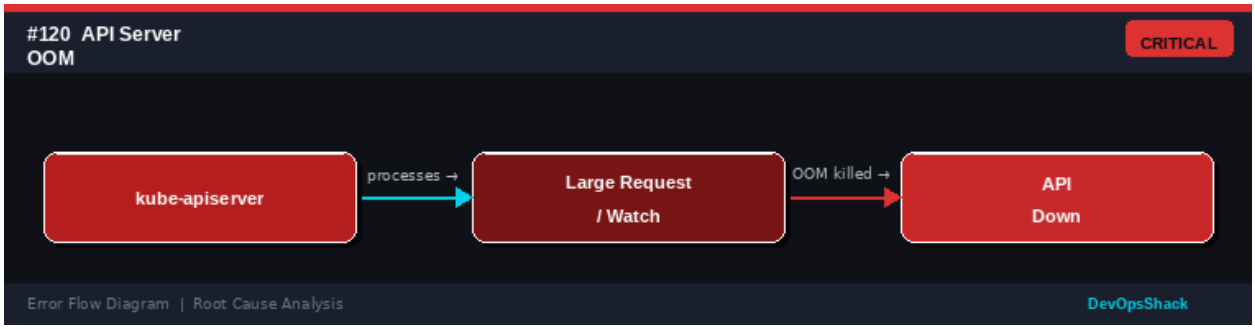
DESCRIPTION	A resource creation or update is blocked by a ValidatingAdmissionWebhook with a message explaining which validation rule was violated.
ROOT CAUSE	ValidatingAdmissionWebhooks implement custom validation logic. Common blockers include: OPA/Gatekeeper policies (e.g., required labels, image registry restrictions), security admission controllers, cost controls blocking large resource requests, and custom validation rules from operators.
SOLUTION	Read the rejection message carefully - it contains the policy and reason. Check the validating webhook: kubectl get validatingwebhookconfigurations. Review the policy that is blocking (OPA constraint, Gatekeeper policy). Update your manifest to comply with the policy. For urgent bypasses, get cluster admin to temporarily disable the webhook or add an exception. Fix policies if they are overly restrictive.

Error Flow Diagram



#120	API Server OOM Killed *Control Plane*	CRITICAL
DESCRIPTION	The kube-apiserver process is killed by the OOM killer due to excessive memory usage. This causes cluster-wide outages until the API server restarts.	
ROOT CAUSE	The API server handles all client requests, stores large Watch result sets in memory, and caches frequently-accessed objects. Memory spikes occur with: large number of watch clients, high object count (many pods/secrets/configmaps), large requests fetching all resources at once, and memory leaks in older versions.	
SOLUTION	Increase API server memory limits on control plane nodes (add RAM). Set --watch-cache-sizes to limit per-resource watch cache. Review what is making large list requests: API server audit logs. Set resource version watch bookmarks to reduce memory. Upgrade to newer Kubernetes with improved memory efficiency. Enable API server request metrics monitoring. Implement pagination in controllers using limit and continue.	

Error Flow Diagram



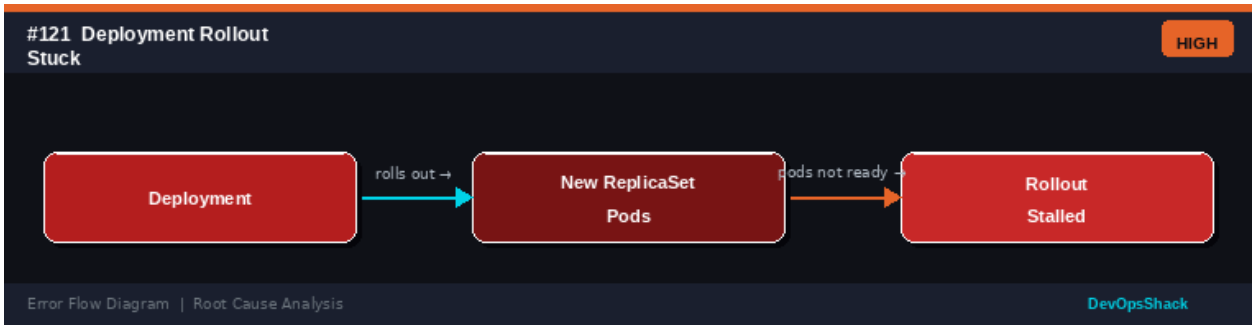
Deployment & Workload Errors

Errors #121 – #140

Workload errors affect higher-level abstractions: Deployments, StatefulSets, DaemonSets, CronJobs, and GitOps tools like Helm, ArgoCD, and Flux. These are the most common day-to-day operational errors.

#121	Deployment Rollout Stuck *Workloads*	HIGH
DESCRIPTION	A Deployment is stuck in a rolling update and does not complete. Some pods are on the new version and some remain on the old version indefinitely.	
ROOT CAUSE	Rolling updates progress by bringing up new pods while terminating old ones. A rollout stalls when: new pods fail readiness probes, there are not enough resources for new pods to start, a PodDisruptionBudget prevents terminating old pods, and a dependent ConfigMap or Secret is missing for the new version.	
SOLUTION	Check rollout status: kubectl rollout status deployment/<name>. View events: kubectl describe deployment <name>. Check pod status for new ReplicaSet: kubectl get pods -l <new-rs-selector>. Check pod logs: kubectl logs <new-pod>. If the rollout is unrecoverable, roll back: kubectl rollout undo deployment/<name>. Fix the underlying issue (probe, config, resources) before re-rolling.	

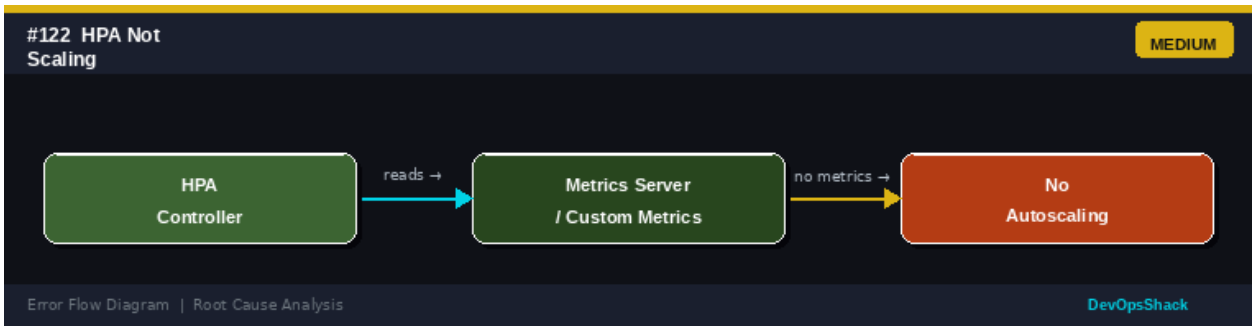
Error Flow Diagram



#122	HPA Not Scaling *Workloads*	MEDIUM
DESCRIPTION	The HorizontalPodAutoscaler is not increasing pod count when load increases, or is not decreasing during low load.	
ROOT CAUSE	HPA reads metrics from the Metrics Server (for CPU/memory) or a custom metrics adapter. Scaling fails when: Metrics Server is not installed or not working, metrics are not available for the target pods, the HPA spec targets non-existent metrics, and scaling is in cooldown period (default 5 minutes scale-down).	
SOLUTION	Check HPA status: kubectl describe hpa <name>. Verify metrics server: kubectl top pods. If no metrics: install or fix Metrics Server. For custom metrics, check the metrics adapter. Verify target metric names match exactly. Check HPA events for messages. Set appropriate minReplicas and maxReplicas. Ensure pods have	

resource requests set (required for CPU-based HPA). Check for RBAC issues: HPA needs permission to get/update scale subresource.

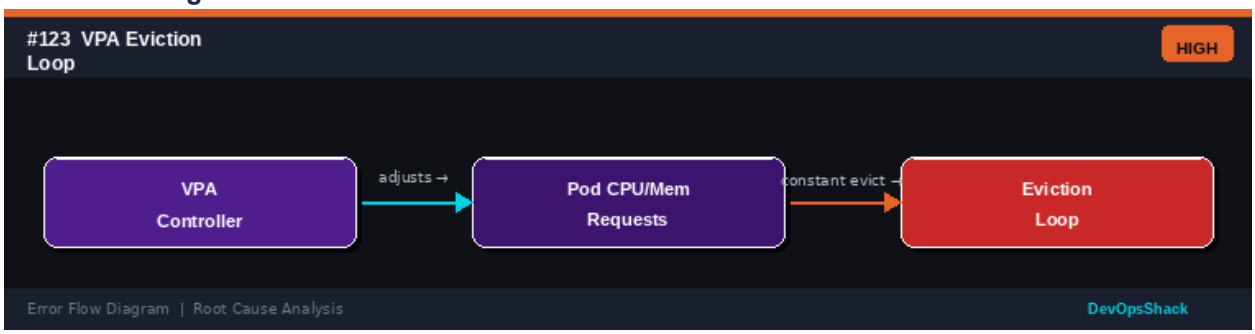
Error Flow Diagram



#123 VPA Eviction Loop *Workloads* — HIGH

DESCRIPTION	The Vertical Pod Autoscaler is continuously evicting and restarting pods to apply new resource recommendations, causing excessive disruption.
ROOT CAUSE	VPA adjusts CPU/memory requests by evicting and recreating pods. An eviction loop occurs when: VPA recommendations keep changing (oscillating metrics), VPA mode is set to Auto with too-frequent updates, the evicted pod immediately triggers another VPA update, and VPA is used alongside HPA on the same resource type (conflict).
SOLUTION	Check VPA recommendations: kubectl describe vpa <name>. Set VPA mode to Off or Initial to get recommendations without automatic eviction. Do not use VPA and HPA on the same resource metric (e.g., CPU). Use VPA only on memory and HPA only on CPU. Set minAllowed and maxAllowed boundaries to prevent extreme recommendations. Consider using KEDA instead for event-driven scaling.

Error Flow Diagram

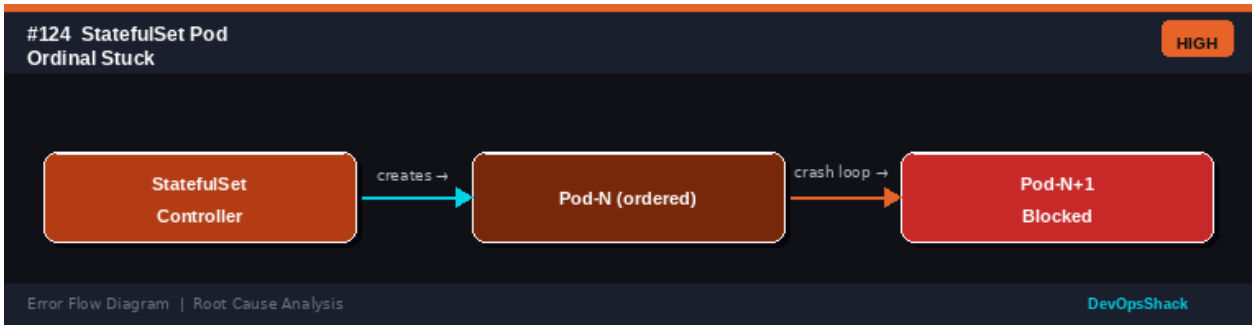


#124 StatefulSet Pod Ordinal Stuck *Workloads* — HIGH

DESCRIPTION	A StatefulSet is stuck and cannot progress because a specific pod (e.g., pod-2) is not becoming ready, blocking the creation of subsequent pods (pod-3, pod-4).
ROOT CAUSE	StatefulSets create pods in order (0, 1, 2...) and each pod must be Running and Ready before the next is created. If any pod in the sequence fails, the StatefulSet

	halts. This is by design to ensure ordered startup (important for leader election, cluster formation). Causes: same as pod CrashLoop but specific to one ordinal.
SOLUTION	Check the stuck pod: kubectl describe pod <statefulset>-<ordinal>. Fix the underlying issue (app config, volume, dependencies). For ordered services requiring all peers first (e.g., Zookeeper), the last peer cannot join until all previous peers are up. Use podManagementPolicy: Parallel for workloads that don't need ordered startup. Roll back the StatefulSet if a bad image was deployed: kubectl rollout undo statefulset/<name>.

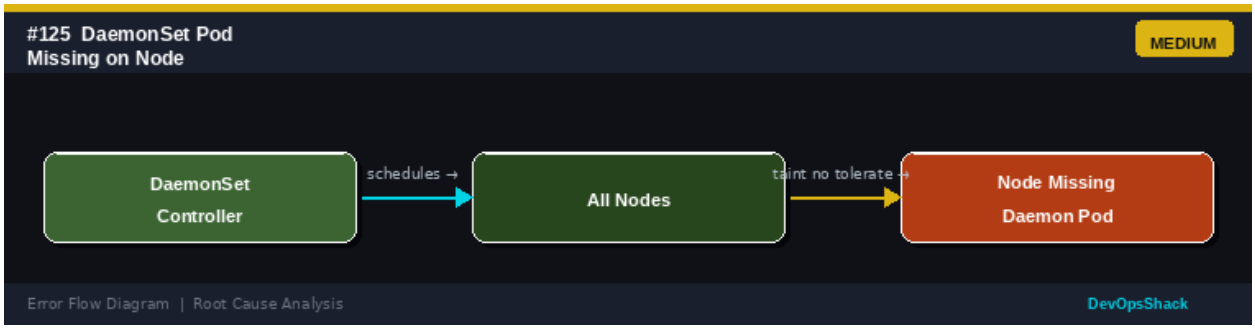
Error Flow Diagram



#125 DaemonSet Pod Missing on Node *Workloads* MEDIUM

DESCRIPTION	A DaemonSet is not running a pod on one or more nodes that should have it. The DaemonSet appears partially deployed.
ROOT CAUSE	DaemonSets run one pod per node but respect scheduling constraints. Pods are missing when: the node has a taint that the DaemonSet does not tolerate, the DaemonSet has a nodeSelector that excludes the node, the node is in a cordoned state, and the DaemonSet update strategy is paused.
SOLUTION	Check DaemonSet desired vs actual: kubectl get daemonset -A. Identify which nodes are missing the pod: kubectl get pods -o wide and compare to node list. Check node taints: kubectl describe node <name> | grep Taint. Add tolerations to the DaemonSet for existing taints. Verify nodeSelector and affinity rules include all intended nodes. For system DaemonSets, add tolerations for all taints including NoSchedule and NoExecute.

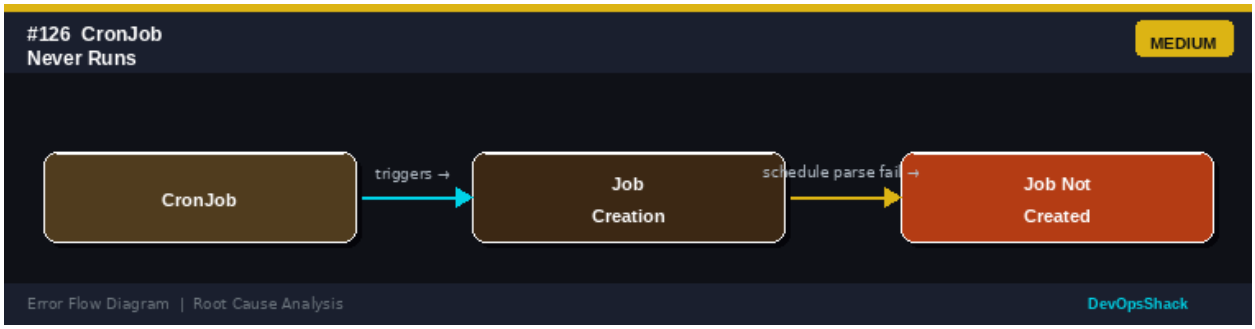
Error Flow Diagram



#126 CronJob Never Runs *Workloads* MEDIUM

DESCRIPTION	A CronJob is defined but jobs are never created. The schedule seems correct but no pods are ever started.
ROOT CAUSE	CronJob scheduling uses UTC by default. Issues: incorrect cron expression syntax (Kubernetes uses standard cron: minute hour day-of-month month day-of-week), timezone mismatch (use spec.timeZone for non-UTC), startingDeadlineSeconds missed, cluster time skew, and successfulJobsHistoryLimit/failedJobsHistoryLimit set to 0 hiding history.
SOLUTION	Verify cron syntax at crontab.guru. Check CronJob status: kubectl describe cronjob <name>. Check if jobs are created: kubectl get jobs -l <cronjob-label>. Use spec.timeZone: 'America/New_York' for explicit timezone (Kubernetes 1.25+). Set startingDeadlineSeconds: 300 to allow late starts. Check cluster time is correct. Set schedule: '*/1 * * * *' to test every minute.

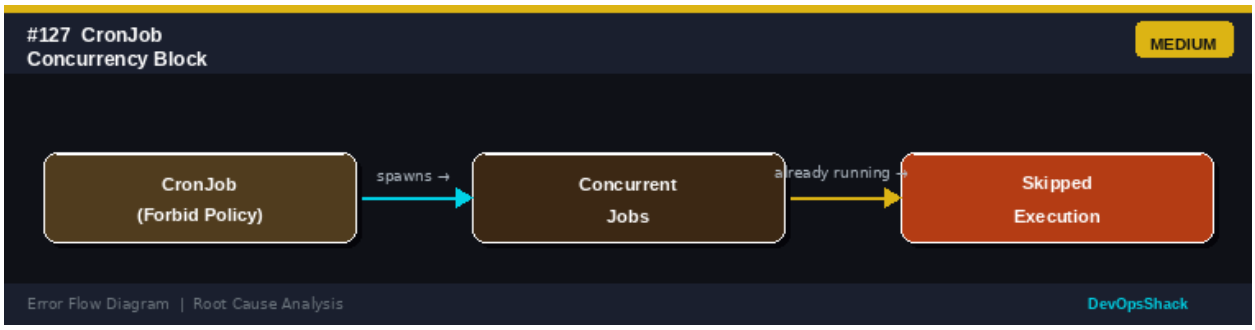
Error Flow Diagram



#127 CronJob Concurrency Block *Workloads* — MEDIUM

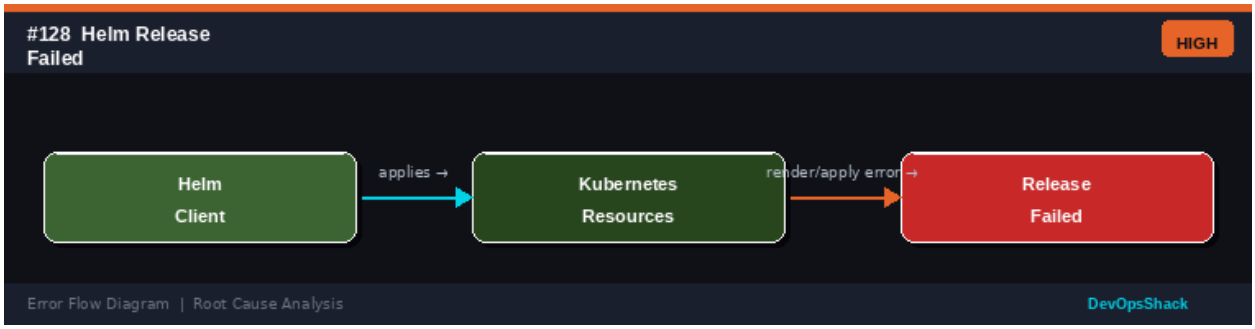
DESCRIPTION	A CronJob is configured with concurrencyPolicy: Forbid or Replace and jobs are being skipped or replaced when a previous execution is still running.
ROOT CAUSE	CronJob concurrency policy controls what happens if a previous Job is still running when the next schedule fires: Allow (multiple simultaneous jobs), Forbid (skip new job if previous is running), Replace (delete running job and start new one). Forbid will skip jobs during slow runs. Replace causes unexpected job termination.
SOLUTION	Check concurrency policy: kubectl get cronjob <name> -o jsonpath='{.spec.concurrencyPolicy}'. If jobs are being skipped (Forbid), optimize the job to complete faster or increase its frequency. If jobs are being replaced (Replace), consider whether partial execution is safe. Change policy to Allow if parallel execution is safe. Monitor job completion times and set appropriate resource limits.

Error Flow Diagram



#128	Helm Release Failed *Workloads*	HIGH
DESCRIPTION	A Helm install or upgrade operation failed, leaving the release in a failed state. Future Helm operations may be blocked by the failed release.	
ROOT CAUSE	Helm releases fail when: Kubernetes resources fail to apply (invalid YAML, schema violation, RBAC), pre-install or post-install hooks fail, timeout waiting for deployments to become ready, and concurrent Helm operations conflict on the same release.	
SOLUTION	Check release status: helm status <release-name> -n <namespace>. View Kubernetes events: kubectl get events -n <namespace> --sort-by='.metadata.creationTimestamp'. Check failed resource: kubectl describe <resource> <name>. If the release is stuck in failed state: helm rollback <release> <revision> or helm uninstall <release>. Dry-run before applying: helm upgrade --dry-run. Fix the underlying Kubernetes resource issue.	

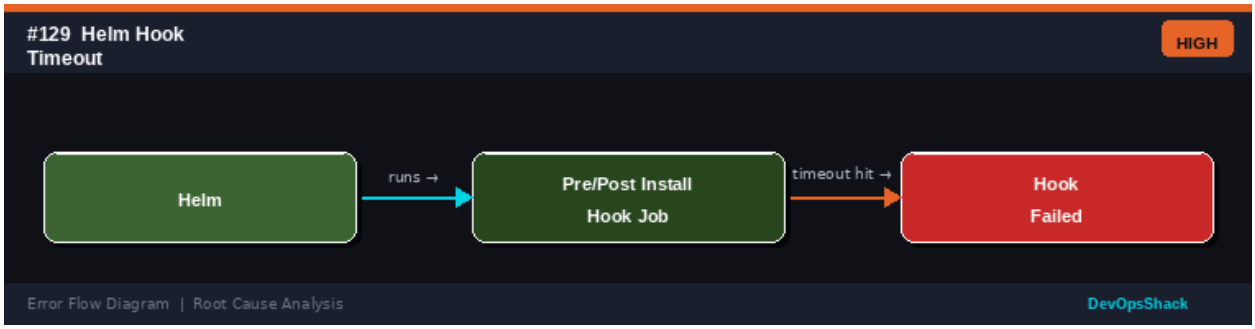
Error Flow Diagram



#129	Helm Hook Timeout *Workloads*	HIGH
DESCRIPTION	A Helm pre-install or post-install hook job did not complete within the timeout period. The Helm release is marked as failed.	
ROOT CAUSE	Helm hooks are Jobs that run at specific points in the release lifecycle. They time out when: the hook job's pod fails to start (image pull, scheduling), the hook takes longer than hook-delete-policy allows, the hook job restarts excessively, and the Helm default timeout (300 seconds) is insufficient.	
SOLUTION	Check hook job: kubectl get jobs -n <namespace>. Check hook pod: kubectl logs <hook-pod>. Increase Helm timeout: helm upgrade <release> <chart> --timeout=600s. Set appropriate resource requests so hook pods can be	

	scheduled. Fix application issues in the hook script. Add --atomic flag to automatically rollback on failure. Set hook-delete-policy: before-hook-creation to clean up old hook pods.

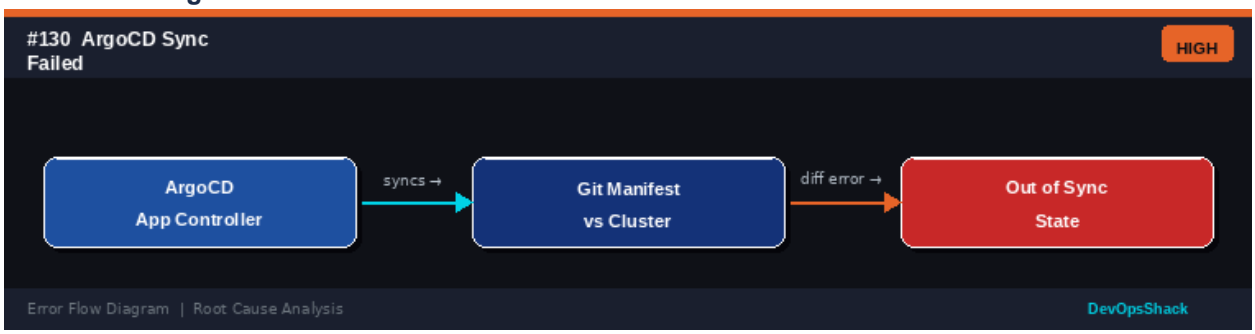
Error Flow Diagram



#130 ArgoCD Sync Failed *Workloads* — HIGH

DESCRIPTION	ArgoCD cannot synchronize an Application with the Git repository. The application shows OutOfSync or Degraded status.
ROOT CAUSE	ArgoCD sync failures occur when: Kubernetes rejects the manifest (validation error, RBAC, webhook), the Git repository is inaccessible, there is a network issue from ArgoCD to the cluster, resource hooks fail, and sync waves have ordering issues.
SOLUTION	Check ArgoCD Application status: kubectl get application <name> -n argocd. View sync details: argocd app sync-status <name>. Check application events in ArgoCD UI. View ArgoCD application controller logs: kubectl logs -n argocd <argocd-application-controller>. Validate manifests locally: kubectl apply --dry-run=server. Check RBAC for ArgoCD service account. Fix Git repository connectivity. Run argocd app sync --force for manual sync.

Error Flow Diagram

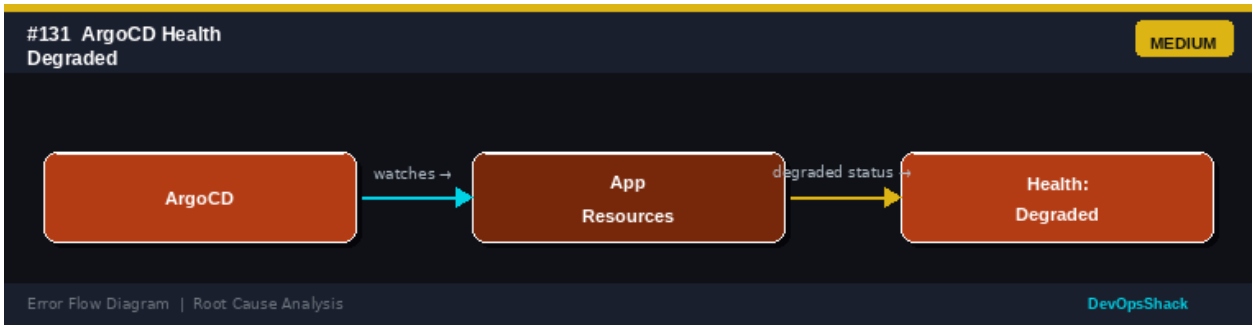


#131 ArgoCD Health Degraded *Workloads* — MEDIUM

DESCRIPTION	An ArgoCD Application shows Health: Degraded, indicating that deployed resources are not in their expected healthy state.
ROOT CAUSE	ArgoCD evaluates resource health using built-in and custom health checks. Degraded health is reported when: Deployments have unavailable replicas,

	StatefulSets pods are not ready, Jobs have failed, custom health check lua scripts determine a resource is unhealthy, and PVCs are stuck pending.
SOLUTION	Check which resources are degraded in the ArgoCD UI or CLI: argocd app get <name>. Fix the underlying resource issues (pod failures, probe failures, scheduling issues). For custom resources, add custom health check scripts in ArgoCD ConfigMap. Set resource.customHealthCheck for unsupported CRDs. Sync the application after fixing: argocd app sync <name>. Check if ignored differences are causing false degraded status.

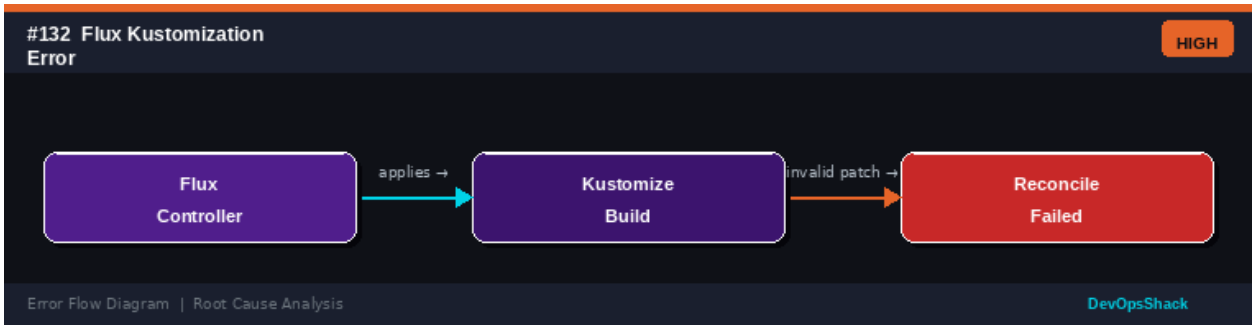
Error Flow Diagram



#132 Flux Kustomization Error *Workloads* HIGH

DESCRIPTION	Flux's Kustomization controller fails to reconcile because the Kustomize build or Kubernetes apply fails.
ROOT CAUSE	Flux uses kustomize to build manifests from Git and applies them. Failures occur when: kustomize patches reference non-existent base resources, invalid patch syntax, Kubernetes validation rejects the output, and resource dependencies are not ready when the kustomization runs.
SOLUTION	Check Kustomization status: kubectl get kustomization -A. Describe for details: kubectl describe kustomization <name> -n flux-system. Test kustomize build locally: kustomize build <path>. Check flux logs: kubectl logs -n flux-system <kustomize-controller>. Use dependsOn in the Kustomization spec to order dependent resources. Fix kustomize patches and validate with kubectl apply --dry-run. Use flux reconcile kustomization <name> to manually trigger.

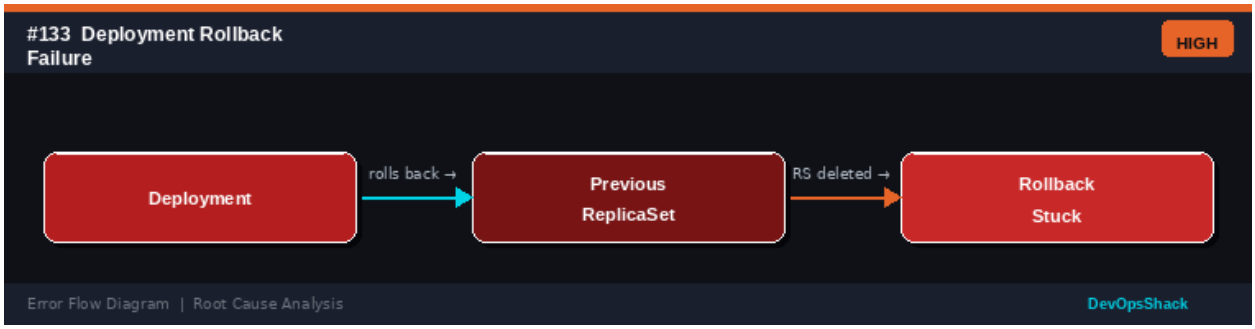
Error Flow Diagram



#133 Deployment Rollback Failure *Workloads* HIGH

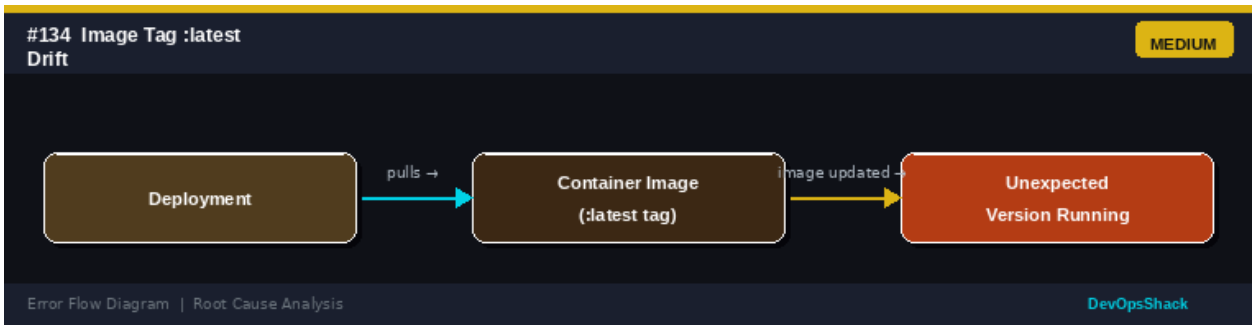
DESCRIPTION	Rolling back a Deployment fails because the previous ReplicaSet configuration is no longer available or compatible.
ROOT CAUSE	Kubernetes keeps a configurable number of old ReplicaSets for rollback (revisionHistoryLimit, default 10). Rollback fails when: the target revision's ReplicaSet was deleted, the image from the previous version is no longer available in the registry, dependent resources (ConfigMaps, Secrets) from the previous version were deleted, and revisionHistoryLimit was set too low.
SOLUTION	Check rollback history: kubectl rollout history deployment/<name>. View specific revision: kubectl rollout history deployment/<name> --revision=<n>. If previous ReplicaSet is available, rollback: kubectl rollout undo deployment/<name> --to-revision=<n>. If not available, manually set the deployment spec to the previous working configuration. Implement a GitOps approach to always have previous configurations in version control. Increase revisionHistoryLimit.

Error Flow Diagram



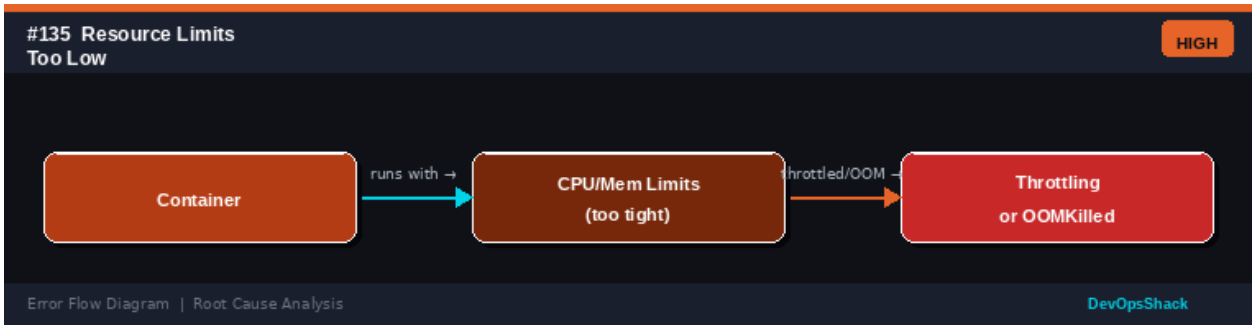
#134	Image Tag :latest Drift *Workloads*	MEDIUM
DESCRIPTION	A Deployment is running an unexpected version of the application because the :latest tag on the container image was updated in the registry, and pods were restarted pulling the new image without a Deployment update.	
ROOT CAUSE	Using :latest tag means the image reference does not uniquely identify a specific build. When pods are restarted (eviction, rolling restart, node replacement), they pull the new :latest image even though the Deployment spec has not changed. This causes inconsistent versions across pods and makes rollback difficult.	
SOLUTION	Use immutable image tags (commit SHA, semantic version): image: myapp:v1.2.3 or image: myapp:sha-abc123. Set imagePullPolicy: IfNotPresent (or Never for truly locked images) to avoid pulling new images on pod restart. Use image digests for absolute immutability: image: myapp@sha256:<hash>. Implement CI/CD that updates the Deployment spec image tag on each build. Never use :latest in production.	

Error Flow Diagram



#135	Resource Limits Too Low *Workloads*	HIGH
DESCRIPTION	Containers are being throttled (for CPU) or OOMKilled (for memory) because resource limits are set too restrictively for the actual workload requirements.	
ROOT CAUSE	Resource limits cap what a container can use. CPU throttling occurs when a container tries to use more CPU than its limit - the Linux CFS scheduler throttles it, causing latency. Memory OOM kills occur when the container's RSS exceeds the memory limit. Both cause performance degradation and instability.	
SOLUTION	Monitor actual resource usage: kubectl top pods --containers. Use VPA in recommendation mode to get right-sized suggestions. Set limits higher than P99 observed usage plus a safety buffer. Use Prometheus/Grafana to track container_cpu_throttled_seconds_total metric. For JVM: set -Xmx to 75% of memory limit. Set CPU limit higher or remove it (use requests only) to allow bursting. Implement proper load testing before setting production limits.	

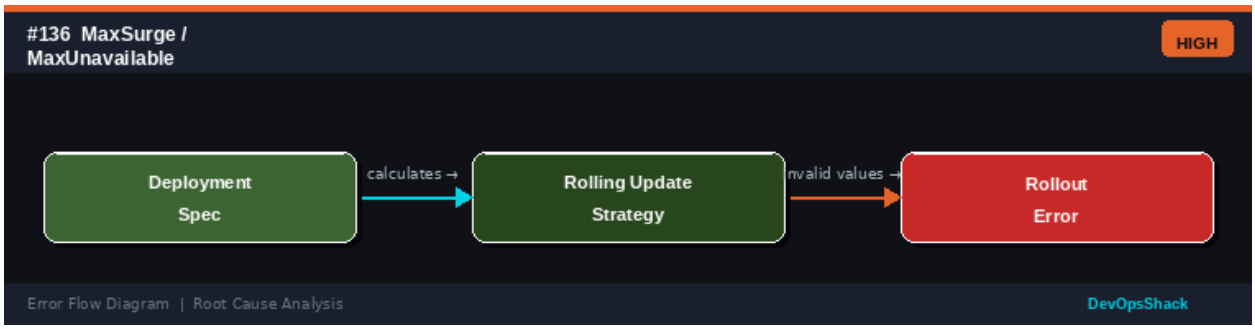
Error Flow Diagram



#136	MaxSurge/MaxUnavailable Error *Workloads*	HIGH
DESCRIPTION	A Deployment rolling update fails or causes unexpected downtime because MaxSurge or MaxUnavailable values are set to 0 simultaneously, deadlocking the rollout.	
ROOT CAUSE	Deployment rolling strategy uses MaxSurge (extra pods above desired count) and MaxUnavailable (pods that can be unavailable). Setting both to 0 is invalid and prevented. However, setting MaxSurge: 0 with MaxUnavailable: 0% percentage-wise can round to 0/0. Also, setting MaxUnavailable too high can cause complete service downtime during rollouts.	

SOLUTION	Check rollout strategy: kubectl get deployment <name> -o yaml | grep -A4 rollingUpdate. Valid configurations: at least one of MaxSurge or MaxUnavailable must be non-zero. Recommended: MaxSurge: 1, MaxUnavailable: 0 for zero-downtime rolling updates. For faster rollouts with tolerance: MaxSurge: 25%, MaxUnavailable: 25%. Recalculate percentage values to ensure they don't round to 0 for small replica counts.

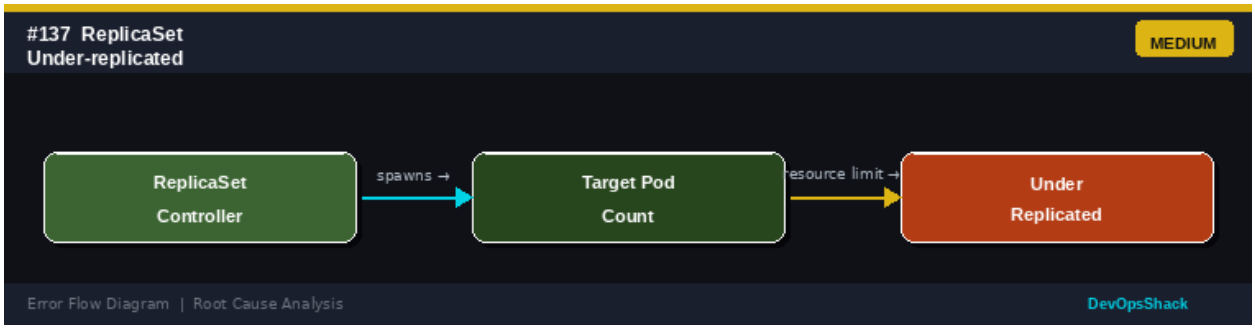
Error Flow Diagram



#137 ReplicaSet Under-replicated *Workloads* MEDIUM

DESCRIPTION	A ReplicaSet has fewer running pods than its desired replica count. The cluster appears to have capacity but new pods are not starting.
ROOT CAUSE	Under-replication occurs when: resource quotas in the namespace are exhausted, pods keep crashing (CrashLoopBackOff) so the ReplicaSet cannot maintain count, scheduling fails for all nodes, and pod disruption budget prevents replacement of terminated pods.
SOLUTION	Check ReplicaSet status: kubectl describe rs <name>. Check pod status in the ReplicaSet: kubectl get pods -l <rs-selector>. If pods are crashing: fix the application. If scheduling fails: check node resources and taints. If quota is exhausted: delete old resources or request quota increase. Check events for specific failures. For persistent under-replication, enable ReplicaSet controller monitoring and alerting.

Error Flow Diagram

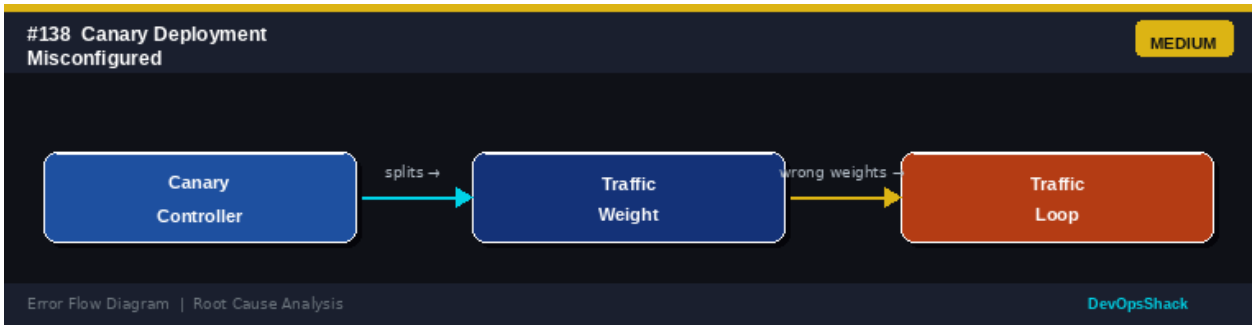


#138 Canary Deployment Misconfigured *Workloads* MEDIUM

DESCRIPTION	A canary deployment is sending incorrect traffic percentages to the canary version, or all traffic is going to canary instead of being split.

ROOT CAUSE	Canary deployments use Ingress annotations, Service Mesh traffic rules, or specialized controllers (Argo Rollouts, Flagger) to split traffic. Misconfigurations occur when: Ingress weight annotations don't add up to 100%, both services have the same labels (Service selects all pods), and the canary controller is not installed.
SOLUTION	Check Ingress canary annotations: kubectl get ingress -o yaml. Ensure canary ingress has nginx.ingress.kubernetes.io/canary: 'true' and canary-weight set correctly. Verify both stable and canary services select only their respective pods using distinct labels. For Argo Rollouts: kubectl argo rollouts get rollout <name>. For Flagger: kubectl describe canary <name>. Test traffic split by checking access logs on both versions.

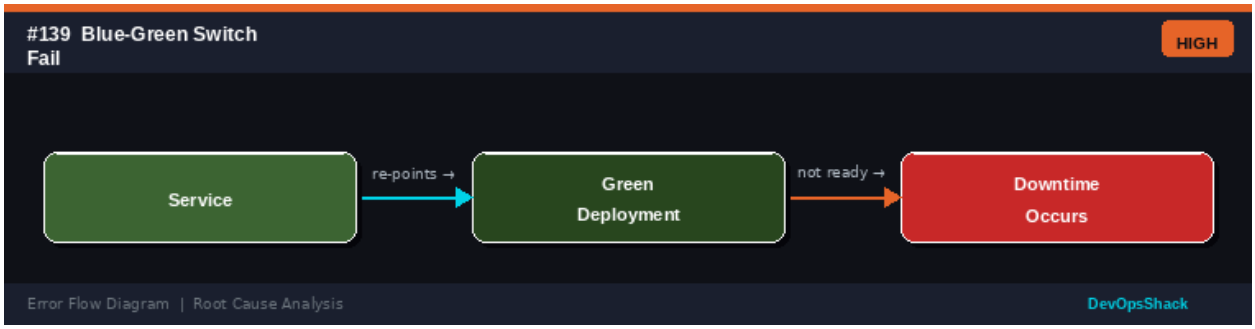
Error Flow Diagram



#139 Blue-Green Switch Failure *Workloads* — HIGH

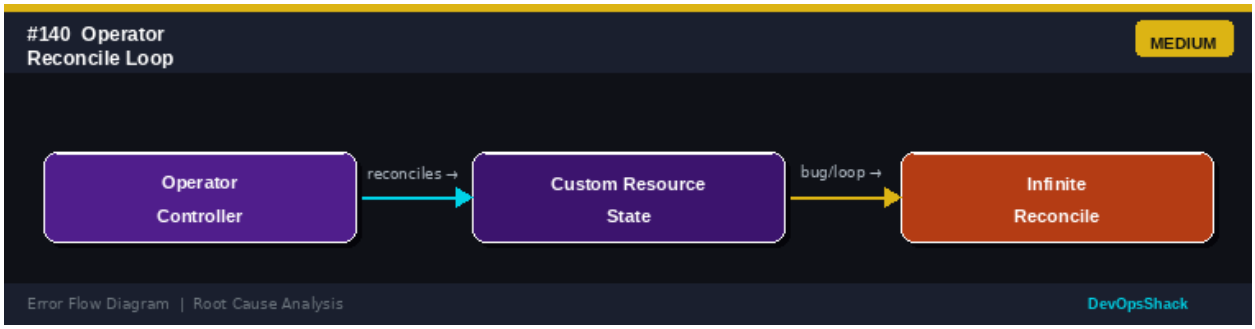
DESCRIPTION	A blue-green deployment switch failed to complete correctly, causing downtime or leaving the wrong version serving traffic.
ROOT CAUSE	Blue-green deployments involve running two identical environments and switching traffic between them. Switch failures occur when: the green deployment is not fully ready before switching, the Service selector was updated but green pods failed readiness, the switch happened during an in-flight request, and rollback re-points to the old blue environment which is already torn down.
SOLUTION	Ensure green deployment is fully ready before switching: kubectl rollout status deployment/green. Use a pre-switch health check. Update Service selector atomically: kubectl patch svc <name> -p '{"spec":{"selector":{"version":"green"}}}'. Keep blue deployment running until green is confirmed stable. Implement a automated rollback trigger based on error rate. Use Argo Rollouts with blueGreen strategy for automated management.

Error Flow Diagram



#140	Operator Reconcile Loop *Workloads*	MEDIUM
DESCRIPTION	A Kubernetes Operator is stuck in an infinite reconciliation loop, consuming excessive CPU and API request quota without making progress.	
ROOT CAUSE	Operators watch for changes and reconcile desired state. A loop occurs when: the reconcile function always returns a requeue request without detecting completion, the operator's action triggers another change that re-triggers reconciliation, status update fails and causes re-trigger, and the condition for stopping reconciliation is never met.	
SOLUTION	Check operator logs for repeated reconcile messages: kubectl logs <operator-pod> | grep reconcil. Add proper requeue logic with backoff: return ctrl.Result{RequeueAfter: 5 * time.Minute}, nil. Use status subresource to avoid triggering reconciles on status updates. Implement proper idempotency checks. Use predicates to filter irrelevant events. Monitor API request rate from the operator pod.	

Error Flow Diagram



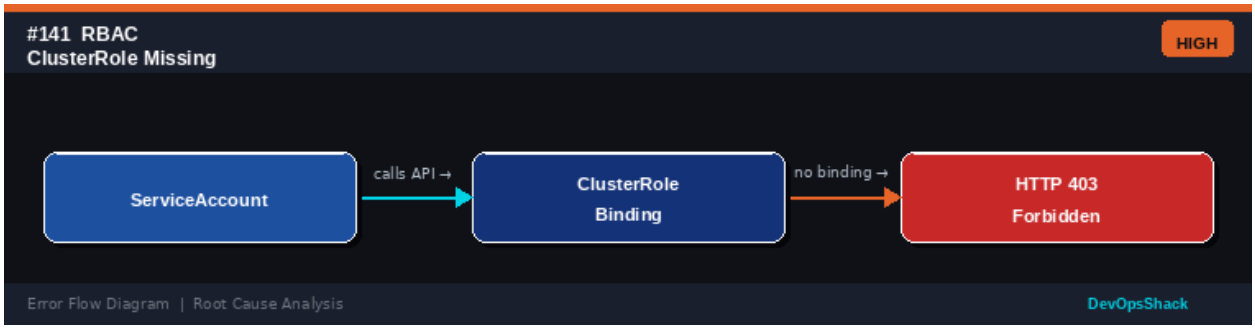
Security Errors

Errors #141 – #150

Security errors in Kubernetes involve RBAC, admission policies, image security, network isolation, and secret management. These errors are critical not just for operations but for maintaining the security posture of your cluster.

#141	RBAC ClusterRole Missing *Security*	HIGH
DESCRIPTION	A ServiceAccount used by an application receives 403 Forbidden because the ClusterRole providing necessary cluster-wide permissions is not bound to it.	
ROOT CAUSE	ClusterRoles provide cluster-scoped permissions. Applications needing to watch resources across all namespaces (operators, monitoring agents) require ClusterRoles. The role may be missing because: it was never created, it was accidentally deleted, the Helm chart or operator did not include RBAC resources, and the binding references a non-existent role.	
SOLUTION	List ClusterRoles and bindings: kubectl get clusterroles,clusterrolebindings | grep <name>. Create and bind a minimal ClusterRole: kubectl create clusterrole <name> --verb=get,list,watch --resource=pods,namespaces followed by kubectl create clusterrolebinding <name> --clusterrole=<name> --serviceaccount=<ns>:<sa>. Use kubectl auth can-i to verify: kubectl auth can-i list pods --as=system:serviceaccount:<ns>:<sa> -A.	

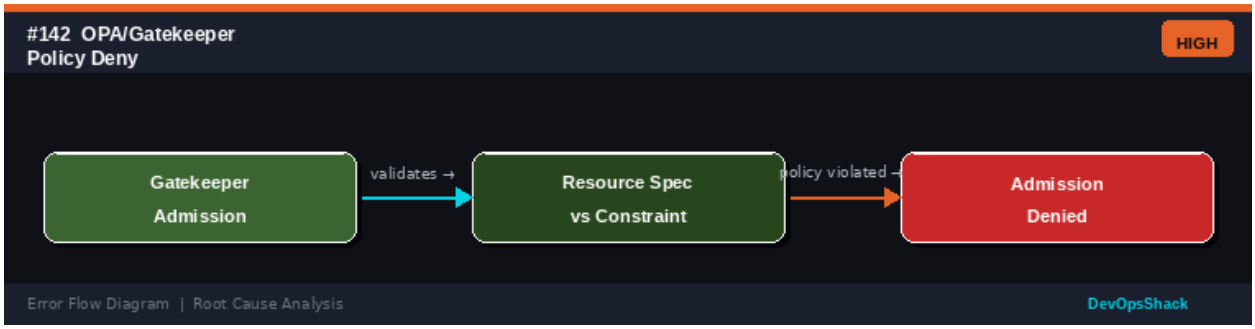
Error Flow Diagram



#142	OPA/Gatekeeper Policy Denial *Security*	HIGH
DESCRIPTION	A resource creation or update is blocked by an OPA Gatekeeper constraint policy. The admission controller rejects the request with a policy violation message.	
ROOT CAUSE	Gatekeeper implements Open Policy Agent constraints as Kubernetes admission webhooks. Common constraint violations: missing required labels (e.g., team, env), disallowed container registries (only internal registry allowed), CPU/memory limits not set, containers running as root, and exposed NodePorts.	
SOLUTION	Read the rejection message for the specific constraint. Check constraints: kubectl get constraints -A. View constraint template: kubectl get constrainttemplate	

	<name> -o yaml. Update your manifest to comply with the policy. For legitimate exceptions, add the resource to the constraint's excludedNamespaces or create an exception annotation. To temporarily disable a constraint: kubectl patch constraint <name> -p '{"spec":{"enforcementAction":"warn"}}' (changes to warning mode). Fix the root cause to comply with policy.

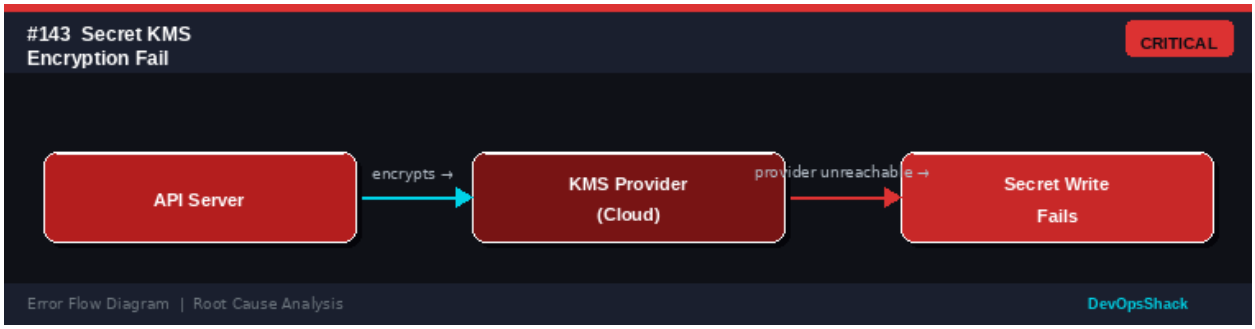
Error Flow Diagram



#143 Secret KMS Encryption Failure *Security* — CRITICAL

DESCRIPTION	Creating or reading Kubernetes Secrets fails because the KMS provider used for envelope encryption is unreachable or misconfigured.
ROOT CAUSE	Kubernetes can encrypt Secrets at rest using KMS providers (AWS KMS, GCP Cloud KMS, HashiCorp Vault). If the KMS provider is unavailable, the API server cannot encrypt new Secrets or decrypt existing ones (depending on the encryption config). Causes: KMS service outage, IAM permission revoked, network connectivity to KMS endpoint lost, and key deleted or disabled.
SOLUTION	Check API server logs for KMS errors: kubectl logs -n kube-system kube-apiserver-<node> | grep kms. Verify KMS key is active in the cloud provider console. Check IAM permissions for the API server to use the KMS key. Check network connectivity from control plane to KMS endpoint. For recovery, temporarily disable KMS encryption in encryption config, restart API server, then re-enable after fixing. Always maintain backup of encryption config.

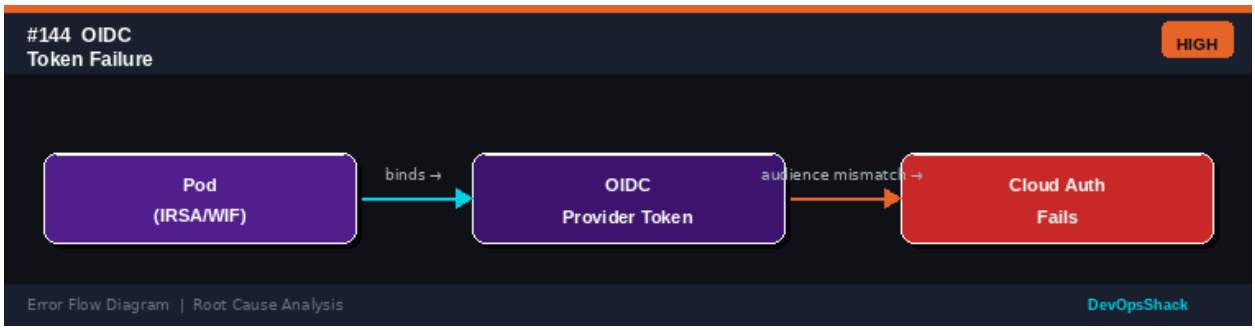
Error Flow Diagram



#144 OIDC Token Authentication Failure *Security* — HIGH

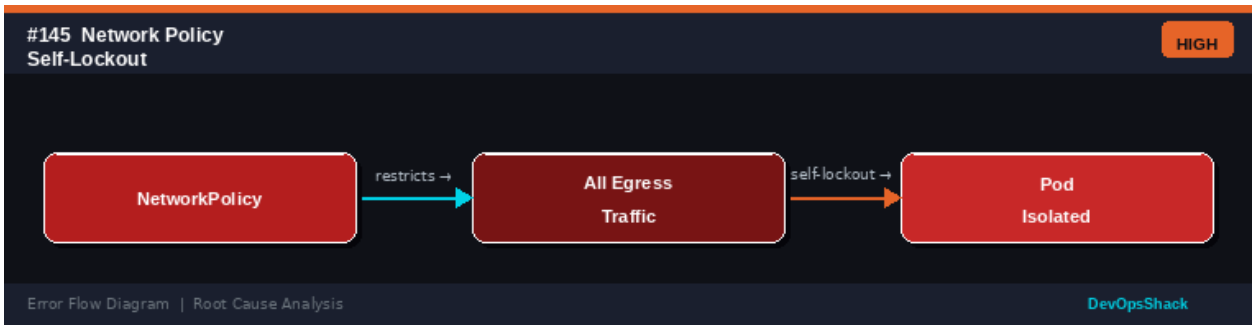
DESCRIPTION	A workload using cloud IAM federation (AWS IRSA, GCP Workload Identity) cannot authenticate to cloud APIs because the OIDC token is invalid or the binding is misconfigured.
ROOT CAUSE	Cloud IAM federation works by exchanging Kubernetes ServiceAccount tokens for cloud credentials via OIDC. Failures occur when: ServiceAccount is not annotated with the cloud IAM role ARN/SA, the OIDC provider URL in IAM does not match the cluster's issuer URL, token audience does not match the cloud provider's expected audience, and the OIDC provider certificate is not trusted.
SOLUTION	For AWS IRSA: check SA annotation: kubectl get sa <name> -o yaml | grep role-arn. Verify IAM role trust policy has the correct OIDC provider and ServiceAccount condition. For GCP Workload Identity: verify SA annotation iam.gke.io/gcp-service-account and IAM binding. Test from pod: kubectl exec -- curl http://169.254.169.254/latest/meta-data/ (AWS) or curl -H 'Metadata-Flavor: Google' http://metadata.google.internal/ (GCP). Verify using cloud CLI.

Error Flow Diagram



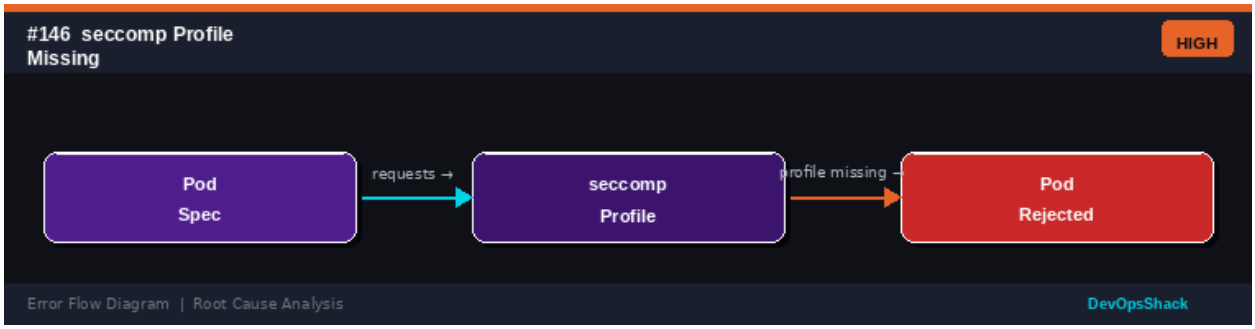
#145	NetworkPolicy Self-Lockout *Security*	HIGH
DESCRIPTION	After applying a NetworkPolicy, the pod that created it (or you as an operator) is locked out and cannot communicate with the cluster or its own services.	
ROOT CAUSE	An overly broad deny-all egress NetworkPolicy can lock a pod out from essential services: DNS (port 53), the Kubernetes API server, metrics endpoints, and its own service. Operators managing network policies can accidentally apply policies that isolate critical services including the policy controller itself.	
SOLUTION	If the policy controller is isolated: access it directly via the API server. Delete the offending policy: kubectl delete networkpolicy <name> -n <namespace>. Always allow DNS before applying egress restrictions: allow egress to kube-system on port 53. Test policies in a non-production namespace first. Use kubectl exec to test connectivity before applying policies broadly. Keep a cluster-admin kubeconfig that bypasses network policies (control plane access).	

Error Flow Diagram



#146	seccomp Profile Not Found *Security*	HIGH
DESCRIPTION	A pod is rejected because it references a custom seccomp profile that does not exist on the node or is not in the configured profile directory.	
ROOT CAUSE	Seccomp profiles restrict system calls a container can make. Custom profiles are JSON files stored on nodes. Pods referencing localhost/ profiles require the profile file to exist at the path referenced, under /var/lib/kubelet/seccomp/ on each node. Missing profiles cause pod rejection at admission or container start.	
SOLUTION	List available seccomp profiles: ls /var/lib/kubelet/seccomp/ on nodes. Copy profile to all nodes or use a DaemonSet to distribute profiles. Alternatively, use the RuntimeDefault or Unconfined seccomp profile as a fallback: seccompProfile.type: RuntimeDefault. Consider using Security Profiles Operator (SPO) to manage seccomp profiles as Kubernetes objects. Document and version-control all custom profiles.	

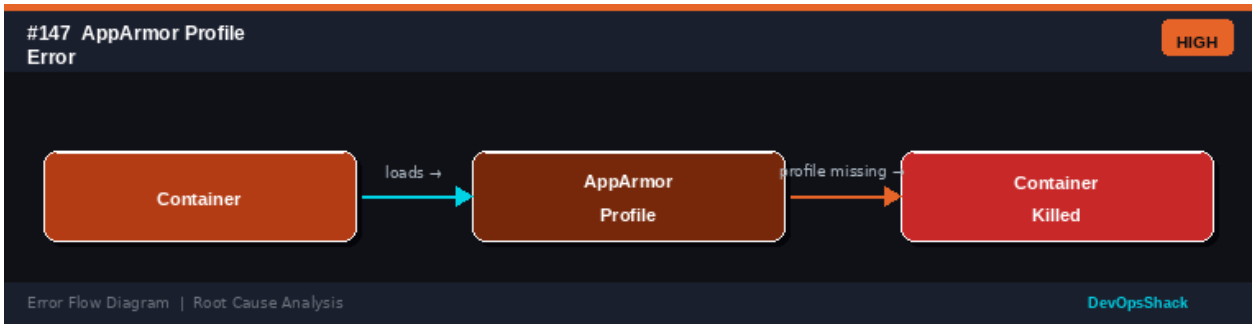
Error Flow Diagram



#147	AppArmor Profile Error *Security*	HIGH
DESCRIPTION	A container is killed or prevented from starting because the AppArmor profile referenced in annotations does not exist on the node.	
ROOT CAUSE	AppArmor profiles are loaded into the Linux kernel and referenced by containers. Pods annotate containers with: container.apparmor.security.beta.kubernetes.io/<container-name>: localhost/<profile-name>. If the profile is not loaded on the node, the container is killed at start. AppArmor must be enabled in the kernel and the profile must be pre-loaded.	

SOLUTION	Check if AppArmor is enabled: aa-status on the node. Load profiles on all nodes: apparmor_parser -r -W /path/to/profile. Use a DaemonSet to load profiles across all nodes on startup. Use container.apparmor.security.beta.kubernetes.io/<name>: runtime/default as a safer default. For Kubernetes 1.30+, use spec.securityContext.appArmorProfile field instead of annotations. Test profile loading before referencing in pods.

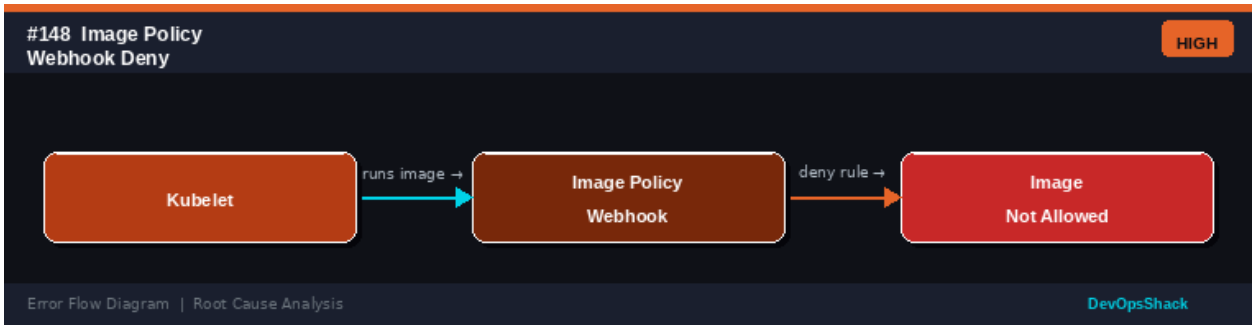
Error Flow Diagram



#148 Image Policy Webhook Denial *Security* — HIGH

DESCRIPTION	A pod cannot start because the image it references is blocked by an image policy webhook that enforces registry allowlists or signature verification.
ROOT CAUSE	Image policy webhooks validate that container images come from allowed registries, have valid signatures (Cosign, Notary), or meet specific naming conventions. Images are blocked when: they come from Docker Hub instead of an internal registry, the image lacks a required signature, the registry is not in the allowlist, and the image tag is not immutable (no digest).
SOLUTION	Check which image policy is blocking: kubectl describe pod shows the admission error message. Review the webhook policy configuration. Update container images to use an approved registry: myregistry.internal/myapp:v1.0.0. Sign images in CI/CD using Cosign: cosign sign --key <key> <image>. For exceptions, add a specific namespace exclusion to the webhook config. Migrate images from blocked registries to internal ones.

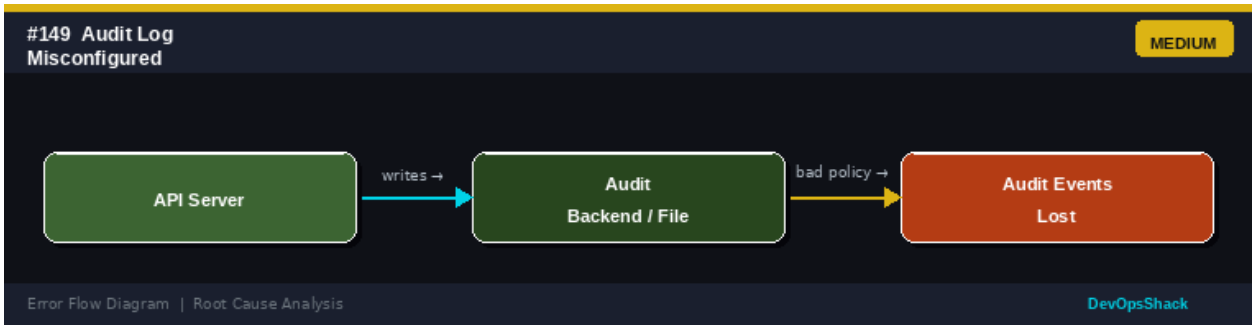
Error Flow Diagram



#149 Audit Log Misconfiguration *Security* — MEDIUM

DESCRIPTION	Kubernetes audit logging is not capturing the expected events, or audit logs are being lost due to a misconfigured audit policy or backend.
ROOT CAUSE	The API server generates audit events based on the audit policy. Misconfigurations include: audit policy file has syntax errors (API server fails to start), log backend path is not writable, webhook backend is unreachable, audit level is set too low (None or Metadata when Request or RequestResponse is needed), and log rotation not configured (disk fills up).
SOLUTION	Check API server startup for audit errors: journalctl -u kubelet | grep audit. Validate audit policy YAML syntax. Ensure the audit log directory is writable by the API server process. Set appropriate audit levels: None (skip), Metadata (headers), Request (body), RequestResponse (full). Configure log rotation: --audit-log-maxage=30, --audit-log-maxbackup=10, --audit-log-maxsize=100. Test webhook backend connectivity from control plane.

Error Flow Diagram



#150	Container Image CVE Critical *Security*	HIGH
DESCRIPTION	A CI/CD pipeline scan or runtime security tool (Trivy, Snyk, Falco) has detected critical CVEs in a container image, blocking deployment or triggering alerts.	
ROOT CAUSE	Container images regularly accumulate vulnerability as base images and dependencies age. Critical CVEs indicate a vulnerability that can be exploited for remote code execution, privilege escalation, or data exfiltration. The image should not be deployed until the vulnerability is remediated.	
SOLUTION	Scan images in CI: trivy image <image-name>. View CVE details including affected package and fix version. Update the base image: FROM ubuntu:22.04 instead of an older version. Update affected packages: apt-get upgrade. Update dependencies to patched versions. Use minimal base images (distroless, scratch) to reduce attack surface. Rebuild and rescan. Implement automated dependency updates (Dependabot, Renovate). Set up admission webhooks to block images with critical CVEs.	

Error Flow Diagram